

Agent × Robot: 从 Harness 到自进化物理智能体

五份源材料的解读 · 25 条主线综述 · 六大趋势与十条洞察 · 18 个研究机会 · 数字口径账本 · 43 份深度解读合订

仓库: github.com/asimfish/awesome_agentic_robot

维护: asimfish · 生成日期: 2026-09-08

本报告由 `scripts/build_full_report.py` 从 `insights/`、`docs/reports/report_zh.md` 与 `notes/` 自动合订; 引用请注明仓库与解读编号。

目录

Part 0 · 执行摘要：趋势、洞察、研究机会与口径账本

1. 趋势与洞察：Agent × Robot 从 Harness 到自进化物理智能体（2026 年 9 月）
2. 研究机会清单：18 个待验证问题，每项配一个起步实验
3. 数字口径账本：本仓库引用的头条数字，逐条标注它们到底在测什么

Part 1 · 五份源材料的解读与 25 条主线综述

1. 导读
2. 1. 材料、方法与边界
3. 2. 解读一：《Harness 之后，Agent+Robot 下一站是什么？》
4. 3. 解读二：Code-as-Policy → 自进化机器人 Agent（讲稿）
5. 4. 解读三：检索地图的 23 条主线（2026 年 9 月状态）
6. 5. 解读四：Lilian Weng 的两篇文章
7. 6. 解读五：具身纪元《GPT-6 未必能当好机器人的大脑，却可能帮王兴兴加速 RobotRSI》

Part A · 主线：Coding Agent 与自进化

1. 01 · Code as Policies 深度解读：让 LLM 写的程序成为机器人策略
2. 02 · CaP-X 深度解读：把 Code-as-Policy 做成可以被系统研究的平台
3. 03 · RHO 深度解读：训练时搜索策略仓库，部署时单轮执行
4. 04 · ASPIRE 深度解读：失败修复沉淀成技能，技能库越大适应越快
5. 05 · SkillOpt 深度解读：把技能文档当作冻结 Agent 的可训练外部状态
6. 06 · ENPIRE 深度解读：把真机学习变成 coding agent 可以管理的优化过程
7. 07 · 自进化续作合评：Zetta、SHAPER、PRACTICE、SkillGLoW、HiSME、MEMENTO
8. 08 · 自动研究谱系：Eureka → DrEureka → HARBOR / Nautilus / AgenticRobotics / Push-T 重访
9. 09 · 技能库谱系：从 LLM 提任务到形式化可验证的技能契约

Part B · 记忆、反思与验证

1. 10 · RoboMME 与 RoboMME-Interference 深度解读：机器人记忆的评测基准与它的两个结论
2. 11 · PonderPounce 深度解读：用 MLLM 的原生因果上下文当机器人记忆
3. 12 · AGM 深度解读：只有物理证据才能推进进度指针
4. 13 · HyMeS 深度解读："技能在权重里，记忆在代码里"
5. 14 · BATON 深度解读：长程问题是技能交接问题
6. 15 · 权重内记忆与记忆选择合评：NativeMEM、LaMem-VLA、Remember Smarter、OnEvoMemory、Memory Anchors、概念中心记忆
7. 16 · 反思与纠错合评：REMAC、PhysReflect-VLA、Agentic RAG-VLM、PhysiAgent

8. 17 · PRIMO R1 深度解读：从"观察者"到"批评者"的过程级视频验证
9. 18 · VERITAS 深度解读：推理时视觉验证，验证过的 rollout 直接成为训练数据
10. 19 · 验证器谱系：LLM-as-a-Verifier、立场论文、Consilience、Agentic Harnesses

Part C · Harness、Runtime、双系统与端侧

1. 20 · Harness VLA 深度解读：不改权重、不扩技能库，学冻结 VLA 的"使用说明书"
2. 21 · PhyAgentOS 深度解读：把 Harness 做成操作系统
3. 22 · Thea 深度解读：物理世界不白送的两样东西——读状态、判结果
4. 23 · Harness Engineering for Physical AI 深度解读：机器人中间件就是 harness 层
5. 24 · Harness 框架合评：RoboHarness、Guava、Cortex 与 Agentic Harnesses
6. 25 · 治理与运行时合评：Runtime Governance、EmbodiedGovBench、ICAN-Deploy、FSAR
7. 26 · 快慢双系统合评：Fast-in-Slow、OneTwoVLA、StreamVLA、LaST0、RationalVLA 与 2026 年的频率分层
8. 27 · 端侧部署合评：PhyAI、EcoVLA、CloudEdgeVLA、ARLI、ST-Merge、Habilis-β

Part D · 学习闭环：RL、数据回流、数字孪生与世界模型

1. 28 · LWD 深度解读：16 台机器人的 fleet-scale 离线到在线 RL
2. 29 · Q-Planning 深度解读：只微调一个小 Q 函数，就能从部署失败里学
3. 30 · TwinRL 与数字孪生谱系：Sim 是 sandbox，不是训练场
4. 31 · RoboClaw 深度解读：让机器人同时学会"做任务"与"恢复现场"
5. 32 · VLA + RL 合评：信用分配、免外部奖励与测试时 RL
6. 33 · 世界模型的角色合评：H-WM、Motus2、ContactGuard、SafeDojo、Online Continual RL、RoboGene

Part E · 总览、接口、多机器人与跨本体

1. 34 · AgenticLab / PPlanAR 深度解读：用规划语言定义 VLM 的推理空间
2. 35 · Agent + Robot 总览合评：Agentic Robot、ManiAgent、VoLo、HoloAgent-0 与"灵活但仍脆弱"
3. 36 · 接口层合评：两篇 ROSClaw、MHS、OpenClaw 生态与 MCP
4. 37 · 多机器人协作合评：RoCo、MALMM、REMAC、RoboOS 系列、DynaHMRC、Scale-Plan、MeCo、Tool-RoCo、对话对齐与通信攻击
5. 38 · 生命周期、跨本体与主动感知合评：Arcadia、PhysCaP、ActiveVLA 与相关工作

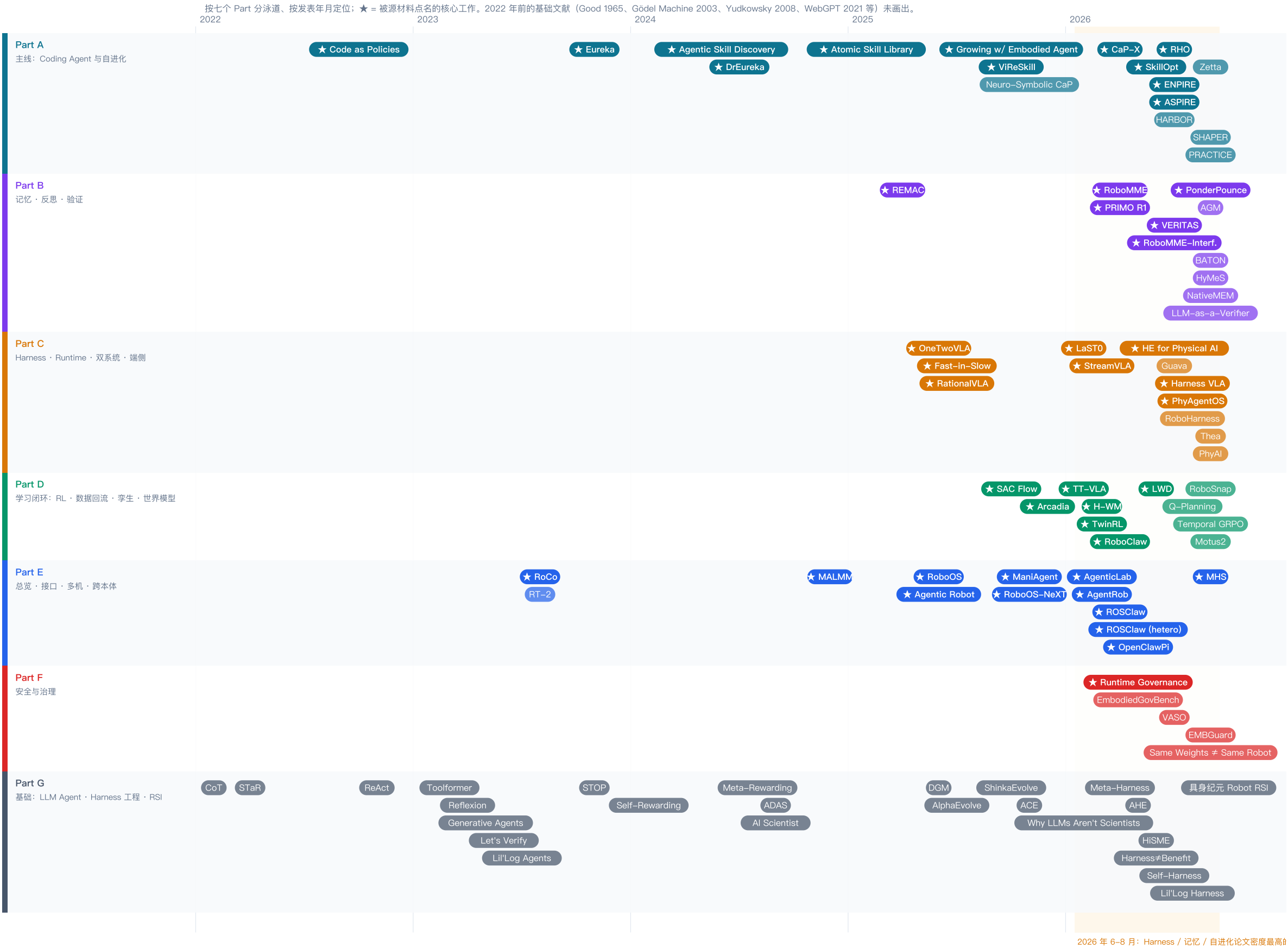
Part F · 安全与治理

1. 39 · 安全合评：EMBGuard、SENTINEL、VASO、Same Weights Different Robot、RationalVLA、通信攻击与 AgentRob

Part G · 基础：LLM Agent、Harness 工程与 RSI

1. 40 · Lil'Log 两篇深度解读:《LLM Powered Autonomous Agents》与《Harness Engineering for Self-Improvement》
2. 41 · RSI 谱系: 从 Good 1965 到 BigBang-V1——软件侧递归自我改进的四个环节
3. 42 · Harness 演化方法合评: ACE、MCE、Meta-Harness、Self-Harness、AHE、Harness Updating ≠ Harness Benefit、DGM、Hyperagents、AlphaEvolve、ShinkaEvolve、ThetaEvolve、GEPA、Promptbreeder、STOP、ADAS、AFlow 与相关工作
4. 43 · AI 研发基准与失败模式合评: PaperBench、RE-Bench、MLE-bench、ScienceAgentBench、CORE-Bench、KernelBench、Early Science Acceleration 与《Why LLMs Aren't Scientists Yet》

图 1 · Agent × Robot 论文时间线 (2022–2026)



2026 年 6–8 月: Harness / 记忆 / 自进化论文密度最高

图 1 · Agent × Robot 论文时间线 (2022–2026): 按七个 Part 分泳道、按发表年月定位, ★ 为源材料点名的核心工作, 橙色竖带为 2026 年 6–8 月。

图 2 · Agent × Robot 分类体系：七个 Part、25 条主线

括号内为该主线的条目数（一篇论文可属多条主线）；右侧为代表工作。与 README 的 25 条主线、合订本的 Part A–G 一一对应。

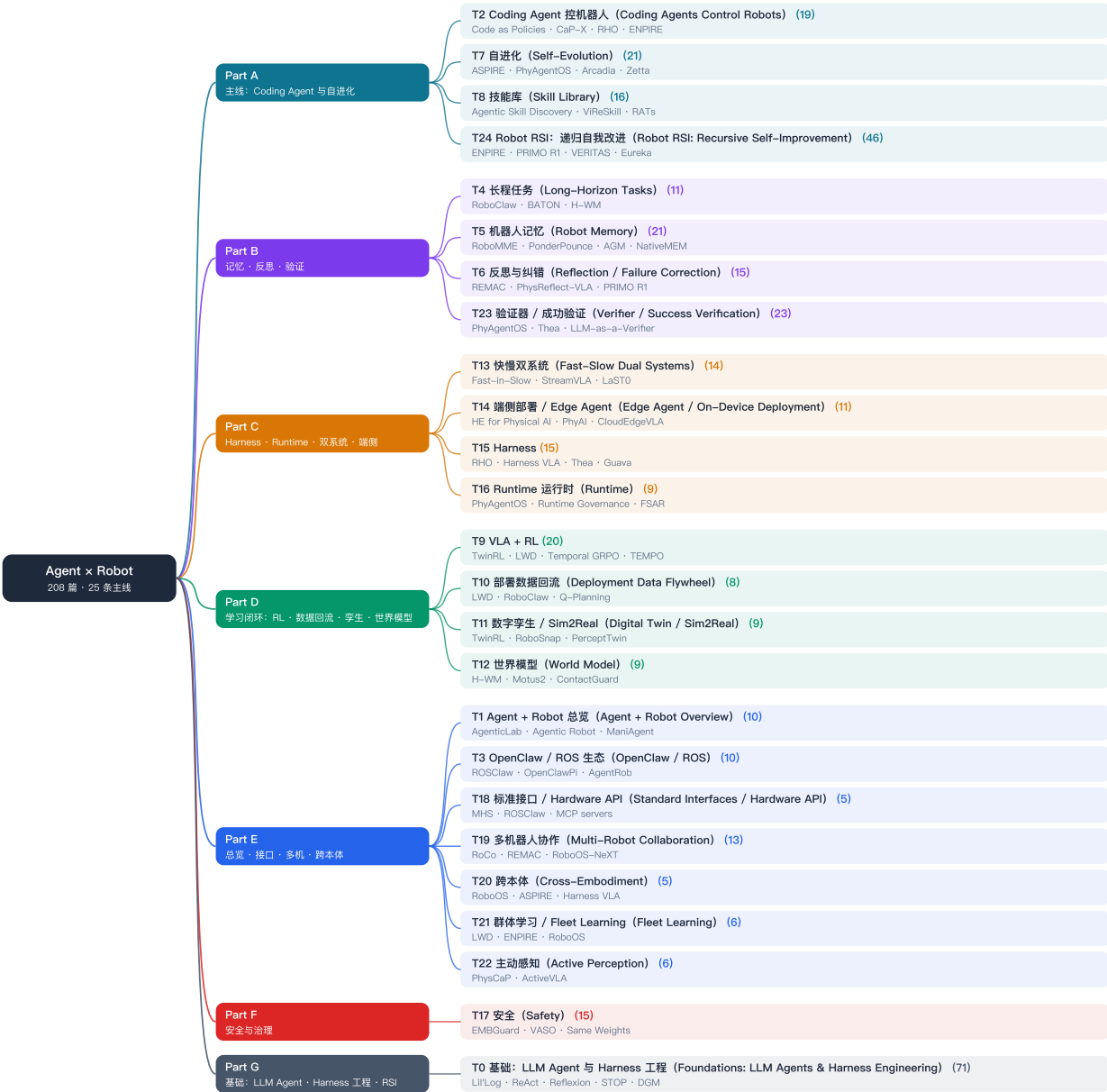


图 2 · 分类体系：七个 Part、25 条主线，括号内为条目数——与 README 与本报告的 Part A–G 一一对应。

PART 0

执行摘要：趋势、洞察、研究机会与口径账本

一页结论 · 六大趋势 · 十条洞察 · 八条可证伪预测 · 18 个研究机会（各配最小可行实验）· 头条数字的口径账本——全部结论先行，细节见后续各章。

1. 趋势与洞察：Agent × Robot 从 Harness 到自进化物理智能体（2026 年 9 月）
2. 研究机会清单：18 个待验证问题，每项配一个起步实验
3. 数字口径账本：本仓库引用的头条数字，逐条标注它们到底在测什么

趋势与洞察：Agent × Robot 从 Harness 到自进化物理智能体 (2026 年 9 月)

本文是仓库的执行摘要：一页结论、领域时间线、六大趋势、十条洞察、可证伪预测。所有数字的口径见 [insights/12_numbers_ledger_zh.md](#)，每条判断对应的深度解读见 [notes/](#)（编号引用为 [notes/NN](#)）。不同工作的成功率禁止直接比大小。

0. 一页结论

1. 优化对象在上移：2022 年 LLM 写一段策略代码 ([notes/01](#))；2026 年 coding agent 优化技能文档 ([notes/05](#))、策略仓库 ([notes/03](#))、技能库 ([notes/04](#))、训练配方 ([notes/06](#))、运行时 critic ([notes/07](#))、整套 harness。讲稿公式 $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$ 是这条线的坐标系。
2. "冻结 VLA + 外围学习"是默认范式，但有天花板：Harness VLA、BATON、AGM、HyMeS、Zetta 不改权重 ([notes/20](#)、[14](#)、[12](#)、[13](#)、[07](#))；LWD 与 Q-Planning 证明真实反馈终要写回某个可训练对象 ([notes/28](#)、[29](#))。
3. 验证器是新瓶颈也是新 scaling 轴：AGM 的证据门控、Thea 的 exit codes、PRIMO R1 的过程批评者、VERITAS 的推理时验证、LLM-as-a-Verifier 的连续评分 ([notes/12](#)、[22](#)、[17](#)、[18](#)、[19](#))。
4. Harness 变成实时系统问题：控制/计算/通信三处介入、Projection/Isolation/Transfer ([notes/23](#))；端侧系统工作给出实证 ([notes/27](#))。
5. 群体是经验规模化的出路，收益亚线性：LWD 16 台 $\rightarrow$ 95% ([notes/28](#))；ENPIRE 8 倍机器人 $\rightarrow$ 2–3 倍加速，多的是假设吞吐量 ([notes/06](#))。
6. Robot RSI 是串起一切的框架：改进环节 $\times$ 人的参与程度两条轴线 ([notes/41](#))，把机器人侧与软件侧放进同一张表。
7. 治理与安全被观点文章低估、被论文高估：Runtime Governance 96.2% 拦截、EmbodiedGovBench 七维 ([notes/25](#))；证据多在仿真 ([notes/39](#))。
8. 长程问题被重述为技能交接问题： $T^K \rightarrow T \cdot K$ ，进入条件缺失 ([notes/14](#))。
9. 记忆分权重内/权重外两路：RoboMME"任务依赖"、RoboMME-Interference"检索可恢复"给出选择规则 ([notes/10](#)、[15](#))。
10. Sim 是 sandbox 不是训练场：建 twin 的成本在下降，twin 的可信度仍取决于接触建模 ([notes/30](#)、[33](#))。

一句话：下一阶段不是 Model Scaling，也不只是 Harness Scaling，而是 **System Scaling + Experience Scaling**——先训练出足够好的机器人让它开始工作，再让工作本身继续训练机器人。

1. 领域快照：通往 2026 年 6–8 月的时间线

- 2022–2023 · 范式起点：Code as Policies (2022–09) 把策略变成 LLM 写的程序 ([notes/01](#))；RoCo (2023–07) 让机器人用 LLM 对话协商 ([notes/37](#))；Eureka (2023–10) 让 LLM 写奖励函数

(notes/08)；软件侧 ReAct、Reflexion、Toolformer、Generative Agents 与 Lil'Log 2023 文确立 Agent = Planning + Memory + Tool use (notes/40)。

- **2024 · 技能与仿真参数的自动化**：Agentic Skill Discovery 让 LLM 从零长出技能库 (notes/09)；DrEureka 让 LLM 写域随机化范围并迁移真机 (notes/08)；MALMM 三 agent 零样本操作 (notes/37)。
- **2025 · 闭环骨架标准化**：Agentic Robot、ManiAgent、PhysiAgent 确立感知 → 分解 → 执行 → 验证 → 重规划 (notes/35、16)；双系统 VLA 三条路线 (Fast-in-Slow、OneTwoVLA、RationalVLA, notes/26)；RoboOS/RoboOS-NeXT 的共享记忆 (notes/37)；Arcadia 的四阶段生命周期 (notes/38)；软件侧 DGM、AlphaEvolve、ShinkaEvolve、ACE 打通 harness 演化 (notes/42)。
- **2026 年 1-5 月 · 记忆、接口、治理登场**：AgenticLab 的规划语言接口 (notes/34)；RoboMME 基准 (notes/10)；StreamVLA、LaST0 解决"何时切换" (notes/26)；TwinRL (notes/30)；RoboClaw 的自复位 (notes/31)；PRIMO R1 (notes/17)；CaP-X 平台 (notes/02)；ROSClaw 的模型无关执行层 (notes/36)；Runtime Governance 与 EmbodiedGovBench (notes/25)；LWD 的 16 台机器人 (notes/28)；SkillOpt 的训练纪律 (notes/05)。
- **2026 年 6-8 月 · 密度最高的季度**：Harness 一词成为中心——RHO、Harness VLA、Harness Engineering for Physical AI、PhyAgentOS、Thea、Guava、HARBOR、Zetta、SHAPER、RoboHarness (notes/03、20-24、07)；ENPIRE 与 ASPIRE 把 coding agent 放进真机学习循环与技能库 (notes/06、04)；记忆论文集中出现——PonderPounce、AGM、HyMeS、BATON、NativeMEM、LaMem、Remember Smarter (notes/11-15)；验证器成为独立主题——VERITAS、LLM-as-a-Verifier、立场论文 (notes/18、19)；VLA+RL 的信用分配 (Temporal GRPO、TEMPO、WCM, notes/32)；端侧系统 (PhyAI、CloudEdgeVLA、ARLI, notes/27)；Anthropic MHS (2026-08-27, notes/36)。
- **2026 年 9 月 · 叙事汇合**：具身纪元文章把这一切命名为 Robot RSI (notes/41)。

2. 六大趋势

趋势一：优化对象沿 Lil'Log 阶梯逐级上移，机器人侧已走完全程

prompt → context (SkillOpt) → workflow (ASPIRE、HARBOR) → harness code (RHO、SHAPER、Zetta) → optimizer code (HiSME、HarnessOpt)。每一级都有 2026 年的机器人侧实例 (notes/40、42)。含义：一篇新方法的贡献越来越取决于它把哪一级的产物变成可搜索、可验证、可复用的对象。

趋势二：冻结基座成为默认，学习发生在外围

Harness VLA、BATON、AGM、HyMeS、Zetta、SHAPER、RoboHarness、AtomBridge、RL²-VLA、VERITAS 全部不改 VLA 权重。原因是权重更新贵、慢、验证难，且闭源前沿模型无权重可改。边界由 BATON (VLA 原语没有进入条件) 与小红书长文的 USB 例子划出：接触相关的连续关系外围学不到 (notes/14、20)。

趋势三：验证器从隐含假设变成显式对象，并开始分层

2026 年前，"成没成"由脚本或人判定；2026 年出现了四类显式验证器——高频几何门控 (AGM)、过程级视频批评者 (PRIMO R1)、推理时动作验证器 (VERITAS)、连续评分与准入判断 (LLM-as-a-Verifier、

Agentic Harnesses) ——加上部署前的形式验证 (VASO) 与数字孪生预演 (PerceptTwin)。立场论文提醒：成功率本身无法证明物理推理 (notes/19)。

趋势四：Harness 从 prompt 工程变成中间件与实时系统

Harness Engineering for Physical AI 命名了这一层并提出 Projection/Isolation/Transfer；PhyAI 的 control-time Roofline、CloudEdgeVLA 的 40 步延迟、EcoVLA 的 20 Hz、ARLI 的延迟破坏马尔可夫假设、UniFS/Latent Bridge 的频率分层是同一问题的实证 (notes/23、27、26)。

趋势五：评测口径开始转向"长期运行"

Habilis-β 的 Tasks per Hour × Mean Time Between Intervention、ENPIRE 的 MRU/MTU、AgenticRobotics 的假阳性晋升率、EmbodiedGovBench 的七维治理指标，都在替代"精心复位下的单次成功率" (notes/27、06、08、25)。这是"让机器人长期工作"的必要度量。

趋势六：接口标准化与安全同步升温，但尚未合流

MHS (2026-08-27)、ROSClaw 的模型无关执行层、MCP 机器人 server、ROS 2 Harness Profile 提案让"任何模型接任何机器人"成为可能；AgentRob 的劫持、多机器人通信攻击 97.8%、Same Weights Different Robot 的元数据脆弱性说明开放接口的攻击面同步扩大 (notes/36、39)。

3. 十条核心洞察

- I1 优化对象上移 (趋势一的判断版)：贡献 = 把哪一级产物变成可搜索、可验证、可复用的对象。
- I2 冻结 VLA + 外围学习有天花板：HyMeS 的"技能在权重里、记忆在代码里"是可操作的分工假设；Q-Planning 的"技能在大权重里、价值在小权重里"是第三条路 (notes/13、29)。
- I3 验证器是瓶颈也是 **scaling 轴**：一个更好的验证器比一个更好的规划器更稀缺，也更容易成为独立贡献；Goodhart 风险要求评估器在可编辑面之外 (notes/19、42)。
- I4 记忆两条路各有适用区：精确时序、计数、低延迟用权重内 (NativeMEM 32.4 → 84.0%)；跨会话、抗干扰、可解释、可编辑用结构化外部记忆 + 检索 (RoboMME-Interference) (notes/10、15)。
- I5 Harness 是实时系统问题："harness 像 OS"在机器人侧是字面意义 (notes/23、21)。
- I6 Sim 是 **sandbox**：TwinRL、RoboSnap、Agentic Real2Sim、PerceptTwin、SafeDojo 让新技能先在物理仿真里跑；缺的是串起来的 harness (notes/30)。
- I7 治理与安全成为一等公民：Runtime Governance、EmbodiedGovBench、ICAN-Deploy、EMBGuard、SENTINEL、Same Weights (notes/25、39)；两篇观点文章都没写它。
- I8 群体收益亚线性：价值在假设与场景的多样性，不在同一策略的 rollout 数；"多样性塌缩"以多个 agent 试同一想法出现 (notes/06、28)。
- I9 Coding agent 成为 **roboticist**：Eureka → DrEureka → HARBOR → ENPIRE → AgenticRobotics；物理世界加三个约束——读状态难、判结果难、有时间 (notes/08、22、23)。
- I10 风险与反例：harness updating ≠ harness benefit；验证器进了可编辑面 (Zetta、MEMENTO、Motus2)；对话降低多机器人任务成功；共享经验会共享污染 (notes/42、07、33、37)。

4. 可证伪的预测（12–24 个月）

写成可以被证明错的形式；到期后应回来逐条核对。

- **P1（验证器分层）**：到 2027 年中，至少一篇机器人论文会同时报告高频门控验证器与低频 LLM/视频验证器的组合，并给出两层各自的假阳性/假阴性率。若届时验证器仍只作为单一模块出现，P1 错。
- **P2（进入条件）**：到 2027 年中，至少一个开源技能库（ASPIRE、RATs、Harness VLA 类）会把"进入条件"作为技能的显式字段（几何分布或谓词），而不是隐含在 MOVE_TO 预定位里。
- **P3（harness 标准）**：到 2027 年底，会出现一个把 Projection/Isolation/Transfer 之一做成 ROS 2 profile 或 MHS 扩展的公开实现；若 MHS 与 ROS 2 Harness Profile 仍是两条互不引用的线，P3 半错。
- **P4（群体加速比）**：ENPIRE 式多 agent 真机研究若加入假设去重（新颖性拒绝采样或过程族去重），8 机器人加速比会从 2–3 倍提高到 ≥ 4 倍；若有人做了去重仍停在 3 倍以下，P4 错——瓶颈不在重复而在等待。
- **P5（只读评估器）**：到 2027 年中，至少一篇自进化机器人论文会明确把评估器/验证器设为不可演化并报告"评估器只读 vs 可演化"的对照；若所有自进化工作继续演化自己的 critic 而无对照，P5 错。
- **P6（Experience Scaling 度量）**：TPH $\times$ MTBI 或 MRU/MTU 类连续运行指标会出现在至少三篇非原作者的论文中，成为通用口径；否则"Experience Scaling"仍是口号。
- **P7（治理进真机）**：EmbodiedGovBench 式的七维评测会在 ≥ 8 台机器人的真机集群上被报告一次（LWD 或 ENPIRE 规模）；若治理评测继续停留在仿真，P7 错。
- **P8（frontier LLM 的角色）**：具身纪元文章的判断——前沿 LLM 先成为认知中枢而非末端控制器——在 2027 年中仍成立：不会有主流工作用 GPT-6 级模型以 ≥ 10 Hz 直接输出连续控制并在接触密集任务上超过专用 VLA。若出现，P8 错，本仓库 I2 需要重写。

5. 开放问题清单

见 `insights/11_open_problems_zh.md`：18 个问题，按 Part A–G 排列，每条配缺口、为什么重要、最小可行实验与相关解读。只做一件事就做第 1 条（分层验证器）。

研究机会清单：18 个待验证问题，每项配一个起步实验

本清单把 43 份解读里散落的"未走之路"与"延伸批判"统一归档，按合订本的 Part A–G 排列。入选标准：边际信息量高于"再发一个新机制"，且原则上可用现有开源资产（LIBERO-Pro、RoboCasa、RoboMME/RoboMemArena、CaP-Gym、Harness VLA/ASPIRE/RATs 的技能库、开源 VLA 如 $\pi 0.5$ /OpenVLA/GR00T）在中等预算内起步。"无人做过"一律限定为"本次检索未见"。每条给出：缺口 · 为什么重要 · 最小可行实验（MVE）· 相关解读。

Part A · 主线：Coding Agent 与自进化

1. 分层验证器及其误差账本（只做一件事就做这条） 缺口：AGM 的高频几何门控（2.43M 参数）、PRIMO R1 的过程级视频批评者（RoboFail 67%）、LLM-as-a-Verifier 的连续评分（RoboRewardBench 87.4%）、VASO 的形式检查各自独立，没有人把它们组合并报告每层的假阳性/假阴性率。为什么重要：验证器是自进化循环的上限（I3）；不知道每层错在哪里，就无法设计选择门的容错。MVE：在 RoboMemArena 或 LIBERO-Pro 上固定一个冻结 VLA + agent 配方，分别用四种验证器（及其两两组合）充当"成没成"的判断，用脚本真值算每种验证器的混淆矩阵，再看自进化（例如 ASPIRE 式技能沉淀）在噪声验证下的收益曲线。相关：notes/12、17、18、19、09。

2. 把 SkillOpt 的训练纪律搬到机器人技能库 缺口：SkillOpt 的四条纪律（有界编辑、held-out 严格提升门、拒绝编辑缓冲、epoch 慢更新）只在便宜验证器的软件任务上验证；ASPIRE、RATs、Harness VLA 的技能库更新没有 held-out 门。为什么重要：没有选择门的技能库会积累"看起来聪明但没有改进"的技能（RSI 定义里的"变化不是改进"）。MVE：用 RATs 或 ASPIRE 的开源技能库，把技能更新改为 SkillOpt 式流程，held-out 用 LIBERO-Pro 的一半任务；对照无门控更新，报告库规模 vs 零样本成功率曲线是否更陡。相关：notes/05、04、09、07。

3. 代码策略与 Harness VLA 的同基座组合 缺口：RHO（代码策略仓库，LIBERO-PRO 45.0%）与 Harness VLA（VLA 调用规则，LIBERO-Pro +38.6pp）从未在同一原语与同一 VLA 上组合。为什么重要：ENPIRE 的混合方案（代码管几何阶段、VLA 管接触阶段）说明两者互补，但只有一个数据点。MVE：同一 $\pi 0.5$ + 同一原语集，三个条件——纯 RHO、纯 Harness VLA、RHO 管交接 + Harness VLA 管调用——LIBERO-Pro 上并排。相关：notes/03、20、06。

4. 群体假设去重 缺口：ENPIRE 8 倍机器人只有 2–3 倍加速，原因是 agent 重复探索相似假设；ShinkaEvolve 的新颖性拒绝采样与 SkillGLOW 的过程族去重是现成工具，没人接到多 agent 真机研究上。为什么重要：决定 fleet autoresearch 的经济性；也是 Lil'Log"多样性塌缩"在机器人侧的形态。MVE：先在 Gym-PushT（仿真）复现 ENPIRE 的 1/4/8 agent 曲线，再加假设去重（对提出的实验配置做嵌入去重），看加速比变化；预测 P4 由此检验。相关：notes/06、42、07。

Part B · 记忆、反思与验证

5. 权重内 vs 权重外记忆的同基准对照 缺口：NativeMEM（32.4 → 84.0%）、Remember Smarter（LIBERO-Plus 53.6 → 70.6%）与 AGM、HyMeS（RoboMemArena 41.3 → 60.1%）在不同基准、不同协议上报数。为什么重要：I4 的选择规则（时序/计数用权重内，跨会话/可编辑用权重外）目前是推断，不是实验结论。MVE：在 RoboMME 的四类记忆任务上，同一 $\pi 0.5$ 基座，加权重内记忆（NativeMEM 式）与加外

部记忆 (AGM 式) 各一版, 再在 RoboMME-Interference 的干扰会话下测衰减。相关: notes/10、11、12、13、15。

6. 技能的显式进入条件 缺口: BATON 指出 VLA 原语只有退出条件; RoboHarness 的 Memory Bridge、Harness VLA 的 MOVE_TO 预定位、RoboClaw 的 EAP 逆向技能终态都是隐含的进入条件, 没有一个技能库把它作为字段。为什么重要: 长程代价从 T^K 变 $T \cdot K$ 的前提是子任务可独立探索, 而独立性由进入条件保证。MVE: 对一个 VLA 原语 (如"抓取") 在仿真里采样起始位姿分布并标注成败, 拟合一个进入条件分类器; 把它接进 BATON 式 agent 的交接决策, 看长程任务成功率与探索 episode 数。相关: notes/14、24、20、31、09。

7. 反思类方法的"每次成功平均尝试数"口径 缺口: PhysReflect-VLA (+5.4pp)、Agentic RAG-VLM (25 → 78.3%)、Zeva 式 CSR@K 都含重试, 与更贵的 retry 无法区分。为什么重要: 不报告尝试数, 就无法判断"反思"是否真的写进了持久对象。MVE: 对任一开源反思框架, 同时报告单次成功率、CSR@K 与每次成功的平均尝试数, 并加一个"随机重试同样预算"的对照。相关: notes/16、07。

8. 评估器进循环的后果 缺口: Zetta 在线演化 critic、MEMENTO 先演化评估器、Motus2 把评估器做成共享权重的接口, 都与 Lil'Log/AHE 的"评估器只读"冲突; 无人对照。为什么重要: 这是 Robot RSI 的核心风险 (Goodhart), 也是预测 P5 的内容。MVE: 在 CaP-Gym 上跑一个自进化 agent, 两条件——评估器冻结 vs 评估器随 agent 一起演化——用脚本真值测"评估器报告的成功"与"真实成功"的差距随轮数如何变化。相关: notes/19、07、33、42。

Part C · Harness、Runtime、双系统、端侧

9. Projection / Isolation / Transfer 的 ROS 2 profile 缺口: Harness Engineering for Physical AI 只提出了三个功能, 没有实现; MHS 有设备发现协议但未公开权限模型。为什么重要: 让延迟预算、deadline、fallback 成为声明式配置, 是 harness 层可移植的前提 (预测 P3)。MVE: 在 ROS 2 上实现 Isolation (VLA 推理的时隙与 deadline) 与 Transfer (超时回退到验证过的基线控制器), 用 PhyAI 的 control-time Roofline 度量, 在 CloudEdgeVLA 的延迟注入设置下测成功率。相关: notes/23、27、36。

10. 双系统的"何时切换"作为验证问题 缺口: StreamVLA 的完成态门控与 RL^2 -VLA 的失败预测门控是两种切换判据, 没有并排; 切换误判的代价未量化。为什么重要: 切换判据本质上是一个在线验证器, 其准确率决定慢系统介入的收益。MVE: 同一双系统骨干上实现两种门控, 记录切换时刻与真实子任务边界的偏差, 报告误切换率与成功率的关系。相关: notes/26、32、19。

11. 治理层与自进化层的接口 缺口: Runtime Governance 的能力准入、EmbodiedGovBench 的升级安全、ICAN-Deploy 的身份稳定都为"新能力上线"准备, 但没有任何自进化系统 (ASPIRE、Zetta、PRACTICE) 接入它们。为什么重要: closed loop 这一格在两篇观点文章里只有定义没有机制。MVE: 把 ASPIRE 式技能库的"新技能入库"改为经过一个只读治理层 (能力边界检查 + 审计记录), 报告治理层拒绝率与任务成功率的权衡。相关: notes/25、04、07、39。

12. 长期运行口径的普及 缺口: Habilis-β 的 TPH × MTBI 只有一个平台; ENPIRE 的 MRU/MTU 只有一篇。为什么重要: "Experience Scaling"需要指标 (预测 P6)。MVE: 为任一开源 VLA + agent 系统跑一小时连续运行协议, 报告 TPH、MTBI 与介入原因分类; 与单次成功率对照, 看排序是否改变。相关: notes/27、06、08。

Part D · 学习闭环

13. 失败样本的价值：Q-Planning vs 只回收成功 缺口：Q-Planning 的对照（90/80 vs 成功样本 SFT 55/30）是同批 rollout 上最干净的一组数字，但只有两个任务；VERITAS、RoboClaw 只回收成功数据。为什么重要：决定数据回流管线该保留什么。MVE：在 VERITAS 或 RoboClaw 式管线上加 Q-Planning 的小 Q 函数，同一批 rollout 下比较"只用成功 SFT"与"成败都用于 Q"的成功率与样本效率。相关：notes/29、18、31。

14. Fleet 数据的准入与锚点保护 缺口：LWD 16 台机器人共享经验，但没有讨论数据准入；Memory Anchors 说明约 10% 锚点经验决定是否遗忘旧任务；通信攻击说明共享经验会共享污染。为什么重要：fleet 学习的第一条安全性质是"一台机器人的坏数据不会毁掉全体"。MVE：仿真多机器人设置里向共享缓冲注入一定比例的错误标注 rollout，测共享策略在新旧任务上的退化；加锚点保护与来源验证（Claim Provenance 式）看能否抵消。相关：notes/28、15、37。

15. twin 作为验证器的假阳性/假阴性率 缺口：RoboSnap、Agentic Real2Sim、PerceptTwin 报告重建质量或"计划成功率"，没有一篇报告"twin 里过了、真机上挂了"的比例。为什么重要：sandbox 的价值不在保真度本身，而在它作为筛选器的错误率。MVE：对同一批 agent 生成的技能/计划，先在 twin 里评测再在真机评测，报告 2x2 混淆矩阵；按任务类型（接触密集 vs 几何为主）分层。相关：notes/30、33、19。

Part E · 总览、接口、多机

16. 共享经验与共享世界知识层 缺口：小红书长文四层共享里，只有状态记忆（RoboOS-NeXT）与策略（LWD）有实证；MeCo 的相似任务记忆化是共享经验的雏形，Thea 的场景图与 STEM 记忆是共享世界知识的候选载体。为什么重要：fleet 的价值在多样性（I8），多样性的前提是经验能被其他机器人读懂。MVE：两台仿真机器人、不同任务，一台的 ASPIRE 式技能与 AGM 式子目标记忆导出为结构化记录，另一台检索使用；报告零样本迁移收益与检索错误率。相关：notes/37、22、04、12。

17. 接口标准的权限模型 缺口：MHS 与 ROSClaw 让任何模型接任何机器人，AgentRob 与通信攻击展示了被劫持的后果；MHS 预览未公开权限模型。为什么重要：接口越开放，"任何被劫持的模型接任何机器人"越容易。MVE：在 ROSClaw 式执行层上定义能力级权限（哪些技能需要人工确认、哪些可自动），复现 AgentRob 式注入攻击，报告拦截率与任务完成率的权衡。相关：notes/36、39、25。

Part F-G · 安全与基础

18. Robot Autoresearch Bench 缺口：软件侧有 PaperBench、RE-Bench、MLE-bench、KernelBench 衡量 AI 做研发；机器人侧的 ENPIRE、HARBOR、Nautilus、AgenticRobotics 没有共同基准，收敛速度、MRU/MTU、假设多样性、真机迁移差距无法跨论文比较。为什么重要：没有基准，"自动研究"的数字只能定性定位（见口径账本）。MVE：以 CaP-Gym 或 RoboCasa 为固定环境，定义 3-5 个有脚本验证器的任务，固定人类专家基线（RE-Bench 式），让不同 coding agent + harness 在固定预算下 hillclimb；报告收敛曲线、MRU/MTU、假设去重后的唯一假设数，以及仿真到真机的迁移差距。相关：notes/43、06、08、25。

数字口径账本：本仓库引用的头条数字，逐条标注它们到底在测什么

本仓库最常被违反的一条规则是"不同工作的成功率禁止直接比大小"。这张账本把报告、幻灯片与解读里反复出现的头条数字放进同一张表，逐条标注任务集、指标类型、对照对象、干预与更新、证据形式与来源。用法：只有当任务、平台、计分方式、示范与更新预算等关键条件可比时，两个数字才能作性能比较；其余情况只能是定性定位。摘要未说明的项一律写"摘要未说明"，不写"无"。

一、指标类型速查

类型	含义	与二元成功率的可比性	出现在
二元成功率	一次 rollout 全部完成才算成功	基准	大多数仿真基准（LIBERO-Pro、RoboCasa、RoboMME）
相对提升（%）	相对基线的相对百分比	取决于基线绝对值，不能换算	ASPIRE +77%/+72%/+32%、REMAC +40%
绝对提升（pp）	百分点差	需要基线绝对值才能定位	Harness VLA +38.6/+27.1、SkillOpt +23.5、RATs +20.6、BATON +11.6、PhysReflect +5.4
多任务平均	若干任务成功率的平均	任务集不同不可比	LWD 95%、Z-1 80.6%
连续成功次数	连续 N 次成功才算达标	远严于单次成功率	ENPIRE pin insertion（50 次）
固定重试预算下的成功率	K 次重试内成功	随 K 单调不减	ENPIRE 99%（固定八次重试）、Agentic RAG-VLM 三级重试
验证器准确率	验证器判断对错的比例	不是被验证策略的成功率	PRIMO R1 67%、LLM-as-a-Verifier 87.4%、Agentic Harnesses 97%
治理/安全指标	拦截率、违规率、不安全继续率	与任务成功率无关	Runtime Governance 96.2%/22.2%/90.7%、通信攻击 97.8%、CPV Gate 70.0→36.6%
系统效率	频率、延迟、加速比、能效、人工时间	与成功率无关	Fast-in-Slow 117.7 Hz、StreamVLA 72% 跳过、PhyAI 1.40-4.65x、RoboClaw -53.7%、ENPIRE 8x → 2-3x
收敛时间	达到目标水平所需时间	取决于目标水平与起点	ENPIRE Push-T 5 h → 2 h

二、头条数字账本

工作	头条数字	任务集与规模	指标类型	对照对象	训练/ 评测期 干预与 更新	证据形式	备注（解读来源）
Code as Policies	62–80%（未见属性/指令）· HumanEval 39.8%	仿真桌面任务，50 次/任务； HumanEval 为代码基准	二元； P@1	CLIPort（未见时接近 0）	few-shot 生成，无更新	论文 + 代码	属性组合有限、脚本判定（notes/01）
CaP-X	12 个模型；抽象撤回则成功率下降	7 任务 × 100 次/tier，8 个 tier	二元（趋势结论）	人类专家程序	多轮交互（M tier）； CaP-RL 有 GRPO 更新	论文 + 平台	真机为零样本展示，规模摘要未说明（notes/02）
RHO	45.0%（LIBERO-PRO）· 70.0%（Robosuite）	扰动抓放	二元	OpenVLA 0.0%、 π 0.5 12.83%、最强多轮 agent $\approx$ 18%	训练时 agent 搜索；部署单轮无更新	论文	与 VLA 非同训练数据对照（notes/03）
ASPIRE	+77%/+72%/+32% · 零样本 31% vs 4%	LIBERO-Pro / Robosuite 双臂 / BEHAVIOR-1K； LIBERO-Pro Long	相对提升； 二元	先前 code-as-policy 方法	技能库随任务增长；参数不更新	论文	相对提升无绝对值；sim-to-real 为初步证据（notes/04）
SkillOpt	52/52 cells 最优或并列 · +23.5/+24.8/+19.1	6 基准 × 7 模型 × 3 harness	软件基准准确率；绝对 pp	人工、一次性 LLM、Trace2Skill、TextGrad、GEPA、EvoSkill	技能文档编辑经 held-out 门；权重不更新	论文 + 代码	非机器人任务（notes/05）
ENPIRE	99%（部分任务）· Push-T 真机 95/75/60 · 8 台 2–3× 加速	PushT、插针、剪扎带；1/4/8 工位	固定八次重试成功率；二元；加速比	Codex vs Claude vs Kimi	agent 全程修改训练代码；策略权重更新	论文 + 讲稿转述	真机；pin insertion 要求 50 次连续成功（notes/06）

工作	头条数字	任务集与规模	指标类型	对照对象	训练/ 评测期 干预与 更新	证据形式	备注（解读来源）
Zetta	90.8% (LIBERO-Pro) · 93.6% (RoboCasa) · 11.1x 推理加速	"当前 rollout 预算下"	二元；加速比	摘要未说明基线	在线演化 critic 与恢复技能；基座冻结	论文	与 Harness VLA、RHO 不同基座与协议 (notes/07)
Eureka / DrEureka	83% 任务超过人工奖励	仿真 RL 任务集	"超过人工"的任务比例	人工设计奖励	LLM 迭代写奖励；RL 训练策略	论文 + 代码	不是成功率 (notes/08)
RATs	+20.6 pp vs CaP-Agent0	LIBERO-PRO	绝对 pp	CaP-Agent0 (同原语族)	玩耍阶段积累技能；测试时库冻结	论文	本线少有的同族可比数字 (notes/09)
VASO	97.2% 规范符合率	技能契约集	规范符合率	—	模型检查反例 → 契约更新	论文	不是任务成功率 (notes/09)
RoboMME / - Interference	14 种记忆变体无一全面领先；检索恢复无干扰水平	16 任务；跨会话干扰	二元（结论为定性）	变体之间	π 0.5 基座；变体重训	论文 + 基准	单基座 (notes/10)
PonderPounce	60.83% (9B) / 50.04% (0.8B) vs 17.93% · 78 ms / 25 ms	RoboMME	二元；延迟	当前观测 π 0.5	权重不更新（推理时上下文）	论文	与 14 种 RoboMME 变体的直接对比摘要未给 (notes/11)
AGM	验证头 2.43M 参数	RoboMemArena 系列	模型规模	"尝试即完成"的记忆基线	冻结 VLA；记忆按证据推进	论文	成功率见正文 (notes/12)

工作	头条数字	任务集与规模	指标类型	对照对象	训练/ 评测期 干预与 更新	证据形式	备注（解读来源）
HyMeS	41.3% → 60.1%	RoboMemArena	二元	无记忆管理基座	coding agent 迭代记忆代码；VLA 冻结	论文	迭代轮数摘要未说明 (notes/13)
BATON	+11.6%	RoboMemArena	提升（相对/绝对需核对）	同配方基线	参数不更新	论文	与 HyMeS 是否同协议需核对 (notes/14)
NativeMEM / Remember Smarter	32.4 → 84.0% · 53.6 → 70.6% (LIBERO-Plus)	各自基准	二元	无记忆基座	需（再）训练	论文	两者不可并排 (notes/15)
PRIMO R1	RoboFail 67%	RoboFail	失败检测准确率	摘要未说明	RL 训练 7B 视频 MLLM	论文（具身纪元转述）	验证器指标 (notes/17)
VERITAS	70% vs 65% (50 条自主 vs 50 条人工示范)	真机单任务，每格 20 次	二元	同量人工示范 SFT	验证过的 rollout 微调	论文（具身纪元转述）	20 次评测差异不显著，应读"相当" (notes/18)
LLM-as-a-Verifier	RoboRewardBench 87.4%	RoboRewardBench	验证器准确率	标准 LM judge	不训练	论文	可作 RL 密集奖励 (notes/19)
Harness VLA	+38.6 pp (LIBERO-Pro) · +27.1 pp (RoboCasa365, v4; 早期版本 +25.4) · 58.4% (RoboTwin C2R)	三个仿真基准	绝对 pp; 二元	同一冻结 VLA 直接执行	规则记忆学习; VLA 冻结	论文 (v4 PDF 已核对)	版本间数字有变动，引用时注明版本 (notes/20)
Runtime Governance	96.2% 拦截 · 100% → 22.2% 不安全继续 · 90.7% 恢复 · RVDR 61.3% vs 0%	1000 次随机试验	治理指标	AutoRT 式过滤、RoboGuard 式护栏	外置治理层	论文	主要为仿真化场景 (notes/25)

工作	头条数字	任务集与规模	指标类型	对照对象	训练/ 评测期 干预与 更新	证据形式	备注（解读来源）
Fast-in-Slow / StreamVLA / LaST0	117.7 Hz · 72% 时间步跳过解码、-48% 延迟、LIBERO 98.5% · +13-14%（真机）	各自设置	频率；比例；二元；相对提升	各自基线	训练	论文	系统指标与成功率混合（notes/26）
CloudEdgeVLA / PhyAI / EcoVLA	63.8-78.0% vs ≤6.4%（40 步延迟）· 1.40-4.65x · +236% 能效	仿真延迟注入；四种 VLA；20 Hz 约束	二元；加速比；能效	对比方法 / 原推理程序	—	论文	延迟为统一窗口（notes/27）
LWD	95% 平均	16 台双臂机器人，8 个真机任务	多任务平均二元	预训练 VLA 起点（摘要未说明）	fleet 离线到在线 RL；含人类纠正	论文	scaling 曲线摘要未给（notes/28）
Q-Planning	40 → 90% · 25 → 80%；成功样本 SFT 55% / 30%	真机两任务	二元	同批 rollout 的成功样本 SFT	只微调小 Q；BC 冻结；无人干预	论文	本仓库最干净的同批对照（notes/29）
TwinRL	接近 100%，20 分钟真机交互	四个真机任务	二元；时间	摘要未说明	twin 内 RL 预热 + 真机 RL	论文	任务复杂度需看正文（notes/30）
PerceptTwin	计划成功率 +39%	GPT-5 系列规划器	计划成功率（twin 内验证）	无 twin 验证	迭代规划	论文（ICRA 2026）	不是真机执行成功率（notes/30）
RoboClaw	长程 +25% · 人工时间 -53.7%	真机长程任务	提升；人工时间	基线流水线	EAP 学习；成功数据回收	论文	剩余人力分布未说明（notes/31）

工作	头条数字	任务集与规模	指标类型	对照对象	训练/ 评测期 干预与 更新	证据形式	备注（解读来源）
Z-1 / RL ² -VLA	80.6% (RoboCasa 24 任务) · OOD +17.3%	各自基准	多任务平均； 相对提升	SFT 基线	RL 后训练 / 测试时引导	论文	不可并排 (notes/32)
ROSClaw	越权动作提议率跨模型 3.4–4.8×	同一执行层，多个前沿模型	比值	模型之间	安全包络内预执行验证	论文	不是成功率 (notes/36)
多机器人通信攻击	97.8% 不安全动作成功率 · CPV Gate 70.0 → 36.6%	DMAS/HMAS 等三种架构	攻击成功率； 违规率	无防护	—	论文	安全指标 (notes/37)
Tool-RoCo / 对话对齐	协作工具 7.09% · 冲突 –40 至 –83 pp 但成功率下降	RoCo 基准；部分可观测协作	比例； pp	—	—	论文	协作质量指标 (notes/37)
Same Weights, Different Robot	28/28 → 2/28	LIBERO-Goal	二元	原元数据	替换一个动作反归一化元数据键	论文	极端单例，说明脆弱性存在 (notes/39)
Memory Anchors	约 10% 锚点；去掉后遗忘 4.5×	持续学习回放缓冲	比例； 倍数	随机回放	训练	论文	与 episode 记忆是不同意义的"记忆" (notes/15)
Push-T 重访	100% 成功，–46% 步数	2D gym 仿真	二元； 步数	200 条示范训练的最佳扩散策略	coding agent 迭代（无示范）	短文	仿真；与 ENPIRE 真机 60–95% 对照 (notes/08)
软件侧 RSI（参考）	Reflexion HumanEval 91% · HiSME MineDojo 0.700 → 0.856 · Meta-Rewarding 22.9 → 39.4% · DGM SWE-bench 20 → 50%	软件/推理基准	准确率 / 胜率	各自基线	见各论文	论文	仅作环节存在性证据，不与机器人数字比较 (notes/41、42)

三、使用规则

1. 同一行内的数字可以与其对照对象比较；跨行比较前先核对任务集、指标类型与更新预算三列是否一致。
2. "相对提升"与"绝对 pp"必须换算到同一口径并补齐基线绝对值后才能并排。
3. 验证器准确率、治理指标、系统效率三类永远不与任务成功率并排；它们描述的是循环的质量与成本，不是能力。
4. 凡标注"摘要未说明"的项，引用时应保留这一限定，不要在二次转述中补成确定值。

PART 1

五份源材料的解读与 25 条主线综述

小红书长文《Harness 之后，Agent+Robot 下一站是什么？》· Code-as-Policy 讲稿 · 23 条主线检索地图 · Lil'Log 两篇 · 具身纪元《GPT-6 ... 加速 RobotRSI》及其小红书图文版；随后是 25 条主线的逐线综述。

1. 导读
2. 1. 材料、方法与边界
3. 2. 解读一：《Harness 之后，Agent+Robot 下一站是什么？》
4. 3. 解读二：Code-as-Policy → 自进化机器人 Agent（讲稿）
5. 4. 解读三：检索地图的 23 条主线（2026 年 9 月状态）
6. 5. 解读四：Lilian Weng 的两篇文章
7. 6. 解读五：具身纪元《GPT-6 未必能当好机器人的大脑，却可能帮王兴兴加速 RobotRSI》

导读

这份报告回答三个问题：2026 年 Agent × Robot 这条线上到底发生了什么；手头五份材料（一篇小红书长文、一份 Code-as-Policy 讲稿、一张 23 条主线的检索地图、Lilian Weng 的两篇博文、具身纪元关于 Robot RSI 的公众号文章）各自说对了什么、漏了什么；沿着这些主线往前看，哪些判断有论文证据支撑，哪些还只是预测。

结论先放在前面。

1. **优化对象在上移。** 2022 年的 Code as Policies 让 LLM 写一段策略代码；2026 年的 SkillOpt、ASPIRE、RHO、ENPIRE 让 coding agent 优化的对象变成技能文档、多文件策略仓库、训练配方和整套 harness。讲稿里的统一公式 $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$ 抓住了这个变化：被学习的东西 z 从动作变成了“外部可训练产物”。
2. **“冻结 VLA + 外围学习”成了 2026 年的默认范式。** Harness VLA、BATON、AGM、HyMeS、Zetta、SHAPER 全都不改 VLA 权重，而是在外面学 VLA 的适用范围、记忆管理规则、恢复技能和运行时 critic。原因很实际：权重更新贵、慢、难验证。但这条路有天花板，LWD、Q-Planning、TEMPO、Temporal GRPO 说明真实反馈最终还要写回策略权重。
3. **验证器是新的瓶颈，也是新的 scaling 轴。** PhyAgentOS 的 SessionVerifier、Thea 的 Evaluation as Exit Codes、AGM 的“物理证据才能推进进度指针”、LLM-as-a-Verifier 把验证当作可扩展维度，以及那篇立场论文“成功率无法证明 VLA 在做物理推理”，都指向同一件事：能不能自进化，取决于能不能可靠地判断“这一步到底成没成”。
4. **Harness 从软件术语变成了机器人中间件问题。** 软件 Agent 的 harness 在工具调用边界介入；机器人的 harness 必须同时在控制、计算、通信三处介入（Harness Engineering for Physical AI），必须知道模型最大延迟、技能 deadline 和断网后的 fallback（小红书长文的 Real-time-aware Harness）。这不再是 prompt 工程，而是 OS 与实时系统设计。
5. **群体是经验规模化的唯一出路，但收益不是线性的。** LWD 用 16 台双臂机器人把单一 VLA 推到 95%；ENPIRE 用 8 个工位把收敛时间从 5 小时压到 2 小时——8 倍机器人换来 2-3 倍加速，多出来的是假设吞吐量，不是 rollout 数量。
6. **Robot RSI 是把这些串起来的框架。** 具身纪元文章的两条轴线（改进的环节：部署时自演化 / 训练时自迭代 / 自我评估 / 自动研究 × 人的参与程度）把 ENPIRE、ASPIRE、RoboHarness、RoboClaw、PRIMO R1、VERITAS、Eureka、DrEureka 与软件侧的 Reflexion、STaR、Let's Verify、Meta-Rewarding、The AI Scientist 放进同一张表；前沿 LLM 更可能先成为机器人研发循环的认知中枢，而不是机器人的末端控制器。

阅读路径：只想知道结论看第 7、8 章；想核对每条主线的论文看第 4 章和附录；想看五份材料各自的解读看第 2、3、5、6 章。仓库根目录的 `README.md` 是按主线组织的 208 篇论文清单，`data/papers.csv` 是机器可读版本。

1. 材料、方法与边界

1.1 四份输入

材料	形态	核心主张
《Harness 只是开始：Agent+Robot 真正值得关注的 6 个下一站》，具身RL日记，小红书，39 张图文卡片	观点 长文	Harness 解决“Agent 怎样稳定使用已有能力”；下一步是 Memory、Reflection & Evolution、双时间尺度、Agent+VLA+RL、Real→Sim→Learn→Real、Multi-Robot；终局是 System Scaling + Experience Scaling
Code-as-Policy 讲稿（25 页 PPTX）	论文 精读 讲稿	从 Code as Policies 到 SkillOpt、CaP-X、ASPIRE、ENPIRE，自进化 = 优化外部产物 z
《Agent + Robot 论文检索地图》，两页表格	检索 框架	23 条主线，每条给检索词与代表工作
Lil'Log: 《LLM Powered Autonomous Agents》(2023.06)、《Harness Engineering for Self-Improvement》(2026.07)	软件 侧综 述	Agent = Planning + Memory + Tool use；Harness 三模式、优化阶梯、RSI 七个挑战
《GPT-6 未必能当好机器人的大脑，却可能帮王兴兴加速 RobotRSI》，Marilyn Liu，公众号具身纪元（另有小红书图文精简版《GPT-6 Astra 开启 Robot RSI 时代》）	观点 长文	RSI 两条轴线（改进环节 × 人的参与程度）；LLM 更可能先成为 Robot RSI 的认知中枢；Robot RSI 缺一个可重复、可扩展的虚拟世界

1.2 方法

我们把地图上 23 条主线的代表工作和长文、讲稿点名的全部工作逐一在 arXiv 上核实（标题、作者、日期、摘要），再沿每条主线按提交时间倒序检索 2025–2026 的新工作，最终保留 208 条：46 条是材料点名的机器人侧核心工作，94 条是扩展检索到的 2025–2026 新工作，68 条是软件侧 Agent / Harness / RSI 的基础工作——包括 Lil'Log 两篇文章参考文献中的全部论文（harness 一文 39 条、agents 一文 21 条，去掉博客与代码库链接后共 54 篇）以及具身纪元文章点名的 Reflexion、STaR、Let's Verify Step by Step、Meta-Rewarding、The AI Scientist、HiSME、BigBang-V1、Gödel Machine、Anthropic 《When AI builds itself》。地图与文章上的非 arXiv 条目也做了溯源：BigBang-V1 是 Endless Frontier 的技术报告；PRIMO R1 对应 arXiv 2603.15600 《From Passive Observer to Active Critic》；VERITAS 对应 arXiv 2606.18247 《Visual Verification Enables Inference-time Steering and Autonomous Policy Improvement》；HiSME 对应 arXiv 2605.28390 《You Live More Than Once》；AgenticLab 对应 arXiv 2602.01662 (v1 题为 PPlanAR, Purdue)；MHS 是 Anthropic 2026 年 8 月 27 日发布的 Model Hardware Standard 研究预览；OpenClawPi 是松灵机器人 (AgileX) 面向 OpenClaw 的技能库，不是论文。

所有对论文的陈述都以摘要和材料原文为依据；报告区分“论文报告的数字”与“我们的判断”。

1.3 边界

机器翻译环节受本机条件限制：SuperTranslate 引擎需要 LLM API，本机所有 key 均不可用，因此六篇核心论文的中文版走的是引擎的手工翻译通路（`export` 导出文本块，译文由人工填入，`cache-only` 模式原位回填

并跑 `inspect` QA)。Code as Policies 完成了正文全文翻译，其余五篇（SkillOpt、CaP-X、ENPIRE、ASPIRE、Harness VLA）翻译了首页或首两页（标题、摘要、引言开头）。`scripts/translate_papers.sh` 可在配置 API key 后一键补全。

2. 解读一：《Harness 之后，Agent+Robot 下一站是什么？》

这篇长文的价值在于它给出了一个可检验的框架，而不只是一组新名词。作者的起点是一个反常识判断：VLA、WAM、Agent、Harness 不是相互替代的概念，而是在拼装同一台机器人系统。下面按六个“下一站”逐条对照论文。

2.1 Memory：机器人没有“过去”

长文的诊断准确：传统 VLA 的输入是当前图像 + 当前指令 + 当前本体状态，长程任务的核心变量因此从 State 变成 History。它点名的两篇论文数据都对得上。

- RoboMME (ICML 2026) 用 16 个操作任务、时间/空间/物体/程序四类记忆的分类，在 $\pi 0.5$ 上系统比较了 14 种记忆变体，结论是记忆表示的有效性高度依赖任务，没有一种设计全面占优。后续的 RoboMME-Interference 进一步发现：感知型记忆在没有干扰时最好，但随着无关会话累积而稳定衰减；子目标型记忆提升较小但更抗干扰；加一个按视觉相似度检索历史片段的步骤能在每个干扰级别上恢复无干扰时的成功率。
- PonderPounce (2026.08) 不设计专门的记忆模块，而是让一个 System 2 的 MLLM (Ponder) 在原生因果上下文里累积整段 episode 的观测、示范和先前认知，再异步地只把最新的一个认知 token 及其“年龄”发给 System 1 的 VLA (Pounce)。RoboMME 上 9B 版本 60.83%，0.8B 版本 50.04%，对比当前观测 $\pi 0.5$ 的 17.93%；认知刷新 p50 延迟 78 ms，动作模型 25 ms，支持 20 Hz 播放。

长文预测机器人记忆会分化为 Working / Episodic / Semantic / Skill 四类。2026 年 6–8 月的论文恰好在这四条线上各有实例：AGM 用带进度指针的子目标序列做 working memory，且只有物理证据验证子目标完成后才推进指针（它的结论很硬——可靠的具身记忆靠的是状态更新纪律，不是记忆容量）；Analytic Concept-Centric Memory 用部件、参数模板、位姿、affordance 组织物体与场景记忆，并连接转移记忆和技能记忆；HyMeS 提出“技能在权重里、记忆在代码里”，让 coding agent 用启发式学习迭代一个可执行的记忆管理系统，RoboMemArena 任务成功率从 41.3% 提到 60.1%；ViReSkill 把验证过的计划存进技能记忆下次直接重放。

长文没有展开、但论文已经给出的一点：记忆可以放在 VLA 权重内部。NativeMEM 复用 VLA 自身的视觉编码器把每帧压成一个 token 追加到输入序列，成功率从 32.4% 提到 84.0%；LaMem-VLA 把短期/长期记忆库在 VLA 原生潜空间里交织；Remember Smarter 用 Mamba 压缩视觉历史加双曲经验空间，LIBERO-Plus 从 53.6% 到 70.6%。权重内记忆与权重外记忆现在是两条并行路线，第 7 章会讨论它们的分工。

2.2 Reflection：Retry 不等于 Learning

长文把“自进化”拆成 L1 Retry、L2 Replan、L3 Reflection、L4 Memory Evolution、L5 Skill Evolution、L6 Policy Evolution、L7 Fleet Evolution 七层，并判断 2026 年正从 L2/L3 快速向 L4–L7 延伸。这个分层是这篇长文最有用的贡献，因为它能直接给论文归位：

层级	代表工作	被更新的对象
L1–L2 Retry / Replan	AgenticLab、Agentic Robot、MALMM、REMAC 的前置/后置条件检查	当前计划
L3 Reflection	PhysReflect-VLA、Agentic RAG-VLM 的 14 类失败分类	下一步动作的纠正指令

层级	代表工作	被更新的对象
L4 Memory Evolution	Harness VLA 学冻结 VLA 的“使用说明书”、AGM、OnEvoMemory	记忆与适用范围规则
L5 Skill Evolution	ASPIRE、ViReSkill、PRACTICE、SkillGLoW、RATs 的玩耍式技能发现	技能库
L6 Policy Evolution	LWD、Q-Planning、Z-1、TEMPO、Temporal GRPO	策略权重
L7 Fleet Evolution	LWD 的 16 台机器人、ENPIRE 的 8 工位、RoboOS-NeXT 的共享记忆	群体共享的数据与策略

长文对 Harness VLA 的概括值得强调：它不修改 VLA 权重，也不扩张技能库，而是从任务专属执行轨迹、全局成功规则和失败模型里学“这个 VLA 什么时候靠谱、什么时候先 MOVE_TO、什么时候重新 grounding、什么时候交给解析原语”。论文数字是 LIBERO-Pro +38.6 个百分点、RoboCasa365 +27.1 个百分点（v4，2026-09-02；小红书长文引用的早期版本为 +25.4）、RoboTwin C2R 58.4%。ASPIRE 的“经验复利”也有数字：LIBERO-90 上积累的技能库让 LIBERO-Pro Long 的零样本成功率随库规模单调上升，N=90 时达到 31%，对比先前方法 4%。

2.3 两个时间尺度：Agent 负责秒，Controller 负责毫秒

长文提出 Slow Loop（几百毫秒到数秒）和 Fast Loop（几十到几百 Hz）的划分，并引出 Fast-in-Slow（117.7 Hz）、StreamVLA（72% 时间步跳过自回归解码，延迟降 48%）、LaSTO（潜空间时空 CoT，低频推理专家给高频动作专家提供物理表征）。这些数字与论文一致。

更重要的是它把这个划分接到了 Harness 上：Harness Engineering for Physical AI 那篇短文说，软件 harness 在工具调用边界介入，机器人 harness 必须同时在控制、计算、通信三处介入，因为学习策略的输出同时改变轨迹、日程和带宽；它提出三个缺失的强制功能——Projection（在输出处约束动作）、Isolation（限定推理的执行与传输时隙）、Transfer（检查失败时回退到经过验证的基线），并建议把它们做成 ROS 2 Harness Profile。长文由此提出的原则“Cloud can think. Edge must survive.”有 2026 年的系统论文支撑：CloudEdgeVLA 在 40 步统一延迟窗口下仍保持 63.8–78.0% 成功率，而对比方法最高 6.4%；EcoVLA 在 20 Hz 约束下把端-边协同的能效提高 236%；ARLI 说明推理延迟会破坏 RL 依赖的马尔可夫假设，需要状态增广才能在延迟下微调。

2.4 Agent + VLA + RL：最终还是要写回权重

长文举的例子是插 USB：第一次偏 3 mm，Agent 可以记一条“下次向左修正一点”放进技能，但视觉特征、接触状态、力反馈、微动作之间的连续关系最终属于策略权重。这个判断有实证：LWD 用 16 台双臂机器人、8 个真实任务，从预训练 VLA 出发做 fleet-scale 离线到在线 RL，单一通用策略随群体经验积累提升到平均 95%，长程任务提升最大；Q-Planning 给大 BC 策略配一个小的 off-policy Q 函数，只微调 Q 就能从部署失败里学，真机 stack-cups 从 40% 到 90%、insert-wallet 从 25% 到 80%，而只用成功 rollout 做 SFT 分别停在 55% 和 30%。

长文对 CaP-X 的评价——重点不是“代码打败 VLA”，而是 agentic test-time compute + 环境反馈 + RL 开始结合——与论文自述一致。对 RoboClaw 的 EAP（把正向技能与逆向恢复技能绑定实现自复位采数据，成功率 +25%、人工时间 -53.7%）的评价也成立，而且它点出了一个常被忽略的成本结构：机器人 RL 最贵的是人，不是 GPU。ENPIRE 的 EN 模块（自动重置 + 验证）解决的是同一个问题。

2.5 Real → Sim → Learn → Verify → Real

长文说 Sim 的角色是安全测试环境，是 Physical Agent 的 sandbox，而不是回到“全部在仿真里训练”。

TwinRL 的流程与此吻合：手机拍摄重建数字孪生，SFT 阶段扩展轨迹分布支撑，twin 内并行 RL 预热真机 RL 并定位易失败配置，四个任务上接近 100% 成功且只用 20 分钟真机交互。2026 年这条线上新增了一批“把真实场景变成可交互仿真”的工具：RoboSnap 从单张 RGB 图生成可交互场景并发布 DROID-Sim（564 个场景）；Agentic Real2Sim 让 VLM agent 把真实交互录像转成可仿真的 episodic twin；PerceptTwin 从机器人感知栈自动构建交互仿真来验证与精炼计划，把 GPT-5 系列规划器的计划成功率平均提高约 39%；SafeDojo 在交互式视频世界模型上做带安全约束的 RL。世界模型在这里的角色确实如长文所说——是“脑内预演”的地方，而不只是动作预测器。

2.6 Multi-Robot：大规模进化不该发生在一台机器人身上

长文的 Shared Skill / Shared Experience / Shared World Knowledge / Shared Policy Improvement 四层共享，目前论文覆盖的是两端：RoboOS-NeXT 的 Spatio-Temporal-Embodiment Memory 做共享的时空-本体记忆；LWD 做共享的策略改进。中间两层（共享经验、共享世界知识）还没有系统性的论文，MeCo 的相似任务记忆化和 FSAR 的联邦式能力注册表算是雏形。长文估算的“100 万台机器人 × 每天 100 次任务 = 1 亿次物理交互/天”是推演，不是数据。

值得补充的是安全侧：多机器人 LLM 系统的通信攻击在 DMAS/HMAS 三种架构下都能把不安全信息变成不安全动作（最高 97.8% 不安全动作成功率），Claim Provenance and Verification Gate 能把违规率从 70.0% 降到 36.6%。群体共享经验意味着一个被污染的经验会被整个群体继承，这是长文没有讨论的风险。

2.7 评价

这篇长文的六个判断里，Memory、Reflection 分层、双时间尺度、Sim-as-sandbox 四条已有 2026 年论文实证；Agent+VLA+RL 的分工有 LWD 和 Q-Planning 两个强证据；Fleet 的中间层仍是预测。它最有价值的产出是 L1-L7 分层和三个闭环（Execution Loop / Learning Loop / Fleet Loop），这两个框架足以给绝大多数 2026 年论文定位。它的盲区是安全与治理：整篇没有出现 Runtime Governance、EmbodiedGovBench 这一类工作，而它们正是“让机器人长期工作”的前提。

3. 解读二：Code-as-Policy → 自进化机器人 Agent（讲稿）

这份 25 页讲稿的叙事线是：Code as Policies（2022）提出范式 → CaP-X（2026.03）把它做成研究平台 → ASPIRE（2026.06）加上技能库 → SkillOpt（2026.05）给出自进化的算法抽象 → ENPIRE（2026.06）把 coding agent 放进真机学习循环。讲稿最后给出一个统一抽象：自进化就是优化外部产物 z 。

3.1 五篇论文各在进化什么

讲稿第 22 页的对比表是全份材料里最有用的一张表，我们补上论文数字后重列如下。

论文	外部状态 z	反馈 → 验证/选择	定位
Code as Policies (2022.09)	policy code	语言指令 + few-shot 示例 → 程序可执行性 / 任务完成	提出范式
SkillOpt (2026.05)	skill.md	打分后的 rollout → held-out 验证分数严格提升才接受	自进化算法抽象
CaP-X (2026.03)	CaP agent / benchmark	执行反馈 / 视觉差分 → CaP-Bench / sim2real	研究平台
ASPIRE (2026.06)	控制程序 + 技能库	多模态轨迹 → 验证过的修复 / 任务成功	经验复用
ENPIRE (2026.06)	训练代码 / 配方 / 基础设施	真机 rollout + 日志 → 物理任务成功	真机自动算法工程

各篇的关键数字：

- Code as Policies：分层代码生成把 HumanEval P@1 提到 39.8%；仿真桌面任务上，属性与指令均未见时 CaP 保持 62–80% 成功率而 CLIPort 降到接近 0。
- SkillOpt：6 基准 × 7 模型 × 3 harness 共 52 格全部最优或并列；GPT-5.5 上直接对话 +23.5、Codex +24.8、Claude Code +19.1 个百分点。
- CaP-X：12 个模型；抽象降低则成功率下降；CaP-Agent0 无需训练恢复到接近人类水平；CaP-RL 以可验证奖励做 RL 并以很小差距 sim2real。
- ASPIRE：LIBERO-Pro 最多 +77%、Robosuite 双臂交接 +72%、BEHAVIOR-1K 最多 +32%；LIBERO-Pro Long 零样本 31% 对比先前方法 4%。
- ENPIRE：前沿 coding agent 自主训练到 99% 成功；8 工位集群把 pin insertion 收敛时间从 1.5 小时以上压到约 40 分钟。

3.2 Heuristic Learning 与统一公式

讲稿第 2 页把 coding agent 迭代修改软件结构的过程定义为 Heuristic Learning (HL)：它和 Deep RL 共享“状态–动作–反馈–更新”的闭环，但更新对象从神经网络参数换成了软件结构；反馈由 coding agent 消化，可以来自环境奖励、测试、日志、视频、回放和人类反馈；更新不走反向传播，coding agent 直接修改策略、状态检测器、测试、配置或记忆；被长期维护的对象叫 Heuristic System (HS)，它至少包含程序策略、状态表示、反馈入口、实验记录、回放/测试、记忆和更新机制。

第 24 页把它压成一个公式： $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$ 。A 是 LLM/VLM coding agent, z 可以是技能文档、`policy.py`、技能库、训练配方或 git 分支, τ 是 rollout 轨迹, r 是验证信号, $\log$ 是错误、视觉差分、机器人 trace、训练曲线等可诊断证据。这个公式的意义在于它把五篇看起来不同的论文放到了同一个坐标系：区别只在 z 是什么、 r 从哪来、 A 被允许改什么。

SkillOpt 把这个类比推到最严格：参数 $\leftrightarrow$ 技能文档, 梯度方向 $\leftrightarrow$ 从轨迹里抽出的编辑方向, 学习率 $\leftrightarrow$ 编辑预算 L_t , 验证 $\leftrightarrow$ held-out 选择门。它的四个设计选择都有消融数字：任意适度的编辑预算 ($L_t=2-8$) 都优于无界重写, 去掉预算后 Spreadsheet 77.5 $\rightarrow$ 75.7、LiveMath 61.3 $\rightarrow$ 57.3；选择门每个 epoch 只放行几十个候选编辑里的 1-4 个, OfficeQA 的 +39 个百分点来自单个被接受的编辑；去掉拒绝编辑缓冲区 Spreadsheet 77.5 $\rightarrow$ 72.9；去掉 epoch 级慢/元更新 Spreadsheet 77.5 $\rightarrow$ 55.0, 是所有消融里最大的退化。SkillOpt-Lite 随后用零阶优化的视角把流水线压到最小, 并把同样的方法推广到整套 harness (HarnessOpt), 让 GPT-5.4-nano 在 SpreadsheetBench 上超过跑标准流水线的 GPT-5.5。

3.3 ENPIRE 的四个实验教训

讲稿第 16-21 页对 ENPIRE 的四组实验做了很好的提炼, 我们逐条核对并补充。

- 1. 仿真成功不等于真机鲁棒。** Gym-PushT 里 Codex 和 Claude Code 约 2 小时达到 95%, Kimi 约两倍时间；迁移到真实 Push-T 后 Codex 接近 95%, Claude 和 Kimi 停在约 75% 和 60%。原因是接触摩擦随时间变化、物体位姿有误差、控制器与真实动力学有偏差、启发式对边界情况敏感。讲稿的结论是“Simulation success $\neq$ Real-world robustness”, 后续实验因此允许 Agent 混用启发式、BC 和 RL。Goldberg 组同月的《Revisiting Push-T with Agentic Robotics》给了另一侧证据：Claude Code 在没有任何示范的情况下写出的算法解在仿真里 100% 成功, 且比用 200 条人类示范训练的最佳扩散策略少 46% 步数——仿真里 coding agent 已经赢了, 真机上还没有。
- 2. 纯 BC 不够。** pin insertion 要求策略连续 50 次真机成功。BC 只学示范分布, 一旦偏离就进入无覆盖状态；iterative BC 让当前策略上真机采集偏移状态、失败接触、恢复动作和新成功轨迹, 逐步覆盖策略诱导分布。Agent 自行尝试了 BC、iterative BC、在线 rollout 聚合、offline RL、online RL、offline-to-online RL、RL + BC 正则, 以及 batch size、更新频率和 BC 权重的调优；hillclimb 时间线上最大的一步是 BC regularization (+10.8 个百分点)。
- 3. 群体提升的是假设吞吐量。** 1/4/8 个 Agent 对应 1/4/8 台机器人, Push-T 归一化得分 1.0 的时间从约 5 小时降到约 2 小时, pin insertion 从 1.5 小时以上降到约 40 分钟。8 倍机器人只有 2-3 倍加速, 因为 Agent 会重复探索相似想法、等待训练或大模型输出、阅读其他分支、整理合并实验、花时间分析日志。讲稿的判断正确：多机器人提升的是 hypothesis throughput, 不是同一策略的 rollout batch。
- 4. 持续学习发生在文本记忆里。** 作者让多个 Agent 在 pin insertion 上积累经验后把研究过程整理成 Markdown 总结（哪些 RL 方法稳定、BC 正则是否重要、哪些 batch size 有效、接触任务的控制器需要什么补偿、哪些失败模式优先检查），启动 GPU insertion 时把这份总结放进新 Agent 的初始上下文, 并刻意删除原始轨迹、checkpoint、隐藏日志和旧任务工作区。迁移路径是 research experience $\rightarrow$ textual memory $\rightarrow$ new autoresearch, 不是模型权重的持续学习。这与 Lilian Weng 说的“文件系统即持久记忆”是同一个机制。

第五组实验（讲稿第 21 页）说明 code policy 与 VLA 的分工：RoboCasa365 里纯 GR00T VLA 抓取前常缺少稳定的预定位动作, Agent 于是调用目标检测、分割、三维位置估计和运动规划先把末端移到物体上方的

hover pose，再把控制交给 VLA 完成接触密集的抓取。平均结果 GR00T N1.5 约 53%，ENPIRE 产生的混合方案约 77%，CaP-X 约 27%。这就是 Harness VLA、BATON 采用的同一分工：代码负责几何确定、可规划的阶段，VLA 负责接触丰富的阶段。

3.4 对讲稿的补充

讲稿的时间线止于 2026 年 6 月底。7–9 月出现了三类直接续写这条线的工作。第一类把 ASPIRE 的“技能库”推向更抽象的复用单元：PRACTICE 训练一个技能学习器对持久技能库做增/改/合/删的结构化批量编辑，执行器冻结；SkillGLOW 主张复用单元应是“一族相关任务共享的求解过程”，而不是单个全局文档或按任务堆积的条目池，未修改的先验库把未见 ALFWorld 任务成功率从 73.9% 提到 83.9%。第二类把“优化外部产物”从 z 推到运行时：Zetta 在冻结基座策略的前提下在线演化代码形式的 runtime critic 和恢复技能，三个时间尺度分别负责动作频率治理、rollout 级 critic–recovery 提议和验证门控的技能更新，LIBERO–Pro 90.8%、RoboCasa 93.6%、推理加速 11.1 倍；SHAPER 让同一个冻结模型既当规划器又当优化器，演化技能与 context–code harness。第三类是 RHO：coding agent 在训练时搜索多文件策略仓库（Repositories–as–Policies），部署时单轮执行、不再有纠错性代码编辑，在 LIBERO–PRO 上 45.0% 对比 $\pi 0.5$ 的 12.83%，Robosuite 70.0% 刷新最好成绩。RHO 直接回应了 CaP 一直以来的批评——多轮代码生成循环不适合实时控制。

讲稿第 9 页的提醒仍然成立：CaP 优化的是程序接口层，不是直接学习连续控制器。ENPIRE 的混合方案和 RHO 的“同样的底层原语”都说明，2026 年的 Code–as–Policy 不再和 VLA 竞争，而是在给 VLA 写外围。

4. 解读三：检索地图的 23 条主线（2026 年 9 月状态）

地图给每条主线列了检索词和代表工作。下面每条主线回答四个问题：这条线在解决什么问题；地图点名的代表工作说了什么；2025–2026 年新增了什么；我们怎么判断它的状态。论文全名、链接与作者见 [README.md](#) 对应小节。

T1 Agent + Robot 总览

问题：把 LLM/VLM 的推理接到真实机械臂上，能不能形成“感知 → 分解 → 执行 → 验证 → 重规划”的闭环。地图点名的 AgenticLab (arXiv 2602.01662, Purdue) 用规划语言接口定义 VLM 的推理空间：物体谓词表示场景状态，动作 schema 带前置条件与效果，符号计划作为可执行的中间表示，每步执行后用板载观测检查符号效果是否达成。Agentic Robot 用 Standardized Action Procedure 协调推理模型、VLA 执行器和时序验证器，LIBERO 长程任务 79.6%。ManiAgent 用多 agent 协作在 SimplerEnv 达到 86.8%，并用它给 VLA 生成训练数据。2026 年新增的 Cortex (32 个规范技能原语的双向对齐规划接口)、HoloAgent-0

(Embodied AgentOS + 3D 空间记忆)、VoLo (VLM 把 VLA/WAM 当作可中断工具中途干预，提出“物理编排”的时序问题)、Guava (系统探索 harness 设计空间，得到三要素：迭代感知–推理–动作循环、语义动作抽象、多模态观测，并蒸馏进 4B 模型) 都在同一条线上。《Agentic AI for Robot Control: Flexible but still Fragile》给了清醒的对照：两台真机上迁移只需改系统 prompt，但非确定性行为、指令遵循错误和对 prompt 措辞的高敏感性普遍存在。判断：闭环骨架已经标准化，可靠性瓶颈在验证与 grounding，不在规划。

T2 Coding Agent 控机器人

问题：让会写代码的模型直接生成机器人策略。这条线从 Code as Policies (2022) 到 2026 年出现了三个分叉。测试时生成：CaP-X 的 CaP-Agent0 用多轮交互、执行反馈、视觉差分、技能合成、集成推理在低层原语上恢复人类水平可靠性；MALMM 用 Planner/Coder/Supervisor 三个 LLM agent 零样本完成 RL Bench 任务；Neuro-Symbolic Code-as-Policies (NeurIPS 2025 Spotlight) 加入符号验证与交互式验证代码，比 CaP 基线成功率 +46.2%。训练时搜索：RHO 让 coding agent 搜索多文件策略仓库，部署时单轮执行；MEMENTO 用记忆引导的单精英模因演化搜索代码策略，并先演化 rollout 评估器；PhysCaP 加一层物理信息驱动的主动探索，从本体感知估计质量和刚度。研究自动化：ENPIRE、HARBOR (把机器人 RL 自动化当作 harness 工程问题)、Nautilus (一句 prompt 生成复现/评估/微调/部署 workflow)、AgenticRobotics (策略改进的控制平面，证据分级技能库与签名验证器)。判断：Coding agent 在这条线上的角色已经从“写策略的人”变成“做实验的研究员”，Lil'Log 里的 autoresearch 进入了物理世界。

T3 OpenClaw / ROS

问题：让任何基础模型接到任何 ROS 机器人，而不是为每对模型–平台写胶水。两篇同名的 ROSClaw 目标不同。Cardenas 等的 ROSClaw 把 OpenClaw agent runtime 接到 ROS 2，提供能力发现与 affordance 注入、观测归一化、安全包络内的预执行动作验证和审计日志，换模型或换平台只是改配置；它同时是测量仪器——在同一基底上，不同前沿模型的越权动作提议率相差 3.4–4.8 倍，且执行层设计比 prompt 措辞更影响任务完成与安全行为。Zhao 等的 ROSClaw 面向异构机器人，用 e-URDF 物理约束构建 sim–real 拓扑映射，把采集、训练与执行放进统一的 VLM 控制器。AgentRob 通过 MCP 把论坛 agent 与 Unitree Go2/G1 相连，也顺带展示了论坛介导的机器人被劫持风险。OpenGo 是 OpenClaw 驱动的 Go2 机器狗，技能库 + 调度器 + 基

于反馈的自学习。OpenClawPi 是松灵机器人的技能库，不是论文。MCP 正在成为机器人侧的通用接口：ROSBag MCP Server、ChemBot 的 MCP 子 agent 编排、Contract-Grounded BT Synthesis 让 coding agent 先向机器人侧 MCP server 拉取技能契约再合成行为树。判断：模型无关的执行层是 2026 年基础设施层最实用的进展，它把“安全包络”和“审计”做成了配置项而不是每个项目重写的代码。

T4 长程任务

问题：多阶段任务的误差累积与阶段间的隐性约束。RoboClaw 用 Entangled Action Pairs 把正向技能与逆向恢复绑定，实现自复位的持续采数据，长程成功率 +25%、人工时间 -53.7%。H-WM 让高层逻辑世界模型与低层视觉世界模型联合预测，为 VLA 提供稳定的中间引导。REMAC 用前置/后置条件检查和场景推理自进化做多机器人长程规划，成功率 +40%。2026 年最重要的新增是 BATON：它指出“冻结 VLA + LLM agent”配方在长程上会坏两次——整任务探索的代价是阶段数的指数 (T^K) 且失败无法归因到阶段，以及 VLA 原语只有退出条件没有进入条件。BATON 以子任务为探索单元（代价变成 $T \cdot K$ ）并用转移感知记忆治理调用、交接和前瞻三种转移，RoboMemArena 任务成功 +11.6%。AtomBridge 在原子技能边界用 LLM 生成过渡动作代码，8 步任务成功率 +10-25%；Foresight Residual RL 用下游成功概率塑形子任务交接状态，三阶段装配 85.6% 对比 54.5%。判断：长程问题正在被重新表述为“技能交接问题”，进入条件、退出条件和交接状态质量成为核心变量。

T5 Robot Memory

见 2.1 节。补充一条主线内的分类：权重内记忆（NativeMEM、LaMem-VLA、Remember Smarter、Dual Latent Memory、VQ-Memory、SkillMemo）与权重外记忆（AGM、HyMeS、BATON、Analytic Concept-Centric Memory、OnEvoMemory、ChemBot 的双层记忆、RoboHarness 的多模态执行记忆）。RoboMME 的结论——记忆表示的有效性高度任务依赖——是这条线目前最可靠的经验事实；AGM 的结论——可靠记忆靠状态更新纪律而非容量——是最有设计指导意义的判断。Memory Anchors 从持续学习角度补了一刀：回放缓冲里 10% 的关键锚点经验决定是否灾难性遗忘，去掉它们遗忘增加 4.5 倍。

T6 Reflection / 反思纠错

问题：失败之后系统改变的是什么。地图点名的 REMAC、AgenticLab、ASPIRE 分别对应 L2、L2-L3 和 L5。2026 年的新增让“反思”有了更细的形态：PhysReflect-VLA 用可行性算子评估候选动作是否产生动力学一致的状态转移、用动作解释算子核查转移一致性、用 LLM 反思模块分析状态差异生成纠正指令，接触密集真机任务平均 +5.4%；Agentic RAG-VLM 用 14 类失败分类和三级自适应重试，杂乱场景抓取从 25% 提到 78.3%；PhysiAgent 让 VLM 根据 VLA 的实时熟练度反馈组织 monitor、memory、reflection 组件；Online Continual RL with World Model Feedback 用 DreamerV3 的预测残差检测 OOD 事件并自动触发微调。判断：反思的价值取决于它能否被写进持久对象（记忆、技能、权重），否则只是更贵的 retry。

T7 Self-Evolution / 自进化

问题：系统能不能在没有人的情况下持续变好。Arcadia 主张具身学习是生命周期问题，四阶段（自进化探索与接地、生成式场景重建与增强、共享具身表征、sim-from-real 评估与演化）不可分解，去掉任一阶段就退回一次性训练。PhyAgentOS 把调度、验证、记忆、评测、安全做成系统级服务，以会话为最小调度单元，用 State-as-a-File 解耦认知与物理执行，SessionVerifier 区分执行终止与语义完成，验证过的结果经

epistemic memory 沉淀为可复用知识与纠正性教训，在 19+ 个仿真与真实本体上验证。Growing with Your Embodied Agent 走在环路线，把纠正编码为可复用技能配合外部记忆与 RAG，解决需要 20 个以上原语的“盖房子”任务。2026 年 8 月的一批工作把自进化的对象继续细分：PRACTICE 训练技能学习器维护技能库；SkillGLOW 以过程族为复用单元；Zetta 演化运行时 critic 与恢复技能；SHAPER 演化技能与 context-code harness；Motus2 用单模型三接口（策略/模拟器/评估器）形成决策-学习闭环；Q-Planning 只微调 Q 函数。与软件侧的 Self-Harness、AHE、Meta-Harness 对照，机器人侧的自进化在“可编辑面”上更保守（几乎都冻结基座策略），在“验证”上更依赖物理证据。判断：自进化的真正门槛是验证门与可编辑面的划定，而非搜索算法。

T8 Skill Library / 技能库

问题：技能从哪来、怎么存、怎么复用。Agentic Skill Discovery (2024) 完全由 LLM 驱动：LLM 提任务、采样奖励与成功判定函数、RL 学策略、VLM 独立验证，技能库从零生长。Atomic Skill Library 用三轮数据驱动流程（VLP 拆子任务 → 抽象技能定义 → VLA 微调）构建可动态扩展的原子技能库。ViReSkill 把验证过的计划存进技能记忆下次直接重放，不再调用 LLM。2026 年三个值得记住的变化：ASPIRE 的技能库跨任务、跨仿真/真机、跨本体持久化，并给出了库规模与零样本成功率的单调曲线；RATs 提出“玩耍”阶段——在下游任务到来前用自主提出的探索任务发现技能，LIBERO-PRO 比 CaP-Agent0 +20.6 个百分点，且技能可直接检索进其他 Code-as-Policy agent 的上下文；VASO 把技能表示为带形式接口的语义契约，模型检查的反例变成文本梯度更新技能契约，97.2% 规范符合率——这是“信任技能”的成本第一次被正面处理。ARCHITECT 把人类语言纠正蒸馏进持久技能库，SkillMemo 用 MoE 门控隐式切分技能原语。判断：技能库正在从“代码片段集合”变成“带验证证据与适用范围的知识库”，VASO 和 AgenticRobotics 的证据分级是方向。

T9 VLA + RL

问题：怎样把真实反馈写回策略权重而不破坏预训练先验。地图点名的 TwinRL（数字孪生预热真机 RL，20 分钟真机交互接近 100%）、LWD（fleet-scale 离线到在线 RL，16 台机器人 95%）、SAC Flow（把 flow rollout 视为残差 RNN，用门控/解码速度网络稳定 off-policy RL）、CaP-RL（对 coding agent 做可验证奖励 RL）、TT-VLA（测试时 RL，逐步任务进度密集奖励）覆盖了从训练时到测试时的全谱。2026 年的新增集中在两个技术点。信用分配：Temporal GRPO 按可检测任务阶段分配优势，解决轨迹级信用混叠；TEMPO 冻结 VLM 主干，语义投影层低频、动作专家高频的双时间尺度更新；WCM 让 critic 同时预测未来 latent 与价值，149 个任务上 SOTA。免外部奖励：T²VLA 用生成置信度作内在奖励；RL²-VLA 只在预测失败时激活 latent 组合式引导，OOD 最多 +17.3%；Z-1 在 $\pi 0.5$ 上用任务级 GRPO 把 RoboCasa 24 任务推到 80.6%。ARLI 解决的是部署侧问题：推理延迟破坏马尔可夫假设，标准 RL 在延迟下完全失效，需要状态增广。判断：VLA + RL 在 2026 年从“能不能”变成了“怎么分配信用、怎么在没有奖励和有延迟时做”，这是一条已经成熟的工程线。

T10 部署数据回流

问题：部署产生的数据怎么回到训练。LWD 的 Deploy → Experience → RL → Updated Policy → Redeploy 是目前最完整的实证；RoboClaw 的 EAP 解决自复位；Arcadia 用 sim-from-real 评估闭环。2026 年的新增：RoboGene 用多样性驱动 + 自反思物理约束的 agent 自动生成真实操作任务，采集 18k 轨迹，用它预训练的 VLA 泛化更好；AgenticRobotics 把“人可以离开”定义为一个操作性声明——因为晋升是证据门控的、状

态可恢复、能力质量由记录导出——并给出了假阳性晋升率 0.001 的硬化数字；Q-Planning 给出了“只用自己的部署 rollout、BC 冻结、无人干预”的真机自改进曲线。判断：数据回流的瓶颈从“采不到”变成了“怎么判断哪些经验值得回流”，这又回到验证器。

T11 数字孪生 / Sim2Real

问题：把真实场景变成低成本试错环境。TwinRL 和 Arcadia 见上。2026 年这条线的进展是“建 twin 的成本”在快速下降：RoboSnap 单张 RGB 图生成可交互场景并给出 sim-real 相关性；Agentic Real2Sim 让开源 VLM 驱动的 agent 把真实交互录像转成 episodic twin，成本远低于前沿模型；ConCent 以接触事件序列为学习目标做 real-to-sim-to-real RL，避免策略利用不真实的仿真接触；BestMan 提供自动场景生成加硬件无关中间件的 real-to-sim-to-real 平台；Real2Sim via Active Perception 让 VLM 生成行为树主动获取缺失的物理参数；PerceptTwin 直接从机器人感知栈构建交互仿真来验证计划。判断：twin 从“训练场”变成了“验证器的一部分”，它的价值是让 Agent 生成的新技能先在物理仿真里跑一遍。

T12 World Model

问题：世界模型在 Agent 系统里扮演什么角色。H-WM 用逻辑 + 视觉分层世界模型给 VLA 提供中间引导。2026 年的新增让角色更清楚：Motus2 用一个共享权重的模型同时暴露策略、模拟器和评估器三个接口，形成闭环；WCM 把世界建模目标加进 critic；ContactGuard 用潜空间世界模型做接触前执行监控，在机器人真正接触前预测并中止可能的失败；SafeDojo 在交互式视频世界模型上做安全 RL；Online Continual RL 用世界模型预测残差做 OOD 检测。判断：世界模型在 Agentic Robot 里最有用的角色是“预演器 + 监控器”，而不是动作生成器；这与小红书长文的“脑内预演”判断一致。

T13 快慢双系统

问题：推理要慢、控制要快，两者怎么共存。Fast-in-Slow 把 System 1 嵌入 System 2 共享参数，异构模态输入、异步频率，action chunk 为 8 时 117.7 Hz；OneTwoVLA 用单一模型自适应切换推理与执行模式；StreamVLA 只在子任务切换时触发慢思考并想象“完成态”作为时间不变的目标锚点，72% 时间步跳过自回归解码，LIBERO 98.5%；LaST0 把推理搬进潜空间，MoT 双专家异频运行，10 个真机任务平均 +13-14%；RationalVLA 用双系统拒绝 RAMA 基准里六维缺陷指令。2026 年新增的 UniFS 把 VLM 各层按更新频率分层（浅层快、深层慢），延迟 36.5 ms → 17.8 ms；Latent Bridge 预测 VLM 输出的帧间 delta，减少 50-75% VLM 调用；Libra-VLA 做异步粗到细；VisualThink-VLA 用视觉中间推理替代文本 CoT，步延迟 8.377 s → 0.367 s；DSWAM 把 System 1 换成世界动作模型。判断：双系统的设计空间已经被系统探索（OpenHelix 有综述与开源实现），剩下的问题是“何时切换”——StreamVLA 的完成态门控和 RL²-VLA 的失败预测门控是两种答案。

T14 Edge Agent / 端侧部署

问题：大模型上云、控制在端，延迟与断网怎么办。Harness Engineering for Physical AI 给了理论框架（Projection / Isolation / Transfer）。2026 年的系统工作：PhyAI 用一个运行时统一 VLA/WAM 在板载、边缘、云端的推理，对 π_0 、 $\pi_{0.5}$ 、GR00T N1.7、MiniCPM-Robot 提速 1.40-4.65 倍，并提出 control-time Roofline 区分推理受限与环境受限的控制；EcoVLA 做端-边协同推理的能效优化；CloudEdgeVLA 把时间错位当作表示学习问题，云端编码慢变任务特征、边缘头结合最新本地视觉，40 步延迟下仍保持 63.8-78.0%；

ST-Merge 训练无关地合并时空视觉 token， $\pi 0.5$ 在 1024×1024 分辨率下提速 8.3 倍；Habilis- β 提出用 Tasks per Hour 与 Mean Time Between Intervention 构成的生产力-可靠性平面，在 1 小时连续运行下评测端侧 VLA。判断：端侧部署的评价指标正在从单次成功率转向连续运行的吞吐与介入间隔，这是“让机器人长期工作”的必要度量。

T15 Harness

问题：围绕冻结模型的外围系统怎么设计、怎么优化。这是 2026 年 6-8 月密度最高的主线。RHO 优化多文件策略仓库；Harness VLA 学冻结 VLA 的适用范围；Harness Engineering for Physical AI 把中间件定义为 harness 层；PhyAgentOS 把 harness 做成 OS；Thea 补上 Scene Graph as Context 与 Evaluation as Exit Codes；Guava 系统探索 harness 设计空间；HARBOR 把机器人 RL 工程 harness 化；Zetta 让 harness 自己闭环演化；SHAPER 演化技能与 context-code harness；RoboHarness 用执行记忆刻画异构策略的能力边界并用 Memory Bridge 把机器人引导到下一策略的分布内区域；Agentic Harnesses 在规划与执行之间放 LLM-as-a-Judge 集成验证层，对抗攻击 97% 拦截；Cortex 在推理期做从任务上下文到技能约束的 harness engineering。软件侧的 Recursive Harness Self-Improvement、Meta-Harness、Self-Harness、AHE 提供方法论。判断：harness 是 2026 年 Agent × Robot 的中心词，但“harness updating $\neq$ harness benefit”的警告同样适用——写出 harness 编辑的能力与利用 harness 的能力是两回事。

T16 Runtime

问题：执行权交给 Agent 之后，谁来保证它可治理。PhyAgentOS 提供会话级调度、预检、监督执行、证据收集与验收。Harnessing Embodied Agents (Runtime Governance) 把治理外置为独立运行时层——策略检查、能力准入、执行监控、回滚、人工接管——1000 次随机试验中 96.2% 越权拦截，运行时漂移下不安全继续从 100% 降到 22.2%，恢复成功 90.7%；与 AutoRT 式宪法过滤和 RoboGuard 式两阶段护栏比较，预执行过滤各方法相当，只有它提供持续的运行时检测（RVDR 61.3% 对 0%）与结构化恢复。同一团队的 EmbodiedGovBench 把治理做成七维评测（越权调用、运行时漂移、恢复、策略可移植、升级安全、人工接管、审计完整性）；ICAN-Deploy 用 TLA+ 验证身份稳定的金丝雀部署；FSAR 主张多机器人协调不需要机器人内部的多 agent 碎片化，而是在 fleet 层联邦。HoloAgent-0 的 Embodied AgentOS、PhyAI 的统一推理运行时、AgenticRobotics 的控制平面也属于这条线。判断：Runtime 是把“可治理”从论文变成工程的地方，七维治理评测比任务成功率更接近部署方的关切。

T17 安全

问题：Agent 驱动的机器人如何不伤人。ROSClaw 的安全包络与审计、Runtime Governance 的拦截与回滚、RationalVLA 的缺陷指令拒绝是地图点名的三条路。2026 年的新增分四类。护栏模型：EMBGuard (ICML 2026) 用 2B/4B 模型对（观测，动作）对做物理风险判断，性能接近 GPT-5.1/Gemini-2.5-Pro 且假阳性更低。形式化：SENTINEL 用时序逻辑在语义、计划、轨迹三层评估；VASO 把形式验证接进技能自进化。世界模型：SafeDojo 在想象中学安全动作；ContactGuard 接触前中止。部署视角：Same Weights, Different Robot 指出动作反归一化元数据是可执行策略的一部分，替换一个看似合理的元数据键就能让 LIBERO-Goal 从 28/28 降到 2/28——安全审查只看 checkpoint 会漏掉到达控制器的真实策略。多机器人通信攻击（见 2.6 节）与 AgentRob 的劫持风险也在这条线。判断：安全正在从“过滤不安全计划”扩展到“运行时持续检测 + 恢复 + 可审计 + 升级安全”，并开始触及动作空间语义这类以前没人看的层面。

T18 标准接口 / Hardware API

问题：Agent 怎样标准化地接入真实硬件。Anthropic 2026 年 8 月 27 日发布的 Model Hardware Standard (MHS) 研究预览是这条线的标志性事件：一套让 agent 发现、操作、排障任意物理设备（显微镜、液体处理器、相机、机械臂、激光系统）的共享规范，被媒体称为“硬件版 MCP”，可通过 MCP、CLI 或代码调用。ROSClaw 的能力发现与 affordance 注入、Contract-Grounded BT Synthesis 的机器人侧 MCP 契约、ROSBag MCP Server、AgentRob 的 MCP 桥接、Harness Engineering for Physical AI 的 ROS 2 Harness Profile 都是同一方向的具体实例。判断：接口标准化会决定 harness 层的可移植性，MHS 与 ROS 2 Harness Profile 是否融合值得跟踪。

T19 多机器人协作

问题：多台机器人怎么用语言协商与协作。RoCo (2023) 让机器人用 LLM 对话协商分工与航点，交给多臂运动规划器；REMAC 加入前置/后置条件检查与自进化；RoboOS 用 Brain-Cerebellum 架构与实时共享记忆协调多本体；RoboOS-NeXT 用 STEM 记忆做终身多机器人协作。2026 年新增：DynaHMRC 的去中心化角色感知 agent 与领导者竞选；Scale-Plan 用 LLM 引导的动作图搜索裁剪 PDDL 问题；MeCo 用相似任务记忆化避免重复规划；Tool-RoCo 发现 LLM agent 很少把其他 agent 当作助手调用（协作工具只占 7.09%）；World-Model Alignment Through Dialogue 发现对话把动作冲突减少 40–83 个百分点却降低任务成功，并给出世界模型对齐的度量；FSAR 主张联邦式而非碎片化。判断：多机器人协作的语言层已经不缺方法，缺的是共享记忆的一致性与通信的可信性。

T20 跨本体

问题：技能与系统能否跨机器人形态迁移。RoboOS 支持异构本体；ASPIRE 的技能跨本体与机器人 API 迁移并有初步 sim-to-real 证据；Harness VLA 在 RoboTwin C2R 达到 58.4%。ROSClaw（异构）用 e-URDF 做物理约束；BestMan 用硬件无关中间件；PhyAgentOS 在 19+ 本体上验证。判断：跨本体在 Agent 层比在策略层容易——代码、技能契约和 harness 配置天然可迁移，而策略权重不能。

T21 群体学习 / Fleet Learning

问题：一台机器人学到的东西怎么变成群体的能力。LWD 是策略层的实证，RoboOS/RoboOS-NeXT 是记忆层的实证，ENPIRE 的 8 工位集群是研究层的实证（并给出 MRU/MTU 两个效率指标）。FSAR 讨论 fleet 层的治理与恢复边界。判断：群体学习目前有两端（共享策略、共享状态记忆），中间的共享经验与共享世界知识还缺系统性工作；Fleet 的加速比不是线性的（ENPIRE：8 倍机器人 2–3 倍加速），因为瓶颈在假设生成与分析而非 rollout。

T22 主动感知

问题：Agent 能不能为了获取信息而行动。AgenticLab 的在线验证需要主动观测；PhysCaP 让 Code-as-Policy agent 通过交互推断质量与刚度（找隐藏物体、检测空罐、挑成熟的鳄梨），Planner 决定何时探索与停止，Prioritizer 过滤不合理交互；ActiveVLA 做关键区域定位加主动视点选择与 3D 放大；Real2Sim via Active Perception 用 VLM 生成的行为树主动获取仿真缺失的物理参数。判断：主动感知在 Agent 系统里的形态是“把探索当作可调用的工具”，成本控制（何时停）是关键。

T23 Verifier / 成功验证

问题：“这一步到底成没成”由谁判断。Harness VLA 的成功规则与失败模型、PhyAgentOS 的 SessionVerifier、REMAC 的前置/后置条件是地图点名的三种形态。2026 年这条线上的新工作最能说明它已成为瓶颈：Thea 的 Evaluation as Exit Codes（检测动作何时应终止、判断是否成功、失败时诊断原因）；AGM 用本体感知线索决定何时验证、用点跟踪与跨视角语言比较决定达成了什么，一个 2.43M 参数的验证头；Agentic Harnesses 的 LLM-as-a-Judge 集成；LLM-as-a-Verifier 把验证当作新的 scaling 轴，对评分 token logits 取期望得到连续分数，RoboRewardBench 87.4%，还能作为 RL 的密集奖励；VASO 用模型检查替代轨迹级证据；PerceptTwin 用仿真验证计划；Consilience 讨论无验证器时如何利用置信度轨迹。那篇立场论文《VLA Cannot Be Verified to Perform Physical Reasoning》指出成功率无法区分语义匹配与物理泛化，需要受控变量的评测设计。判断：验证器决定了自进化循环的上限——奖励作弊、过度乐观、“仿真 100% 真机 60%”都在这里发生。

T24 Robot RSI：递归自我改进（新增主线）

问题：机器人系统能否参与改进“怎样让自己变得更好”，并把成果带进下一轮。这条主线来自具身纪元的文章（见第 6 章），把软件侧的 RSI 谱系（Gödel Machine 的“可证明有益才修改”、STaR、Reflexion、Let's Verify Step by Step、Meta-Rewarding、The AI Scientist、HiSME、BigBang-V1、Anthropic《When AI builds itself》）与机器人侧的四个环节并列：部署时自演化（ASPIRE、RoboHarness、Zetta）、训练时自迭代（RoboClaw、LWD、Q-Planning）、自我评估（PRIMO R1 的过程级视频批评者、VERITAS 的推理时视觉验证器）、自动研究（Eureka → DrEureka → ENPIRE 的“奖励 → 仿真参数 → 整个研究循环”演进）。判断：Robot RSI 是把 T2、T7、T9、T10、T23 串起来的框架而不是新方法；它的两个独有难题是可重复、可扩展的物理验证环境，以及当评价器本身也在演化时如何保持“评估器在循环之外”。

5. 解读四：Lilian Weng 的两篇文章

5.1 《LLM Powered Autonomous Agents》(2023.06)

这篇文章给出的分解——Agent = LLM 大脑 + Planning（子目标分解、反思与精炼）+ Memory（短期 = 上下文、长期 = 外部向量库与快速检索）+ Tool use（调用外部 API）——是小红书长文那张完整架构图的直系祖先。长文把 Planning 放进 Agent/System 2 层，把 Memory 拆成 Memory/Skill/Dataset，把 Tool use 具体化为 Harness/Runtime 的 Tool Routing 和 Skill Layer 的四类原语（Code Skill、Analytic Primitive、VLA Primitive、RL Policy）。2023 年文章列出的三个挑战——有限上下文长度、长程规划与任务分解的困难、自然语言接口的不可靠——在机器人侧对应的正是 Memory 主线、长程任务主线和 Verifier 主线。

5.2 《Harness Engineering for Self-Improvement》(2026.07)

这篇文章把 harness 定义为“围绕基座模型的系统，它编排执行，决定模型如何思考与规划、如何调用工具与行动、如何感知与管理上下文、如何存储产物、如何评估结果”。它的三个设计模式在机器人论文里都有对应物。

Lil'Log 的 harness 模式	机器人侧对应
工作流自动化：plan → execute → observe/test → improve 的目标导向循环	ENPIRE 的 reset → execute → verify → refine；ASPIRE 的执行引擎 → 诊断 → 修复 → 验证；RoboClaw 的 Collection ↔ Learning ↔ Deployment 单一 Agent 循环
文件系统即持久记忆：状态与产物存文件而不是塞进上下文	PhyAgentOS 的 State-as-a-File（跨层状态物化为 Markdown + YAML）；ENPIRE 的 Markdown 研究总结迁移；AgenticRobotics 的 commit 键控崩溃恢复与证据分级技能库
子代理与后台任务：并行搜索假设、监控作业、合并结果	ENPIRE 的 Agent 团队 × 机器人集群；HARBOR 的分阶段专门 agent；RATs 的 Robotics Agent Teams

文章提出的优化对象阶梯——instruction prompts → structured context → workflow → harness code → optimizer code——与讲稿的 z 完全对应：SkillOpt 在 structured context 一级 (skill.md)，ASPIRE 在 workflow 与技能库一级，RHO 与 SHAPER 在 harness code 一级，Meta-Harness 与 SkillOpt-Lite 的 HarnessOpt 在 optimizer code 一级。

文章里有两个结论对机器人尤其重要。第一，STOP 的自学优化器在 GPT-4 上有效、在 GPT-3.5 和 Mixtral 上退化，Lin 等人 2026 年进一步拆出两个轴：写 harness 的能力（从 Qwen3.5-9B 到 Claude Opus 4.6 几乎持平）和利用 harness 的能力（非单调，中等模型受益最大）。这意味着 ENPIRE 里 Codex、Claude、Kimi 在真机上的差距，更可能来自利用 harness 的能力而非提出改进的能力。第二，评估器和权限控制必须放在演化循环之外：AHE 把 runs 目录、tracer、verifier 和 LLM 配置设为只读，才能把每一次收益归因到 harness 编辑而不是奖励作弊。机器人侧的 Runtime Governance、ICAN-Deploy 和 AgenticRobotics 的签名验证器做的是同一件事。

两篇文章引用的全部论文（harness 一文 39 条：从 Good 1965 与 Yudkowsky 2008 的 RSI 概念、Absolute Zero / Self-Rewarding / SPIN 的自博弈，到 ADAS / AFlow / ShinkaEvolve / ThetaEvolve 的搜索方法与 PaperBench / RE-Bench / MLE-bench / KernelBench 等 AI 研发基准；agents 一文 21 条：CoT、ToT、

ReAct、Reflexion、Toolformer、HuggingGPT、Generative Agents 等）已收入 README 的基础主线 T0，RSI 相关者同时挂在 T24。

文章列出的 RSI 七个挑战——弱而模糊的评估器、上下文与记忆的生命周期、负面结果、多样性塌缩、奖励作弊、长期成功、人的角色——在机器人侧有更具体的形态：评估器弱对应 Verifier 主线（“这一步成没成”要靠物理证据）；记忆生命周期对应 RoboMME-Interference 的干扰衰减；负面结果对应 ASPIRE 与 Zetta 强调的失败轨迹是最有价值的数据；多样性塌缩对应 ENPIRE 里多个 Agent 重复探索相似想法；奖励作弊对应仿真里 100% 成功、真机上 60% 的 Push-T；人的角色对应 LWD 与 TwinRL 里的人工干预与 human-in-the-loop rollout。

5.3 两篇文章没有覆盖的部分

Lil'Log 讨论的 harness 生活在数字世界：状态可读、结果可判、失败可重试一千次。Thea 那篇论文说得最清楚——物理世界拒绝提供软件白送的两样东西：读取世界状态和判断动作结果。它用 Scene Graph as Context 和 Evaluation as Exit Codes 补这两个缺口。Harness Engineering for Physical AI 补的是第三个缺口：时间。软件 harness 不关心一次推理花 2 秒，机器人 harness 必须关心，因为这 2 秒会改变控制日程和轨迹。这三点是机器人 harness 与软件 harness 的本质差异，也是第 7 章几条洞见的出发点。

6. 解读五：具身纪元《GPT-6 未必能当好机器人的大脑，却可能帮王兴兴加速 RobotRSI》

这篇文章（Marilyn Liu，公众号具身纪元，2026 年 8-9 月）从一个反差出发：GPT-6 Astra 在 Robocurve 第三方测试的“方块放入碗中”拿到 19/20（Claude Fable 5.1 为 8/20），却在精细拼图插入上与对手同为 2/20，拧洗衣机按钮用了 4 分钟。作者的判断是，模型升级增强了视觉理解、空间规划、纠错和工具调用，但涉及接触、摩擦、精确对齐和毫米级误差时，LLM 的提升没有转化成控制能力；因此与其关注 LLM 对 policy generator 的贡献，不如关注它对具身领域 RSI（Recursive Self-Improvement，递归自我改进）的贡献。这与本报告 I2 的判断（冻结 VLA + 外围学习有天花板，接触相关的连续关系要写回权重）是同一件事的两面。

6.1 RSI 的定义与两条轴线

文章把 RSI 拆成三个词：Self（被改的可以是回答方式、记忆、工具、代码、训练数据、评价器或权重，不必是重写全部源代码）、Improvement（新版本必须在某个可检查目标上更好，无法验证的变化只能叫变化）、Recursive（上一轮成果进入下一轮并让系统更有效地制造下一版，“改进能力本身也被再次用于改进”）。它随后给出两条轴线，这是全文最有用的工具：

轴线一：改进的环节	LLM 侧代表	机器人侧代表（文章点名）
部署时自演化：权重不变，改推理、记忆、工具、代码、harness	Reflexion（反思存记忆再重试，HumanEval 91%）；HiSME（从执行轨迹学“怎样生成与修改技能”的元技能，MineDojo 0.700→0.856）；LiLog 的 harness 判断	ASPIRE（失败修复存成技能）；RoboHarness（执行记忆调度异构策略，切换前先把机器人带到下一策略熟悉的状态，135 次真机实验）
训练时自迭代：上一轮数据、推理、偏好、奖励回到训练，更新权重	STaR（答对保留、答错看答案重推、微调下一版）；BigBang-V1（出题者 / 批评者 / 元批评者合成约一万条可验证难题更新权重）	RoboClaw（同时学“完成任务”与“恢复现场”两套策略，正反交替回收成功数据，人工时间 -53.7%）
自我评估：改进 judge、reward model、process reward model、verifier	Let's Verify Step by Step（过程奖励模型逐步定位错误）；Meta-Rewarding LMs（同一模型当回答者、评审者和“评审的评审”，AlpacaEval 2 LC 22.9%→39.4%）	PRIMO R1（以初始 / 当前画面锚定过程视频，判断进度与失败位置，RoboFail 67%）；VERITAS（冻结策略 + 无梯度视觉验证器，50 条验证过的自主轨迹 70% vs 同量人工示范 65%）
自动研究：提出假设、改算法、跑实验、分析结果、安排下一轮	The AI Scientist（从代码模板到自动评审的全流程）	ENPIRE（自动复位、成功验证、策略更新、真机试验封装成工具）；Eureka（LLM 写奖励，83% 任务超过人工奖励）；DrEureka（LLM 同时写奖励与域随机化范围，四足机器人站瑜伽球迁移真机）

轴线二是人的参与程度：Human-in-the-loop（AI 提议、人逐次确认）、Human-on-the-loop（数据、奖励、评价、执行大体自动，人监督结果、设置权限、控制发布）、Closed loop（在预先授权范围内自行提出、验证并采用改进）。文章没有把论文逐一放进这条轴，但本报告第 4 章 T16 的 Runtime Governance、ICAN-Deploy 和 AgenticRobotics 的“人可以离开”的操作性定义，正是从 on-the-loop 走向 closed loop 时需要的基础设施。

6.2 与本报告框架的对照

两条轴线与前面几章的框架可以直接叠放。轴线一的四个环节对应讲稿公式里 A 被允许改的对象：部署时自演化改 z 中的 harness / 记忆 / 技能，训练时自迭代改策略权重，自我评估改 r 的来源，自动研究改整个实验流程。它也对应小红书长文的 L1-L7：部署时自演化覆盖 L3-L5，训练时自迭代是 L6，自动研究把 L7 的群体经验变成研究流程本身。文章对 ENPIRE 的概括——“把自动复位、成功验证、策略更新和真机试验封装成工具，让编程智能体连续执行—检查—改代码—再执行”——与讲稿第 3 章的四个教训一致。

文章新增的价值有三点。第一，它把 PRIMO R1 与 VERITAS 放进“自我评估”环节，补上了本报告 T23 里两类此前缺席的验证器：过程级的视频批评者（不只判断最终画面像不像成功，而是判断进展到哪一步、从哪里开始失败）和推理时的动作验证器（生成器-验证器框架，验证过的 rollout 直接成为微调数据，且效率与专家示范相当）。第二，它把 Eureka / DrEureka 这条 2023-2024 年的“LLM 写奖励与仿真参数”线接回自动研究，说明 ENPIRE 式的 physical autoresearch 有更早的源头：先自动化训练方法（奖励、域随机化），再自动化整个研究循环。第三，它给出了机器人 RSI 与大模型 RSI 的核心差异：评估包含两个问题——评价器能否判断成功、进度与失败位置，以及系统能否提供可重复、可扩展的验证环境；后者是 LLM 从未遇到、机器人必须解决的。这与本报告 I6（Sim 是 sandbox）和 I3（验证器是瓶颈）的结论重合，作者把它表述为“Robot RSI 还缺一个虚拟世界”，并认为世界模型是比传统仿真更可扩展的解法。

6.3 小红书图文版

同一观点还有一个小红书图文版《GPT-6 Astra 开启 Robot RSI 时代! howto 实现》（账号“♥VLA和RL的具身未来”，2026-09-06，五张文字卡片，带 #具身纪元 话题）。它没有新增论文，只点名 ASPIRE、RoboClaw、ENPIRE，但把 Robot RSI 压成了一段可以直接引用的定义：“机器人先执行任务。失败后，系统判断错在哪，智能体修改代码和策略，再去仿真和真机验证。有效经验会继续进入下一轮。”这段话恰好就是讲稿公式 $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$ 的自然语言版本：执行产生 τ ，判断错在哪产生 r 与 $\log$ ，改代码与策略就是 A 更新 z ，仿真与真机验证是选择门，进入下一轮是递归。结尾一句“GPT-6 未必先成为机器人的大脑，它更可能先成为制造下一代机器人能力的引擎”是对公众号长文核心判断的压缩。

6.4 评价

文章的核心判断——前沿 LLM 更可能先成为 Robot RSI 的认知中枢而不是机器人的末端控制器——有本报告核实过的证据支持：ASPIRE 用 Claude Opus 4.6 读多模态执行记录并修复程序，RoboHarness 用 GPT-5.5 改策略编排代码，ENPIRE 让 coding agent 查文献、提假设并在真机比较，Zetta 在冻结 VLA 下持续更新 critic、恢复技能和工具。它对产业动向的记录（Anthropic《When AI builds itself》、OpenAI 的 RSI 团队、Recursive Superintelligence / Trajectory / Discovery Loop 的融资与创立、王兴兴在 2026 世界机器人大会上的“直接让物理 AI 机器人模型实现自进化”）是观察，不是证据；Robocurve 测试与方舟无限视频也属第三方报道。文章没有触及的部分与小红书长文相同：治理与安全——closed loop 这一格在文章里只有定义，而 Runtime Governance 的 96.2% 越权拦截、EmbodiedGovBench 的七维评测正是让 closed loop 可被授权的前提；另一个空缺是 Lil'Log 强调的“评估器必须在演化循环之外”，当自我评估环节本身也在被系统改进（Meta-Rewarding 的“评审的评审”）时，这条原则如何在机器人上落实，是 Robot RSI 真正的难题。

PART A

主线：Coding Agent 与自进化

从 Code as Policies 到 CaP-X、RHO、ASPIRE、SkillOpt、ENPIRE，以及 7-9 月的自进化续作、自动研究谱系与技能库谱系。优化对象沿阶梯上移：策略代码 → 技能文档 → 技能库 → 训练配方 → 运行时 → 整套 harness。

1. 01 · Code as Policies 深度解读：让 LLM 写的程序成为机器人策略
2. 02 · CaP-X 深度解读：把 Code-as-Policy 做成可以被系统研究的平台
3. 03 · RHO 深度解读：训练时搜索策略仓库，部署时单轮执行
4. 04 · ASPIRE 深度解读：失败修复沉淀成技能，技能库越大适应越快
5. 05 · SkillOpt 深度解读：把技能文档当作冻结 Agent 的可训练外部状态
6. 06 · ENPIRE 深度解读：把真机学习变成 coding agent 可以管理的优化过程
7. 07 · 自进化续作合评：Zetta、SHAPER、PRACTICE、SkillGLoW、HiSME、MEMENTO
8. 08 · 自动研究谱系：Eureka → DrEureka → HARBOR / Nautilus / AgenticRobotics / Push-T 重访
9. 09 · 技能库谱系：从 LLM 提任务到形式化可验证的技能契约

01 · Code as Policies 深度解读：让 LLM 写的程序成为机器人策略

Code as Policies: Language Model Programs for Embodied Control arXiv 2209.07753 (2022-09) · Robotics at Google · Jacky Liang, Wenlong Huang, Fei Xia et al. (共 8 位作者) · 项目页 code-as-policies.github.io 所属主线：T2 · 材料来源：讲稿第 3-9 页；本仓库有正文全译 papers/pd_f_zh/2209.07753_zh.pdf

1. 一句话定位

Code as Policies (CaP) 把"机器人策略"重新定义为 LLM 写出的一段程序：程序处理感知模块的输出（例如开放词汇目标检测器给出的物体位置）、调用控制原语 API 并给参数赋值，还能表达反馈循环。它是本仓库 T2 主线的起点，也是 2026 年 CaP-X、RHO、ASPIRE、ENPIRE 全部工作的共同祖先。

2. 要解决的问题

2022 年之前，把自然语言接到机器人有两条路：端到端学习语言条件策略（需要大量数据，且难以泛化到新指令），或者用 LLM 做高层规划、把步骤映射到预定义技能（技能集固定，无法表达"把苹果往左移一点"这类需要空间推理和参数化的指令）。CaP 的判断是：LLM 在代码补全上的能力可以直接复用——代码天然具备组合性、能调用第三方库（NumPy、Shapely）做空间几何推理，也能表达条件与循环。

3. 方法

核心概念是语言模型程序（Language Model Programs, LMP）：给 LLM 少量"指令 → 代码"示例（few-shot prompt），让它为新指令生成 Python 代码；代码里可以调用感知 API（如 `detect_objects`）、控制 API（如 `pick_place`、`set_velocity`）、第三方库，以及尚未定义的函数——分层代码生成（hierarchical code generation）让 LLM 递归地为未定义函数生成实现。论文区分了反应式策略（例如"看到橙子就后退"的 while 循环）与航点式策略，并展示 LMP 可以组合：一个 LMP 解析指令、另一个解析物体名、另一个解析航点。

4. 实验结果与口径

- **HumanEval**：分层代码生成把通用代码生成基准的 P@1 提到 39.8%（当时的 state of the art）。这是软件侧指标，不是机器人指标。
- **仿真桌面任务**（UR5e + 积木/碗，50 次试验/任务）：按属性与指令"已见/未见"划分四格。属性与指令均已见时 CaP 与 CLIPort 相当；属性或指令未见时 CLIPort 掉到接近 0，CaP 保持 62-80%（长程 SA-UI 80.0%、空间几何 UA-UI 62.0%）。口径：二元成功率，仿真，成功由脚本判定。
- **真机**：移动机器人导航与操作、桌面抓放；论文主要以定性示例展示，未给出与仿真同规模的统计。

5. 局限

1. CaP 优化的是程序接口层：它假设感知 API 与控制原语已经存在且可靠，本身不学习连续控制器（讲稿第 9 页的提醒）。接触密集的任务（插入、倒水）超出它的表达范围。

2. 论文自己列出的三条边界：感知 API 描述范围有限、可用原语有限、依赖 few-shot 示例的质量；对多轮交互与部署时的实时性没有讨论。
3. "未见指令仍有 62-80%"的口径是仿真、脚本判定、属性组合有限，不能与 2026 年 LIBERO-Pro 这类扰动基准直接比较。

6. 关系定位

CaP 之后这条线分成三叉：测试时生成（CaP-X 的 CaP-Agent0、MALMM、Neuro-Symbolic CaP，见 notes/02）、训练时搜索（RHO 的多文件策略仓库，notes/03；MEMENTO 的演化搜索）、研究自动化（ENPIRE，notes/06；HARBOR、Nautilus，notes/08）。讲稿把这条演化压成公式 $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$ ：CaP 是 z = 一段策略代码、 A = 单次 few-shot 生成、没有 r 与 $\log$ 回流的起点。ENPIRE 的 RoboCasa365 实验（代码做 hover pose、VLA 做接触阶段）给了 CaP 与 VLA 分工的实证答案：代码管几何确定的阶段。

7. 延伸批判

CaP 的成功率与"原语抽象层级"强相关，而这一点直到 CaP-X（2026）才被系统量化：把人工设计的高层原语撤掉后，同一批模型的成功率随之下降。因此 2022 年 CaP 的数字应当理解为"在设计者提供的脚手架上 LLM 能做到什么"，而不是 LLM 对机器人控制的裸能力。另一个未走之路是安全：LLM 生成的代码直接驱动真机，论文没有讨论执行前验证与权限边界，这个空缺由 2026 年的 ROSClaw 安全包络与 Runtime Governance 填补（notes/25、36）。

02 · CaP-X 深度解读：把 Code-as-Policy 做成可以被系统研究的平台

CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation arXiv

2603.22435 (2026-03) · NVIDIA · UC Berkeley · Stanford · CMU · Letian Fu, Justin Yu, Karim El-Refai et al.

(共 16 位作者) · 项目页 capgym.github.io 所属主线: T2、T9、T22 · 材料来源: 讲稿第 10–12 页、小红书长文 04 节; 本仓库有首页中译

1. 一句话定位

CaP-X 是 Code-as-Policy 的"研究平台四件套": CaP-Gym (交互环境)、CaP-Bench (分层评测)、CaP-Agent0 (无需训练的 agentic 框架)、CaP-RL (对 coding agent 做可验证奖励的强化学习)。它第一次把"Agent 的能力"与"设计者给的脚手架"拆开衡量, 结论是成功率随人工抽象而升、随抽象撤除而降, 但测试时计算 (多轮交互、执行反馈、视觉差分、技能合成、集成推理) 能把低层原语上的鲁棒性补回来。

2. 要解决的问题

先前的 CaP 系统依赖高层的人工原语, 因此无法回答两个问题: LLM 到底会不会控制机器人, 还是只会调用别人写好的宏? 不同模型、不同交互方式、不同感知接地方式之间的差异有多大? 没有统一的交互环境与分层评测, 这些问题无法被系统研究。

3. 方法

- **CaP-Gym**: 建立在 Gymnasium 接口上的分层控制框架, 把低层环境循环 (物理仿真或真机) 与有状态的代码执行器循环绑在一起, agent 通过合成并执行组合感知/控制原语的程序来控制机器人。
- **CaP-Bench**: 三条轴——抽象层级 (人工宏 → 原子原语)、时序交互 (单轮 S1-S4 / 多轮 M1-M4)、感知接地 (不同视觉反馈模态); 7 个核心任务 (Cube Lift、Cube Stack、Spill Wipe、Peg Insertion、Cube Re-stack、Two-Arm Lift、Two-Arm Handover), 每个 tier 100 次试验, 12 个开源与闭源模型。
- **CaP-Agent0**: 多轮交互 + 视觉差分模块 (把场景变化转成文本) + 自动合成的任务无关技能库 + 并行多模型代码生成 (集成推理), 不做任何训练。
- **CaP-RL**: 用 GRPO 在 CaP-Gym 里对 Qwen2.5-Coder-7B-Instruct 做可验证奖励的后训练; 跨 sim-to-real 迁移的是"代码即动作空间"接口, 而不是像素到电机的映射。

4. 实验结果与口径

- 12 个模型上, 成功率随人工抽象单调上升、随抽象撤除下降 (Takeaway 2); 多轮 + 视觉差分 + 低层 API (M4) 显著优于单轮低层 API (S3) 和单轮高层 API (S2)。
- CaP-Agent0 在 7 个任务中的 4 个上达到与人类专家程序相当的成功率 (论文图 8); 在仿真与真机 (Franka Panda、AgiBot) 上有零样本展示。

- CaP-Bench++ 把 coding agent 与 VLA 直接对比：LIBERO-PRO 位置/指令扰动下 CaP-Agent0 对比 OpenVLA、 $\pi 0.5$ （表 2）。
- 口径：每 tier 100 次试验、脚本判定成功；真机数字为零样本展示，试验规模摘要未说明。讲稿与小红书长文引用的"抽象降低则成功率下降"是趋势结论，不是单一数字。

5. 局限

1. 论文自己指出程序化控制在接触丰富、需要紧密视觉伺服的任务（插入、倒水）上仍脆弱；给出的方向是 CaP-VLA 混合策略。
2. 多轮交互的代价是时间：CaP-Bench 的多轮 tier 不测实时性，RHO（notes/03）正是针对这一点。
3. 12 个模型的差异同时来自模型能力与 prompt/接口设计，CaP-Bench 控制了接口但不能控制各模型的训练数据。

6. 关系定位

CaP-X 是讲稿五篇论文里的"研究平台"一格（ $z = \text{agent/benchmark}$ ， r 来自执行反馈与视觉差分）。它对本仓库的三条主线都有贡献：T2（Code-as-Policy 的系统化）、T9（CaP-RL 是最早对 coding agent 本身做 RL 的机器人工作之一）、T22（视觉差分是一种"为获取信息而观察"的弱形式主动感知）。ENPIRE 的 RoboCasa365 对照（GR00T $\approx 53\%$ 、ENPIRE 混合 $\approx 77\%$ 、CaP-X $\approx 27\%$ ）说明纯代码在 RoboCasa 这类家居长程任务上落后于混合方案。

7. 延伸批判

CaP-X 最重要的贡献是方法论：先把脚手架拆掉再谈能力。这个思路应当被反向应用到 VLA 侧——VLA 的成功率里有多少来自基准设计的"脚手架"（固定初始位姿、有限指令模板）？LIBERO-Pro 的扰动评测（RHO、Harness VLA 都在用）是同一思路的另一面。另一个未走之路：CaP-Agent0 的技能库跨试验持久化，但论文没有报告库规模与成功率的关系曲线，这条曲线由 ASPIRE（notes/04）补上。

03 · RHO 深度解读：训练时搜索策略仓库，部署时单轮执行

RHO: Your Coding Agent is Secretly a Robotician arXiv 2606.16458 (2026-06) · Karim Elmaaroufi, Justin Svegliato, Sarunas Kalade et al. (共 6 位作者) 所属主线: T2、T15 · 材料来源: 检索地图 T2/T15 代表工作; 报告第 3.4 节

1. 一句话定位

RHO (Robotics Harness Optimization) 把 Code-as-Policy 从"测试时多轮生成"改成"训练时搜索、部署时单轮执行": 带工具的 coding agent 在训练阶段提出并搜索可解释的、神经符号的**多文件策略仓库**

(Repositories-as-Policies), 用环境奖励与执行反馈做反思式搜索, 部署时不再有纠错性的代码编辑。它直接回应了 CaP 一直以来的批评——多轮代码生成循环不适合实时控制。

2. 要解决的问题

CaP-X 已经证明多轮交互 + 执行反馈能显著提升低层原语上的成功率, 但每一轮都是一次 LLM 调用, 真机控制等不起。RHO 问的是: 能否把这些"纠错"提前到训练阶段, 把纠错的结果沉淀进一个可以单轮执行的仓库?

3. 方法

优化对象从单个 prompt / 函数 / 文件扩大到多文件策略仓库; 搜索信号是环境奖励与执行轨迹 (不需要遥操作示范); 搜索过程由 coding agent 的反思驱动。低层原语与对比系统相同, 因此差异归因于仓库结构与搜索过程。

4. 实验结果与口径

- **LIBERO-PRO** (扰动的抓放): OpenVLA 0.0%, π 0.5 平均 12.83%, 最强的多轮 agentic 系统约 18% (按"2.5 倍"反推), RHO 45.0%。
- **Robosuite**: 70.0%, 刷新此前多轮方法的最好成绩。
- 口径: 两个基准都是仿真、脚本判定; LIBERO-PRO 的"扰动"是初始位置与指令层面的扰动 (与 CaP-X 表 2 同一基准)。RHO 与 VLA 的对比是"同一原语上的代码策略 vs 端到端策略", 不是同训练数据的对照。

5. 局限

1. 训练阶段的搜索成本 (LLM 调用次数、rollout 数) 摘要未给出; "部署时单轮"把成本搬到了训练时而不是消除了它。
2. 仓库对任务分布的泛化边界不清楚: LIBERO-PRO 的扰动集是否覆盖了搜索时未见的扰动类型, 需要看正文。
3. 仍然依赖可靠的低层原语; 接触密集任务未在摘要中出现。

6. 关系定位

RHO 位于 T2 与 T15 的交点：它既是 Code-as-Policy 的训练时版本，也是"harless 优化"这一 2026 年中心词的机器人侧代表——优化的是围绕原语的整个仓库，而不是模型。与 Lil'Log 的优化阶梯对照，RHO 在 harless code 一级；与 SkillOpt (notes/05) 对照，两者都用 held-out/环境反馈做选择，但 SkillOpt 的 z 是一份文档，RHO 的 z 是一个仓库。标题"你的 coding agent 其实是个机器人学家"与 ENPIRE 的 physical autoresearch (notes/06) 是同一判断的两种表述。

7. 延伸批判

RHO 最值得追问的是**可解释性的代价**：神经符号仓库可读、可审计，但仓库越大，人类审查的成本也越高——"可解释"是否只是"可以被 coding agent 解释"？其次，RHO 与 Harless VLA (notes/20) 是互补的两半：前者让代码策略在扰动下达到 45%，后者让冻结 VLA 在同一基准上 +38.6pp；两者在同一原语与同一 VLA 上的组合（代码管交接、VLA 管接触）尚无人报告。

04 · ASPIRE 深度解读：失败修复沉淀成技能，技能库越大适应越快

ASPIRE: Agentic Skills Discovery for Robotics arXiv 2607.00272 (2026-06-30) · NVIDIA · UMich · UIUC · UC Berkeley · CMU · Runyu Lu, Yubo Wu, Ethan Kou et al. (共 14 位作者) · 项目页
research.nvidia.com/labs/gear/aspire 所属主线：T2、T6、T7、T8、T20、T24 · 材料来源：讲稿第 13 页、小红书长文 02 节、具身纪元文章"部署时改进"节；本仓库有首页中译

1. 一句话定位

Aspire (Agentic Skill Programming through Iterative Robot Exploration) 是一个持续学习系统：coding agent 以 code-as-policy 范式编写并精炼机器人控制程序，把经过验证的修复蒸馏进一个跨任务、跨仿真/真机、跨本体持久化的技能库；随着遇到的任务增多，技能库带来越来越快的适应——LIBERO-Pro Long 上零样本成功率随库规模单调上升，N=90 时达到 31%，先前方法 4%。它是讲稿五篇里"经验复用"的那一格，也是小红书长文 L5 Skill Evolution 的代表。

2. 要解决的问题

现有机器人 coding agent 受限于"朴素执行环境"——只给粗粒度的任务级反馈。一次失败的 rollout 只说明任务没成，不说明根因是感知错误、抓取不稳、规划错误还是下游恢复失败。没有细粒度诊断，就没有可复用的修复；没有可复用的修复，每个新任务都要从零试错。

3. 方法

三个组件构成开放式学习循环。**闭环执行引擎**暴露细粒度多模态轨迹——感知叠加层、抓取候选、运动轨迹、碰撞反馈——让 agent 自主诊断失败、合成修复、验证结果。**持续扩展的技能库**把验证过的修复蒸馏为可复用、可迁移的机器人知识。**演化搜索**生成多样的任务序列与控制程序并系统地调试它们，超越单轨迹精炼进行探索。具身纪元文章给的例子：机器人因导航目标落在桌子避障区而拿不到收音机，系统改为从多个方向尝试，找到可达位置后完成抓取，并把这个修复存为技能。讲稿注明 Aspire 使用 Claude Opus 4.6 读取多模态执行记录。

4. 实验结果与口径

- 扰动下的操作任务 (LIBERO-Pro) 最多 +77%；Robosuite 双臂交接任务 +72%；长程家居任务 (BEHAVIOR-1K) 最多 +32%。这些是相对先前方法的提升幅度，摘要没有给出绝对值。
- 零样本泛化：LIBERO-Pro Long 未见长程任务 31%，对比先前方法（大量依赖测试时推理与重试）4%；讲稿给出库规模 vs 成功率的单调曲线 (N 从 0 到 90)。
- Sim-to-real：仿真里发现的技能提供了迁移的"初步证据"，在本体与机器人 API 不同的情况下减少了真机编程工作量；真机试验规模摘要未说明。
- 口径：仿真基准为脚本判定的二元成功率；"+77%"等为相对提升，与 RHO 的 45.0% 绝对值不可直接并排。

5. 局限

1. 技能库的"验证过"依赖执行引擎的成功判定；对验证器本身的可靠性（notes/17–19 的主题）论文没有专门评估。
2. 库随任务单调增长，摘要没有讨论技能之间的冲突、过时技能的淘汰与库的检索成本——这正是 PRACTICE（增/改/合/删）与 SkillGLOW（过程族去重）随后处理的问题（notes/07）。
3. Sim-to-real 只有"初步证据"；技能跨本体迁移的成功率没有量化。
4. 依赖前沿闭源模型（Claude Opus 4.6）读多模态轨迹；开源模型能否支撑同样的诊断质量未知——Lil'Log 提到的"利用 harness 的能力非单调"在这里同样适用。

6. 关系定位

ASPIRE 同时落在六条主线上，是本仓库连接度最高的核心工作之一。对 T8 技能库谱系（notes/09）它是"验证过的修复即技能"的代表；对 T6 反思它把反思的结果写进了持久对象，符合小红书长文"Retry ≠ Learning"的标准；对 T24 它是具身纪元矩阵里"部署时自演化"的机器人侧例子。与 ENPIRE（notes/06）比较：两者同出 NVIDIA GEAR，ASPIRE 的 z 是控制程序 + 技能库，ENPIRE 的 z 是训练配方与基础设施；ASPIRE 主要在仿真基准上证明经验复利，ENPIRE 在真机上证明研究流程自动化。

7. 延伸批判

"库规模 vs 零样本成功率单调上升"是本仓库最想看到的那类曲线，但它有两个隐含条件：任务分布相关（LIBERO-90 → LIBERO-Pro Long 同属 LIBERO 家族），且库由同一 agent 在同一执行引擎上积累。曲线在跨基准、跨本体条件下是否仍然单调，是检验"经验复利"是否成立的关键实验。另一个未走之路：ASPIRE 的技能是代码，Harness VLA（notes/20）的"技能"是 VLA 的适用范围规则——两种技能库能否合并为一个既含代码又含 VLA 调用条件的库，直接对应 BATON 指出的"VLA 原语没有进入条件"问题。

05 · SkillOpt 深度解读：把技能文档当作冻结 Agent 的可训练外部状态

SkillOpt: Executive Strategy for Self-Evolving Agent Skills arXiv 2605.23904 (2026-05) · Microsoft · 上海交通大学 · 同济大学 · 复旦大学 · Yifan Yang, Ziyang Gong, Weiquan Huang et al. (共 15 位作者) · aka.ms/SkillOpt 续作: **SkillOpt-Lite: Better and Faster Agent Self-evolution via One Line of Vibe**, arXiv 2607.03451 (2026-07) 所属主线: T7、T8、T0 · 材料来源: 讲稿第 14-15、23 页; 本仓库有首页中译

1. 一句话定位

SkillOpt 主张技能 (skill.md) 应当被当作冻结 Agent 的外部状态来训练, 并采用使权重空间优化可复现的同一套纪律: 独立的优化器模型把打分后的 rollout 转成对单一技能文档的有界增/删/改编辑, 只有严格提升 held-out 验证分数的编辑才被接受; 文本学习率预算、拒绝编辑缓冲区与按 epoch 的慢速/元更新使训练稳定, 部署时零额外推理调用。它不是机器人论文, 却是讲稿把"自进化"抽象成 $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$ 的算法原型。

2. 要解决的问题

现有 Agent 技能要么人工编写、要么一次性生成、要么通过松散控制的自我修订演化——没有一种像深度学习优化器那样对待技能, 也没有一种能在反馈之下可靠地超越起点。对闭源前沿模型, 权重适配不可用; 技能文档是唯一可训练的对象, 却缺少"训练"的纪律。

3. 方法

类比表 (讲稿第 15 页): 参数 $\leftrightarrow$ 技能文档; 梯度方向 $\leftrightarrow$ 从轨迹中抽出的编辑方向; 学习率 $\leftrightarrow$ 编辑预算 L_t ; 验证 $\leftrightarrow$ held-out 选择门; 负样本 $\leftrightarrow$ 拒绝编辑缓冲区; epoch $\leftrightarrow$ 慢速/元更新。流程: 目标模型带当前技能跑任务 $\rightarrow$ 评分 $\rightarrow$ 优化器模型读轨迹并提出有界编辑 $\rightarrow$ 在 held-out 集上验证, 严格提升才接受, 否则进拒绝缓冲区供后续参考 $\rightarrow$ 每个 epoch 做一次慢速元更新整理文档结构。SkillOpt-Lite 把这套流程用零阶优化 (中心差分、信任域) 重新形式化, 提出三条原则——基于文件系统的轨迹探索、共识属性挖掘、独立验证门控——并去掉冗余组件。

4. 实验结果与口径

- 6 个基准 $\times$ 7 个目标模型 $\times$ 3 种执行 harness (直接对话、Codex、Claude Code) 共 52 个单元, 全部最优或并列最优, 击败人工、一次性 LLM、Trace2Skill、TextGrad、GEPA、EvoSkill 技能。
- GPT-5.5: 无技能基线上直接对话 +23.5、Codex 循环内 +24.8、Claude Code 内 +19.1 个百分点。
- 消融 (讲稿第 15 页转述): 任意适度预算 ($L_t=2-8$) 优于无界重写, 去掉预算 Spreadsheet 77.5 $\rightarrow$ 75.7、LiveMath 61.3 $\rightarrow$ 57.3; 选择门每 epoch 只放行几十个候选里的 1-4 个, OfficeQA 的 +39pp 来自单个被接受的编辑; 去掉拒绝缓冲 Spreadsheet 77.5 $\rightarrow$ 72.9; 去掉 epoch 慢/元更新 Spreadsheet 77.5 $\rightarrow$ 55.0 (最大退化)。
- 迁移: 优化后的技能跨模型规模、跨 Codex/Claude Code 执行环境、迁移到邻近数学基准时无需再优化即保持价值。

- SkillOpt-Lite: LiveMath 在 GPT-5.5 上 +8.8、GPT-5.4-nano 上 +25.4, nano 超过用 SkillOpt 优化的标准 GPT-5.4; 并把方法推广到整套 harness (HarnessOpt), GPT-5.4-nano 在 SpreadsheetBench 上超过跑标准流水线的 GPT-5.5。
- 口径: 全部是软件/推理基准的准确率; "+23.5 个点"是对无技能基线的绝对百分点提升。

5. 局限

1. 任务是数字世界的可验证任务 (表格、数学、办公问答), held-out 验证廉价; 搬到机器人上, "held-out 集"意味着真机 rollout, 验证成本是 SkillOpt 纪律能否成立的第一道门槛。
2. 优化器模型与目标模型的关系摘要未展开: 优化器更强时收益如何归因, 是"更强模型代写技能"还是"目标模型学会了"? Lil'Log 引用的 Lin 等 (Harness Updating $\neq$ Harness Benefit) 正是对这一点的追问。
3. 52/52 是"最优或并列", 并列的比例摘要未说明。

6. 关系定位

SkillOpt 在 Lil'Log 优化阶梯的 structured context 一级; 在讲稿公式里 $z = \text{skill.md}$ 、 $r = \text{held-out 分数}$ 、 $A = \text{优化器模型}$ 。机器人侧的对应物是 ASPIRE ($z = \text{技能库}$)、Harness VLA ($z = \text{VLA 使用规则}$)、Zetta ($z = \text{运行时 critic 与恢复技能}$)。它对机器人侧最有价值的输出不是数字, 而是"选择门 + 有界编辑 + 拒绝缓冲 + 慢更新"这四条纪律——具身纪元文章的 RSI 定义 (无法验证的变化只能叫变化) 与这里的严格提升门是同一原则。

7. 延伸批判

SkillOpt 的成功很大程度上来自任务有便宜且客观的验证器。机器人任务缺的正是这个 (notes/17-19、报告 I3)。因此把 SkillOpt 搬到机器人的最小可行实验, 不是换任务, 而是先换验证器: 用 PRIMO R1 式的过程级判断或 VERITAS 式的视觉验证器充当 held-out 分数, 看选择门在噪声验证下是否仍能保证"严格提升"。另一个未走之路: SkillOpt 只训练一份文档; 机器人技能库是多份文档 + 代码, PRACTICE 的批量编辑与 SkillGLOW 的过程族 (notes/07) 是向这个方向的两步。

06 · ENPIRE 深度解读：把真机学习变成 coding agent 可以管理的优化过程

ENPIRE: Agentic Robot Policy Self-Improvement in the Real World arXiv 2606.19980 (2026-06-19) · NVIDIA GEAR · CMU · UC Berkeley · Wenli Xiao, Jia Xie, Tonghe Zhang et al. (共 17 位作者) · 项目页
research.nvidia.com/labs/gear/enpire 所属主线: T2、T7、T9、T10、T21、T24 · 材料来源: 讲稿第 16-21 页
(最详细的一份材料)、小红书长文 02/04 节、具身纪元文章; 本仓库有首页中译

1. 一句话定位

ENPIRE 是面向 coding agent 的真机 harness, 用四个模块把"重置场景、执行策略、验证结果、改进下一轮"这条物理反馈回路实例化: 环境模块 EN (自动重置与验证)、策略改进模块 PI、Rollout 模块 R (单台或多台真机并行评估)、演化模块 E (coding agent 读日志、查文献、改训练基础设施与算法代码)。借助它, 前沿 coding agent 自主开发出在 PushT、插针入盒、剪扎带等灵巧任务上 99% 成功率的策略; 8 工位集群把收敛时间压到 1/3-1/2; 并提出 MRU / MTU 两个效率指标。它是本仓库"physical autoresearch"的代表, 也是具身纪元文章 Robot RSI 矩阵里"自动研究"一格的机器人侧例子。

2. 要解决的问题

真机灵巧操作严重依赖人类监督与算法工程: 重置场景、判断成败、调参、换算法都是人。coding agent 已经在数字环境里自动化算法搜索, 但缺一个"可重复的真机策略改进反馈回路"作为抽象。ENPIRE 的假设是: 把重置、验证、更新、真机试验都做成 agent 可调用的工具, 机器人研究就变成 agent 可以管理的可控优化过程, 同时还能对训练配方与 agent 变体做公平消融。

3. 方法

EN 模块负责自动重置 (接触密集任务用 CaP-X 式的程序化工具调用, 把环境直接复位到最近一次完整尝试的起点) 与成功验证 (例如扎带插入后由两个相机共同判断是否穿过锁头); PI 模块启动策略精炼, 支持启发式学习、工具调用、行为克隆、离线/在线 RL 等多种机制; R 模块用一台或多台并行真机评估; E 模块里 coding agent 分析日志、查阅文献、改训练基础设施与算法代码。研究经验以 Markdown 总结的形式在任务之间迁移。

4. 实验结果与口径

讲稿第 16-21 页的四组实验 (数字为讲稿转述):

1. **Gym-PushT → 真实 Push-T**: 仿真里 Codex 与 Claude Code 约 2 小时达 95%、Kimi 约两倍时间; 真机上 Codex 接近 95%, Claude 约 75%, Kimi 约 60%。结论: 仿真成功 ≠ 真机鲁棒 (摩擦漂移、位姿误差、控制器偏差、启发式对边界敏感)。
2. **Pin insertion** (要求连续 50 次真机成功): Agent 自行尝试 BC、iterative BC、在线 rollout 聚合、offline RL、online RL、offline-to-online、RL + BC 正则及超参调优; hillclimb 最大一步是 BC regularization +10.8pp。结论: 纯 BC 不够, 需要策略诱导分布上的迭代数据。

3. **集群规模**: 1/4/8 个 Agent 对应 1/4/8 台机器人, Push-T 归一化 1.0 的时间约 5h → 2h, pin insertion 1.5h+ → 40min; 8 倍机器人 2-3 倍加速, 瓶颈是 Agent 重复探索、等待训练、阅读分支、分析日志。
4. **文本记忆迁移**: 多个 Agent 的 pin insertion 研究总结写成 Markdown 放进 GPU insertion 新 Agent 的初始上下文, 并刻意删除轨迹、checkpoint、隐藏日志、旧工作区。路径: research experience → textual memory → new autoresearch。

第五组 (RoboCasa365 仿真): GR00T N1.5 约 53%, ENPIRE 产生的混合方案 (代码做 hover pose, VLA 做接触阶段) 约 77%, CaP-X 约 27%。摘要另给: 固定八次重试设置下部分任务 99%; pin insertion 收敛到 100% 快于一个前沿的 human-in-the-loop 方法。口径: 真机数字为特定任务、特定重试预算下的成功率; MRU (机器人主动执行时间占比) 与 MTU (token 利用率) 是效率指标而非成功率。

5. 局限

1. 任务集小且都有清晰的自动验证器 (PushT 位姿、插针、剪扎带); 验证器难做的任务 (叠衣、倒水) 不在其中——验证器是 ENPIRE 能否推广的前提 (报告 I3)。
2. 三个模型在真机上的差距 (95/75/60) 来自哪一步 (提假设、写代码、读日志) 没有拆分; Lil'Log 引用的 "写 harness 与利用 harness 是两种能力" 提示这一差距未必是 "提出改进的能力"。
3. 集群加速亚线性的原因是观察性的 (重复探索、等待), 没有做去重或分工机制的对照实验。
4. "人力降到最低" 针对的是研发循环; 硬件维护、任务定义、验证器搭建仍是人。

6. 关系定位

ENPIRE 在讲稿公式里 z = 训练代码/配方/基础设施, r = 物理任务成功, $\log$ = 真机日志与训练曲线, A = 前沿 coding agent。对小红书长文它同时是 L6 (策略权重被 RL 更新) 与 L7 (8 工位群体) 的实证; 对 T21 它给出了 "群体提升假设吞吐量而非 rollout 数" 这一最诚实的数据; 对 T10 它的 EN 模块解决了 RoboClaw (notes/31) 用 EAP 解决的同一个自复位问题。与 HARBOR (仿真 RL 工程 harness 化)、Nautilus (一句 prompt 到工作流)、AgenticRobotics (策略改进控制平面) 同属 T2 的 "研究自动化" 分叉 (notes/08)。

7. 延伸批判

ENPIRE 最有价值的发现是负面的: 仿真 100% 不等于真机 60%, 8 倍机器人不等于 8 倍速度, 纯 BC 到不了 50 次连续成功。这三条分别对应 Lil'Log 的 RSI 挑战里的奖励作弊/过度乐观、多样性塌缩、负面结果。下一步最值得做的不是更多任务, 而是把 E 模块里 Agent 的假设去重 (ShinkaEvolve 式新颖性拒绝采样) 和把 Markdown 研究总结换成结构化、可检索的记忆 (HyMeS 式 "记忆在代码里"), 看 8 工位的加速比能否逼近线性。另一个开放问题: ENPIRE 的验证器 (两台相机判断扎带是否穿过) 是人写的; 当验证器也交给 agent 演化时, 如何保持 AHE 式的 "只读 verifier" 原则, 是 Robot RSI 的真正难题。

07 · 自进化续作合评：Zetta、SHAPER、PRACTICE、SkillGLOW、HiSME、MEMENTO

覆盖：Zetta ζ (arXiv 2608.16590, 2026-08) · SHAPER (2608.11350, 2026-08) · PRACTICE (2608.30760, 2026-08) · SkillGLOW (2609.02217, 2026-09) · HiSME (2605.28390, 2026-05, 清华 + 华为) · MEMENTO (2607.22832, 2026-07) 所属主线：T7、T8、T15、T24 · 材料来源：报告第 3.4 节；具身纪元文章点名 HiSME 与 Zetta

1. 一句话定位

讲稿的时间线止于 2026 年 6 月；7-9 月的六篇工作把"优化外部产物 z"推向三个方向：**运行时**（Zetta 在线演化代码形式的 critic 与恢复技能；SHAPER 演化技能与 context-code harness）、**技能库的维护**（PRACTICE 训练一个技能学习器做增/改/合/删；SkillGLOW 以"过程族"为复用单元；HiSME 演化"怎样演化技能"的元技能）、**搜索算法**（MEMENTO 的记忆引导单精英模因演化）。共同点是基座策略全部冻结。

2. 各篇要点

Zetta ζ (闭环具身 harness)。诊断：现有 harness 基本是开环的——rollout 中按固定技能执行，只在 episode 结束后反思，而物理交互需要以超过大 agentic 模型频率的速度跟踪机器人-环境状态。方法：三个时间尺度分离的循环——动作频率的治理（运行时 critic）、rollout 级的 critic-恢复提议、验证门控的技能更新；配 Z-Infra 把 agent 逻辑与异构执行资源解耦。结果：LIBERO-Pro 90.8%、RoboCasa 93.6% ("在当前 rollout 预算下")，推理加速 11.1 倍。口径：仿真基准；90.8% 与 Harness VLA 的 +38.6pp、RHO 的 45.0% 分别对应不同基座与协议，不能并排。

SHAPER (技能-harness 演化)。面向"固定接口"场景——很多 train-free 代码方法依赖可编程机器人 API，而这类 API 未必存在；SHAPER 让同一个冻结模型既当规划器又当优化器，通过目标环境 rollout 演化可复用技能与 context-code harness。它把 Lil'Log 的"optimizer code"一级搬进了具身场景。

PRACTICE (从经验到专长)。诊断：现有基于经验的方法靠人工设计的 prompt 流程抽取与更新技能，面对新而多样的经验时僵化。方法：训练一个技能学习器，读历史技能与新轨迹，输出结构化批量编辑（增、改、合、删），再分层整合成一致的技能库；执行器冻结；两阶段课程训练学习器。意义：技能库维护本身成为被训练的对象。

SkillGLOW (过程族技能整合)。诊断：文本技能要么是一份全局文档（塌缩为泛泛的纪律），要么是按任务堆积的条目池（膨胀且绑定在写它的实例上）；两者在"每个任务都需要不同解法"的长程负载上以相反方式失效。方法：把任务写出的局部技能聚合成过程族，压缩成去实例化的全局先验，实例细节按任务重新生成而不存储；commit gate 只在真实执行证明不劣化时接纳先验。结果：四个基准（数学推理、终端自动化、软件修复、具身控制）× 三个模型，先验带来 +17.2 个百分点（摘要截断处）；报告第 3.4 节引用的 ALFWorld 73.9→83.9% 来自未修改先验库的零样本迁移。

HiSME (分层技能元进化)。诊断：测试时技能演化要么是硬编码策略，要么依赖昂贵的参数更新。方法：从执行轨迹学"元技能"——怎样生成与修改技能——同时优化技能与技能演化策略；权重不变。结果（具身纪元文章转述）：MineDojo 任务成功率 0.700→0.856。它是 Lil'Log 优化阶梯里"optimizer code"一级的另一个实例。

MEMENTO（记忆引导的模因演化）。先演化一个 rollout 评估器（把 rollout 映射为标量适应度与结构化反馈指标），再用适应度选精英、用反馈指标条件化记忆引导的爬山与宏突变。它提醒我们：在代码策略的演化搜索里，评估器本身也是被演化的对象——这与 AHE“verifier 只读”的原则形成张力。

3. 局限（共同）

1. 六篇几乎全部在仿真（LIBERO-Pro、RoboCasa、MineDojo、ALFWorld）验证；只有 Zetta 强调运行时频率，但真机数据摘要未说明。
2. “冻结基座 + 演化外围”的收益都以基座能力为前提；跨基座的收益是否单调（Harness Updating ≠ Harness Benefit 的问题）没有系统报告。
3. 技能库维护（PRACTICE、SkillGLOW）的验证仍是任务成功率；技能之间的一致性、冲突与可审计性缺少专门指标。

4. 关系定位与延伸批判

这六篇加上 ASPIRE（notes/04）与 SkillOpt（notes/05）形成 T7 的完整谱系： z 从技能文档（SkillOpt）→ 技能库（ASPIRE、PRACTICE、SkillGLOW）→ 元技能（HiSME）→ 运行时 critic 与恢复技能（Zetta）→ 整套 harness（SHAPER）， A 被允许改的东西越来越靠近执行时刻。Zetta 的“动作频率治理”把 harness 拉进了实时系统的领域，与 Harness Engineering for Physical AI 的 Projection/Isolation/Transfer（notes/23）是同一问题的两种表述。最值得追问的是**验证门**：SkillGLOW 的 commit gate、Zetta 的验证门控更新、PRACTICE 的分层整合都以“真实执行不劣化”为准，但这些执行都在仿真里；当验证要付出真机成本时，谁来承担 held-out rollout？这是 T7 与 T11（数字孪生作为验证器，notes/30）必须合流的原因。

08 · 自动研究谱系：Eureka → DrEureka → HARBOR / Nautilus / AgenticRobotics / Push-T 重访

覆盖：Eureka (arXiv 2310.12931, ICLR 2024, NVIDIA · UPenn · Caltech · UT Austin) · DrEureka (2406.01967, RSS 2024) · HARBOR (2606.08610, 2026-06) · Nautilus (2605.11665, 2026-05) · AgenticRobotics (2608.07555, 2026-08) · Revisiting Push-T with Agentic Robotics (2608.18227, 2026-08, Goldberg 组) 所属主线：T2、T9、T11、T24 · 材料来源：具身纪元文章"自动研究"节；报告第 3.3、6.2 节

1. 一句话定位

"让 AI 改进制造机器人能力的方法"这条线比 ENPIRE 早三年就开始了：Eureka (2023) 让 LLM 写奖励函数、RL 学策略、结果反馈回 LLM 改奖励；DrEureka (2024) 让 LLM 同时写域随机化的参数范围，把自动研究推进到 sim-to-real；2026 年 HARBOR 把仿真 RL 的整条工程流水线 harness 化，Nautilus 把"复现/评测/微调/部署"做成一句 prompt，AgenticRobotics 把策略改进做成可以让人离开的控制平面，Goldberg 组的 Push-T 重访则展示了 coding agent 在仿真里"不要示范也能赢"。具身纪元文章把这条线归入 Robot RSI 矩阵的"自动研究"环节。

2. 各篇要点

Eureka：大模型编写奖励函数代码，RL 在仿真中训练策略，实验结果反馈给大模型修改奖励。在 83% 的测试任务上超过人工设计的奖励；标志性演示是五指机械手转笔。它的分工——LLM 决定"什么动作值得奖励"，连续控制仍由 RL 学出——正是后来 ENPIRE 混合方案（代码决定阶段、VLA/RL 做接触）的雏形。

DrEureka：在自动写奖励之外，让 LLM 设置摩擦、质量、外力等仿真参数的变化范围，训练能适应多种物理条件的策略；四足机器人在仿真里学会站上瑜伽球并带球移动，随后迁移真机。它回答的是"自动研究出来的能力如何在现实中成立"。

HARBOR：把机器人 RL 自动化定义为 harness 工程问题——给定仿真代码库与任务规格，自动化从环境搭建到策略训练的全流程；高层目标被分解为有界阶段，由专门 agent 通过标准化命令、持久产物、可执行门与可复用知识执行，并用去中心化并行试验与跨运行的经验学习扩展迭代。6 个基准 16 个任务（操作、行走、双臂灵巧）。它是 Lil'Log 三模式（工作流自动化、文件系统即记忆、子代理）在机器人 RL 工程上的完整实例。

Nautilus：诊断是机器人学习研究在策略族、基准套件与真机之间碎片化，通用 coding agent 缺少机器人研究的过程先验与验证习惯；提供带蒸馏先验的 agent 技能集、策略/仿真/真机之间的类型化契约、统一接口与执行环境、每个里程碑自动验证的可信 agentic 工作流。

AgenticRobotics (You Don't Need To Stay in The Loop)：把 Claude Code / Codex 式的"主 agent 管循环、子 agent 分析执行、工具干活"架构移植到策略改进，核心差异是机器人工具（训练好的策略、训练流水线、数据采集）会经常失败，因此每次调用都要测量、记录工具质量，产物变化时过期。设计：不可变目标、控制器拥有的测量、commit 键控的崩溃恢复、证据分级技能库、带标准化记录调用面的工具注册表。"人可以离开"是操作性声明：晋升是证据门控的、状态可恢复、能力质量由记录导出。

Push-T 重访：Claude Code (Fable 5) 在没有任何示范的情况下，自己找到 2D gym 仿真、用仿真实验学习推的力学、迭代优化，达到 100% 成功且比用 200 条人类示范训练的最佳扩散策略少 46% 步数；还用自生成

课程解决 Push-A 到 Push-Z，并生成 Franka 与 UR5 的 3D 跨本体仿真代码。与 ENPIRE 真机 Push-T 的 60-95% 对照，这是"仿真里 coding agent 已经赢了，真机上还没有"的最清楚证据。

3. 口径与局限

- Eureka 的 83% 是"超过人工奖励的任务比例"，不是成功率；DrEureka 的真机结果是演示级。
- HARBOR、Nautilus 都在仿真/工程层验证，摘要没有真机策略成功率。
- AgenticRobotics 的关键数字是运行层的（硬化后假阳性晋升率 0.001），不是任务成功率。
- Push-T 重访是短文，100% 为仿真、脚本判定。

4. 关系定位与延伸批判

把这六篇与 ENPIRE (notes/06) 放在一条线上，可以看到自动化的对象逐级上移：奖励函数 (Eureka) → 仿真参数 (DrEureka) → 整条训练流水线 (HARBOR) → 研究 workflow (Nautilus) → 真机改进循环 (ENPIRE) → 让人离开的控制平面 (AgenticRobotics)。这正是 Lil'Log 优化阶梯从 context 到 optimizer code 的机器人版本。两个未走之路：其一，Eureka 式"LLM 写奖励"与 2026 年"LLM-as-a-Verifier 当密集奖励" (notes/19) 尚无直接对照——前者写的是奖励的代码，后者用模型当奖励，谁更抗奖励作弊值得一测；其二，AgenticRobotics 的"证据分级技能库"与 ASPIRE 的"验证过的修复"是同一概念的两种实现，把证据等级作为技能检索的排序信号还没有人做。

09 · 技能库谱系：从 LLM 提任务到形式化可验证的技能契约

覆盖：Agentic Skill Discovery (arXiv 2405.15019, 2024-05) · Atomic Skill Library (2501.15068, 2025-01) · ViReSkill (2509.24219, 2025-09) · Growing with Your Embodied Agent (2509.18597, 2025-09) · Neuro-Symbolic Code-as-Policies (2510.21302, NeurIPS 2025 Spotlight) · RATs / Playful Agentic Robot Learning (2606.19419, 2026-06) · VASO (2606.05395, 2026-06) 所属主线：T8、T2、T6、T17 · 材料来源：检索地图 T8 代表工作；报告第 4 章 T8

1. 一句话定位

技能库这条线回答三个问题：技能从哪来、怎么存、怎么信。2024-2025 年的工作解决"从哪来"（LLM 提任务 + RL 学 + VLM 验：Agentic Skill Discovery；VLP 拆子任务 + VLA 微调：Atomic Skill Library；失败重规划、成功即存：ViReSkill；人类纠正编码为技能：Growing）；2026 年的工作开始解决"怎么信"（RATs 用步级验证与失败诊断筛技能；VASO 用模型检查替代轨迹级证据）。

2. 各篇要点

Agentic Skill Discovery：完全由 LLM 驱动的技能发现——LLM 根据场景描述与机器人配置提出任务，为任务采样奖励函数与成功判定函数，RL 学策略，独立的 VLM 验证；技能库从零生长。它把 Eureka 式"LLM 写奖励"用到了技能库的构建上，并指出了 bottom-up 组合的盲区：只含"推"的库永远长不出"抓"。

Atomic Skill Library：三轮数据驱动流程——Vision-Language-Planning 把任务拆成子任务、抽象出原子技能定义、采集数据并微调 VLA 构建技能库；库随三轮更新动态扩展，覆盖的任务范围随之增长。这是"技能 = VLA 微调出的原子动作"的路线，与"技能 = 代码"的路线形成对照。

ViReSkill：诊断 LLM/VLM 规划的两个障碍——符号计划很少接地到场景几何与物体物理、相同 prompt 输出不稳定。方法：失败时基于当前场景重规划；成功时把执行过的计划存为技能，下次直接重放而不再调用 LLM/VLM。LIBERO、RLBench 与真机评测。它是 L5 技能演化里"最便宜"的形态：技能就是一次成功的计划。

Growing with Your Embodied Agent：人在环路的终身代码生成——把纠正编码为可复用技能，外部记忆 + RAG + hint 机制动态复用；在 Ravens、Franka Kitchen、MetaWorld 与真机上评测，解决需要 20 个以上原语的"盖房子"任务。它承认 LLM 在极长程任务上的推理不足，用人来补。

Neuro-Symbolic Code-as-Policies：在代码生成中加入显式符号验证与交互式验证——生成探索性代码主动与环境交互获取缺失观测，同时保持任务相关状态；比 CaP 基线成功率 +46.2%（RLBench 与真机、动态与部分可观测场景）。它是"验证进入技能生成过程"的早期形态。

RATs (Playful Agentic Robot Learning)：在下游任务到来前用自主提出的探索任务"玩耍"——提出新颖但可学的任务、规划并执行代码策略、验证中间进度、诊断失败、用密集步级反馈重试、把成功执行蒸馏进持久代码技能库；测试时从冻结库检索技能。LIBERO-PRO 比 CaP-Agent0 +20.6 个百分点，技能可直接检索进其他 Code-as-Policy agent 的上下文。

VASO：主张基础模型压低了创建技能的成本，却没有压低信任技能的成本——执行反馈、单元测试、环境奖励、LLM 自评都只是轨迹级证据。方法：每个技能是带两个接口的语义契约（形式接口把状态/观测/控制命令

对齐到逻辑命题供模型检查；面向规划器的接口引导可执行行为生成)；模型检查器先过滤逻辑不一致的契约，再验证契约诱导的计划是否满足时序安全约束，反例变成文本梯度更新契约；97.2% 规范符合率。

3. 口径与局限

- Agentic Skill Discovery 与 Atomic Skill Library 的数字为各自仿真/真机设置，摘要未给统一指标。
- ViReSkill 的收益取决于"同一场景再次出现"的频率，重放对场景变化的鲁棒性未量化。
- RATs 的 +20.6pp 对比对象是 CaP-Agent0 (同一原语族)，是本线少有的可比数字。
- VASO 的 97.2% 是"规范符合率"，不是任务成功率；形式接口需要人写命题与状态的对齐，这部分成本论文未量化。

4. 关系定位与延伸批判

技能库的两条路线——代码技能 (CaP 家族、ViReSkill、RATs、ASPIRE) 与权重技能 (Atomic Skill Library、Harness VLA 调用的 VLA 原语) ——在 2026 年的分工由 ENPIRE 的混合实验与 HyMeS 的口号"技能在权重里、记忆在代码里"给出 (notes/13)。本线最重要的转折是 VASO：技能库从"代码片段集合"变成"带验证证据与适用范围的知识库"，与 AgenticRobotics 的证据分级技能库、Harness VLA 的适用范围规则同向。未走之路：没有一篇工作把技能的**进入条件** (BATON 指出 VLA 原语只有退出条件，notes/14) 作为技能库的一等字段；把 VASO 的形式接口用于描述进入条件，是把 T8 与 T4 接起来的最短路径。

PART B

记忆、反思与验证

RoboMME 的两个结论、PonderPounce / AGM / HyMeS / BATON 四种记忆路线、权重内记忆合评、反思与纠错、PRIMO R1 与 VERITAS 两类验证器、验证器谱系。

1. 10 · RoboMME 与 RoboMME-Interference 深度解读：机器人记忆的评测基准与它的两个结论
2. 11 · PonderPounce 深度解读：用 MLLM 的原生因果上下文当机器人记忆
3. 12 · AGM 深度解读：只有物理证据才能推进进度指针
4. 13 · HyMeS 深度解读："技能在权重里，记忆在代码里"
5. 14 · BATON 深度解读：长程问题是技能交接问题
6. 15 · 权重内记忆与记忆选择合评：NativeMEM、LaMem-VLA、Remember Smarter、OnEvoMemory、Memory Anchors、概念中心记忆
7. 16 · 反思与纠错合评：REMAC、PhysReflect-VLA、Agentic RAG-VLM、PhysiAgent
8. 17 · PRIMO R1 深度解读：从"观察者"到"批评者"的过程级视频验证
9. 18 · VERITAS 深度解读：推理时视觉验证，验证过的 rollout 直接成为训练数据
10. 19 · 验证器谱系：LLM-as-a-Verifier、立场论文、Consilience、Agentic Harnesses

10 · RoboMME 与 RoboMME-Interference 深度解读：机器人记忆的评测基准与它的两个结论

RoboMME: Benchmarking and Understanding Memory for Robotic Generalist Policies, arXiv 2603.04639 (2026-03, ICML 2026) · Yinpei Dai, Hongze Fu, Jayjun Lee et al. (共 9 位作者) **RoboMME-Interference: Benchmarking Robot Memory Under Interference**, arXiv 2606.22338 (2026-06) · Soumil Rath 所属主线: T5 · 材料来源: 检索地图 T5 代表工作; 小红书长文 01 节

1. 一句话定位

RoboMME 是第一个大规模、标准化的"记忆依赖操作"基准: 16 个任务按时间、空间、物体、程序四类记忆的分类构建, 配 14 种记忆增强的 $\pi 0.5$ 变体做系统比较; 结论是**记忆表示的有效性高度任务依赖, 没有一种设计全面占优**。RoboMME-Interference 把它扩到跨会话: 在查询任务的相关示范之后插入受控数量的无关会话, 测记忆在干扰下的衰减; 结论是感知型记忆无干扰时最好但随干扰稳定衰减, 子目标型记忆提升较小但更抗干扰, 加一个按视觉相似度检索历史片段的步骤能在每个干扰级别恢复无干扰时的成功率。

2. 要解决的问题

长程、依赖历史的任务(数重复动作、操作暂时被遮挡的物体)要求策略记住过去; VLA 开始加记忆机制, 但评测局限在狭窄、非标准的设置里, 无法系统比较与度量进展。真实部署中机器人还会跨多个会话积累经验, "多个会话之前的信息"如何被记住与检索, 此前几乎没有基准。

3. 方法

RoboMME: 16 个操作任务, 分类覆盖时间记忆(顺序、计数)、空间记忆(位置)、物体记忆(遮挡后的身份)、程序记忆(步骤进度); 14 种记忆变体在同一 $\pi 0.5$ 基座上实现, 控制训练配方以比较记忆表示本身。RoboMME-Interference: 对每个查询 episode 构造会话历史——相关的先验示范 + 受控数量的无关会话——作为记忆提供给 VLA, 测准确率随干扰数量的变化; 并测试检索(按视觉相似度选最相关历史片段)作为缓解手段。

4. 实验结果与口径

- RoboMME: 14 种变体没有一种在四类记忆上全面领先("任务依赖"); 具体数字见论文表格, 摘要不给单一头条数字。
- RoboMME-Interference: 感知型记忆在零干扰时最好, 随无关会话数增加稳定下降; 子目标型记忆提升幅度小但衰减慢; 检索步骤在每个干扰级别都能恢复到接近无干扰的水平。
- 口径: 仿真、脚本判定的成功率; 基座固定为 $\pi 0.5$; "干扰"为无关会话的数量, 不含对抗性干扰。

5. 局限

1. 单一基座($\pi 0.5$)上的比较结论是否迁移到其他 VLA 未知。
2. 记忆类型分类是人为设计的四类; 真实任务往往同时需要多类记忆, 交叉效应未单独评估。

3. 干扰是"无关会话", 不包含误导性或对抗性内容——多机器人共享经验时的污染问题（报告 2.6 节）在此基准之外。

6. 关系定位

RoboMME 是本仓库 T5 主线上最可靠的经验事实来源：它让"记忆"从各说各话变成可比较。PonderPounce (notes/11) 用它报告 60.83% vs 17.93%，AGM (notes/12)、HyMeS (notes/13) 在其衍生基准 RoboMemArena 上报数。RoboMME-Interference 的"检索可恢复"结论支持报告 I4 的选择规则：跨会话、抗干扰的需求用结构化外部记忆 + 检索；精确时序与计数用权重内记忆。

7. 延伸批判

"记忆表示高度任务依赖"是一个诚实但令人不安的结论——它意味着没有免费的通用记忆模块，系统必须按任务选记忆，而选择本身需要一个判断"当前任务需要哪类记忆"的元层，这正是 HyMeS 让 coding agent 迭代记忆管理系统的动机。RoboMME-Interference 的下一步显然是把"检索"从视觉相似度换成语义/任务相关性，并把干扰从无关会话换成相似但错误的会话——那才是部署中最常见的记忆失效方式。

11 · PonderPounce 深度解读：用 MLLM 的原生因果上下文当机器人记忆

PonderPounce: A Pretrained MLLM as an Episode Context Engine for Robot Control arXiv 2608.24115

(2026-08-25) · Suhwan Choi, Jaeyoon Jung, Sungkyung Kim et al. (共 5 位作者) 所属主线: T5、T13 · 材料来源: 小红书长文 01 节 ("用 MLLM 当 episode context engine")

1. 一句话定位

PonderPounce 不设计专门的记忆模块：一个 System 2 的 MLLM (Ponder) 在自己的原生因果上下文里累积整段 episode 的观测、示范和先前认知，可以生成子目标文本与示范推理供内部使用；System 1 的 VLA (Pounce) 直接接收当前观测、指令与本体感知，并通过 Ponder-Pounce 接口异步地只收到最新的一个认知 token 及其"年龄"。RoboMME 上 9B 版本 60.83%、0.8B 版本 50.04%，对比只看当前观测的 $\pi 0.5$ 17.93%；认知刷新 p50 延迟 78 ms、动作模型 25 ms，支持 20 Hz 播放。

2. 要解决的问题

MLLM 本来就能整合长视觉历史、在部分可观测下推理、从几个例子推断行为，但 VLA 继承预训练表征时没有把这种上下文能力用作 episode 记忆；已有的记忆依赖策略靠专门设计的历史机制。PonderPounce 问：能不能直接复用 MLLM 的因果上下文当记忆，而不另造模块？

3. 方法

双系统 + 异步接口。Ponder 以低频运行，上下文里堆积 episode 的全部观测与示范，输出"认知"（子目标、对示范的推理）；Pounce 以高频运行，只额外接收一个认知 token 和它距今多久（年龄），因此动作频率不受 Ponder 延迟拖累。认知的"年龄"让 Pounce 知道这条指导有多陈旧。

4. 实验结果与口径

- RoboMME：60.83% (9B Ponder)、50.04% (0.8B Ponder)，对比当前观测 $\pi 0.5$ 的 17.93%。口径同 RoboMME（仿真、脚本判定）。
- 延迟：认知刷新 p50 78 ms，动作模型 25 ms，支持 20 Hz 播放——这组数字说明"用大模型当记忆"没有牺牲控制频率。
- 与 14 种 RoboMME 记忆变体的直接对比摘要未给出；60.83% 与它们的关系需看正文。

5. 局限

1. 上下文长度是硬上限：整段 episode 的观测都进上下文，跨会话 (RoboMME-Interference 的设置) 如何处理未在摘要中说明。
2. 只传"最新一个认知 token"是很强的信息瓶颈；哪些任务因此受损（例如需要多个并行约束的任务）未分析。

3. 9B 与 0.8B 之间 10 个点的差距说明记忆质量随 Ponder 规模变化，端侧部署时的取舍未讨论。

6. 关系定位

PonderPounce 同时属于 T5（记忆）与 T13（双系统）：它是"记忆 = 慢系统的上下文"这一路线的代表，与 NativeMEM 等"记忆 = 权重内 token"（notes/15）、AGM/HyMeS 等"记忆 = 外部结构"（notes/12、13）构成三条路线。它的异步接口与 StreamVLA、LaST0 的低频推理/高频动作分工（notes/26）同构，认知"年龄"字段是小红书长文"Real-time-aware Harness"思想的一个微观实例。

7. 延伸批判

PonderPounce 最有趣的地方是它证明了"不造记忆模块也能拿到记忆收益"，但这把记忆的可解释性与可编辑性也一起放弃了——上下文里的历史无法像 AGM 的进度指针那样被验证、像 HyMeS 的代码记忆那样被审查。报告 I4 的分工建议在这里具体化为一个实验：同一 RoboMME 任务上，PonderPounce（隐式上下文记忆）与 AGM（显式证据门控记忆）在干扰会话下谁衰减更慢。另一个未走之路：Ponder 的认知是文本，能否让它输出 BATON 式的"进入条件"检查而不只是子目标？

12 · AGM 深度解读：只有物理证据才能推进进度指针

AGM: Achievement-Grounded Memory for Closed-Loop Agents with Frozen VLA Policies arXiv 2608.29537
(2026-08-30) · Hongbo Gao, Zeyu Ni, Xin Wen et al. (共 5 位作者) 所属主线: T5、T23 · 材料来源: 报告第 2.1、4 章 T5/T23、第 7 章 I3

1. 一句话定位

AGM 给冻结 VLA 配一个轻量的闭环框架：把任务表示为带进度指针的子目标序列，**只有当前子目标被物理证据验证后才推进指针**；本体感知的交互线索决定何时验证，连贯的点跟踪与语言条件的跨视角比较决定达成了什么；验证头只有 2.43M 参数。论文的结论很硬：可靠的具身记忆靠的是状态更新纪律，不是记忆容量。

2. 要解决的问题

冻结 VLA 提供广泛的操作技能，但开环地执行动作块、不跟踪任务进度，agent 无法可靠决定继续、重试还是终止。外部记忆是自然的补救，却可能有害——如果"尝试过的动作"被当作"完成的进度"，局部执行错误就变成持久的任务状态错误。

3. 方法

记忆 = 子目标序列 + 进度指针。三个决策：**何时验证**——用本体感知的交互线索（接触、力、运动停止等）触发；**验证什么**——点跟踪（物体是否移动到位）与语言条件的跨视角比较（两个视角下的场景是否满足子目标描述）；**如何推进**——只有验证通过才推进指针，否则重试或终止。验证头 2.43M 参数，可以跑在控制回路里。

4. 实验结果与口径

- 摘要给出的核心主张是与"把尝试当完成"的记忆基线相比，AGM 避免了任务状态错误的累积；具体基准与数字见正文（RoboMemArena 系列，见 HyMeS/BATON 同期工作）。
- 口径：冻结 VLA、外部记忆、验证器为小模型；成功率为任务级。

5. 局限

- 子目标序列来自何处（LLM 规划还是人写）决定了 AGM 的上限；摘要未说明规划错误时如何修正序列本身。
- 点跟踪与跨视角比较对遮挡、非刚体（布料、液体）的鲁棒性未知；验证器的假阳性/假阴性率没有单列。
- "只有验证后才推进"在验证器保守时会导致卡死或过度重试；终止策略的设计摘要未展开。

6. 关系定位

AGM 是 T5 与 T23 的交点：它把记忆问题转化为验证问题。与 PonderPounce（隐式上下文，notes/11）相比，AGM 的记忆是最小、最可审计的结构；与 Thea 的 Evaluation as Exit Codes（notes/22）和 PhyAgentOS 的 SessionVerifier（notes/21）同属"把'成没成'做成显式判断"的 2026 年潮流。它的结论"纪律 > 容量"直接支持报告 I3（验证器是瓶颈）与 I4（结构化外部记忆的适用区）。

7. 延伸批判

AGM 的验证器只有 2.43M 参数，这是它最可贵也最可疑的地方：小验证器便宜、可以高频运行，但能判断的"达成"类型有限（位置到位、视角一致），对"拧紧了没有"、"插到底了没有"这类接触语义无能为力——而这正是 ENPIRE 用两台相机判断扎带是否穿过锁头、PRIMO R1 用 7B 视频模型判断进度（notes/17）要解决的。一个自然的组合是分层验证：AGM 的小验证器做高频门控，PRIMO R1 式的大模型做低频复核，LLM-as-a-Verifier（notes/19）给连续分数——这正是报告开放问题第 1 条。

13 · HyMeS 深度解读："技能在权重里，记忆在代码里"

Skills in Weights, Memory in Code: Hybrid Learning for Memory-Dependent Robot Manipulation (HyMeS)
arXiv 2608.09410 (2026-08-10) · Yunhao Zhao, Zhenyang Ni, Haoyang Chen et al. (共 5 位作者) 所属主线：
T5 · 材料来源：报告第 2.1 节、第 7 章 I2/I4

1. 一句话定位

HyMeS 提出一个可操作的分工假设：低层运动技能用梯度法从模仿数据学进权重，高层的记忆管理策略由 coding agent 通过启发式学习（heuristic learning）迭代一个**可执行的记忆管理系统**来获得，再用它引导马尔可夫的 VLA 完成依赖记忆的操作。RoboMemArena 任务成功率 41.3% → 60.1%。

2. 要解决的问题

现代 VLA 从当前观测或固定短历史生成动作块，但真实操作往往是非马尔可夫的：下一步取决于长程交互历史里的任务相关信息（哪些子任务已完成、物体被移到了哪里）。把记忆学进权重需要大量长程数据；把记忆做成固定规则又不能适应新任务。

3. 方法

两条学习路径并行：VLA 用基于梯度的模仿学习学运动技能；coding agent 通过迭代修改代码（启发式学习——讲稿第 2 页定义的那种“更新对象是软件结构而非参数”的学习）获得高层记忆管理策略——记什么、何时更新、如何用记忆条件化 VLA 的输入或调用。记忆系统是代码，因此可读、可测试、可版本化。

4. 实验结果与口径

- RoboMemArena：任务成功率 41.3% → 60.1%（对比无记忆管理的基座）。口径：仿真基准、任务级成功率；RoboMemArena 是 RoboMME 家族的竞技场版本，具体任务组成需看正文。
- coding agent 的迭代轮数、每轮代价摘要未说明。

5. 局限

1. 启发式学习的收敛没有理论保证：coding agent 可能在记忆规则空间里过拟合训练任务。
2. "记忆在代码里"要求记忆的内容能被符号化（子任务完成、物体位置）；连续的接触状态记忆（例如"上次插入时偏了 3 mm 的感觉"）不在这个假设之内——这正是小红书长文 USB 例子指出的天花板。
3. 与权重内记忆（NativeMEM 等，notes/15）在同一基准上的直接对照缺失。

6. 关系定位

HyMeS 是报告 I2（冻结 VLA + 外围学习）与 I4（记忆两条路线分工）的交点，也是讲稿"Heuristic Learning / Heuristic System"定义在记忆问题上的具体实例：被维护的 Heuristic System 就是那份记忆管理代码。与

AGM (notes/12) 相比, AGM 的记忆结构是人设计的 (子目标 + 指针), HyMeS 的记忆结构是 agent 演化出来的; 与 PonderPounce (notes/11) 相比, HyMeS 的记忆可审计。

7. 延伸批判

"技能在权重里、记忆在代码里"是一个漂亮的口号, 但边界在哪里? 报告开放问题第 3 条把它写成实验: 用 RoboMME 的四类记忆分类测哪些记忆类型放代码里有效 (程序记忆、物体身份), 哪些必须进权重 (精确时序、连续量)。另一个未走之路是把 HyMeS 的代码记忆与 SkillOpt 的训练纪律 (held-out 严格提升门、拒绝缓冲) 结合——现在的启发式学习没有说明它如何防止记忆规则"看起来聪明却没有改进"。

14 · BATON 深度解读：长程问题是技能交接问题

Don't Drop the BATON: Long-Horizon Robot Manipulation via Agentic Subtask Exploration and Transition-Aware Memory arXiv 2608.16889 (2026-08-17) · Bingxin Xu, Yuzhang Shang, Emilio Ferrara 相关：

AtomBridge (2602.09430, 2026-02) · Foresight Residual RL (2607.16506, IROS 2026) 所属主线：T4、T5

· 材料来源：报告第 4 章 T4、第 7 章 I2、第 8 章开放问题 2

1. 一句话定位

BATON 指出“冻结 VLA + LLM agent”这一流行配方在长程任务上会坏两次：(1) 能力来自测试时的整任务探索，代价是阶段数的指数——一个阶段需要 T 个 episode， K 个阶段就需要约 T^K ，且一次失败不揭示是哪个阶段的错；(2) VLA 原语有退出条件却没有进入条件，前一个子任务会悄悄地约束下一个。BATON 以子任务为探索单元（代价变为 $T \cdot K$ ），并用转移感知记忆治理三种转移——调用、交接、前瞻；RoboMemArena 任务成功 +11.6%，不更新任何参数。

2. 要解决的问题

VLA 越来越会做单个接触密集技能，但把技能串成多阶段任务仍然失败：误差累积超出策略的纠正能力，阶段之间存在隐性约束。LLM agent 在语言里规划、用解析原语在自由空间移动、只在接触段调用 VLA、把适应写进语言记忆——这个配方在短任务上有效，在长程上失效于探索代价与交接条件。

3. 方法

- **子任务级探索**：把测试时探索的单位从整任务改为子任务，使探索代价从乘法变加法，且失败可归因到阶段。
- **转移感知记忆**：记录并治理三类转移——调用（何时把控制交给 VLA）、交接（前一子任务的终态是否满足下一子任务的进入条件）、前瞻（为下游子任务预先安排当前子任务的终态）。
- **参数冻结**：VLA 与 LLM 都不更新。

4. 实验结果与口径

- RoboMemArena 任务成功率 +11.6%（相对同一冻结 VLA + LLM agent 配方的基线）。口径：仿真、任务级成功率、参数不更新。
- 相关工作的数字：AtomBridge 在原子技能边界用 LLM 生成过渡动作代码，8 步任务成功率 +10–25%（科学实验室场景）；Foresight Residual RL 用离线估计的“下游子任务成功概率”塑形每个子任务的稀疏奖励，三阶段装配 85.6% 对比 54.5%。

5. 局限

1. 子任务划分来自 LLM 的语言规划；划分错误时“子任务级探索”的加法优势不再成立。
2. 转移感知记忆是语言记忆，交接条件以语言描述——几何/接触层面的进入条件（例如夹爪与物体的相对位姿分布）需要 Foresight Residual RL 那样的数值量。

3. +11.6% 的绝对水位摘要未给；RoboMemArena 上不同工作（HyMeS 60.1%、BATON +11.6%）的基线与协议是否一致需核对。

6. 关系定位

BATON 把 T4 长程主线重新表述为"技能交接问题"，这是报告第 4 章 T4 的核心判断。三篇工作从三个层面处理交接：BATON 在语言记忆层治理转移，AtomBridge 在技能边界插入过渡代码，Foresight Residual RL 在 RL 奖励层塑形交接状态质量。它们共同指出 Harness VLA（notes/20）学到的"什么时候该先 MOVE_TO"其实就是一种进入条件的近似。RoboHarness（notes/24）的 Memory Bridge——切换前把机器人带到下一策略熟悉的状态——是同一问题的第四种答案。

7. 延伸批判

$T^K \rightarrow T \cdot K$ 是本仓库最清晰的一个复杂度论证，但它假设子任务之间可以独立探索；当交接约束很强时（下一子任务的可行性强依赖上一子任务的终态），子任务不再独立，探索代价会回到某种乘法形式——这正是 Foresight 项存在的理由。开放问题（报告第 8 章第 2 条）：为每个原语学显式的进入条件与交接质量值，让技能库（notes/09）把进入条件作为一等字段；VASO 的形式接口是描述这些条件的现成语言。

15 · 权重内记忆与记忆选择合评：NativeMEM、LaMem-VLA、Remember Smarter、OnEvoMemory、Memory Anchors、概念中心记忆

覆盖：NativeMEM (arXiv 2607.06678, 2026-07) · LaMem-VLA (2607.07608, 2026-07) · Remember Smarter (2608.15269, 2026-08) · OnEvoMemory (2608.08749, 2026-08, ECCV 2026 Workshop) · Memory Anchors (2608.26545, 2026-08) · Analytic Concept-Centric Memory (2606.29774, 2026-06) 所属主线：T5 · 材料来源：报告第 2.1 节、第 7 章 I4

1. 一句话定位

与 AGM/HyMeS/BATON 把记忆放在 VLA 外面相反，这一组工作把记忆放进 VLA 的表征空间：NativeMEM 复用 VLA 自己的视觉编码器把每帧压成一个 token (成功率 32.4% → 84.0%)；LaMem-VLA 把短期/长期记忆库重建为潜记忆 token 并与 VLA 推理交织；Remember Smarter 用 Mamba 压缩视觉历史并用双曲空间存经验 (LIBERO-Plus 53.6% → 70.6%)；OnEvoMemory 用价值引导从在线 rollout 学"该保留哪些经验"；Memory Anchors 发现回放缓冲里约 10% 的锚点经验决定是否灾难性遗忘；概念中心记忆则用部件、参数模板、位姿、affordance 的结构化概念组织外部记忆。

2. 各篇要点

NativeMEM：问题是预训练 VLA 如何以高频更新保留长程视觉历史又不牺牲效率——外部记忆管理限制了记忆范围或反应性。方法：Native Memory Compression，用 VLA 自身视觉编码器把每个相机视角的每帧历史压成单个 token 追加到输入序列，延迟开销可忽略，不需要外部规划器或新初始化的模块。数字：32.4% → 84.0% (基准见正文)。

LaMem-VLA：诊断现有记忆增强 VLA 要么扩观测窗、要么从记忆库检索历史作为策略侧辅助上下文，都把记忆留在 VLA 原生潜空间之外。方法：把历史重建为潜记忆 token 直接与 VLA 推理交织，四个协调组件 (curator 等，摘要截断)。

Remember Smarter：即插即用模块，两个分支——视觉分支用双向空间 Mamba + 因果时间 Mamba 压缩多视角 patch 历史，经残差交叉注意力暴露给面向动作的隐状态，不改变 VLM 视觉 token 流；经验分支把成功的 VLM 末层状态存入 Poincaré VAE 空间、分层组织、异步转成测地线 prompt token 而不阻塞动作推理。LIBERO-Plus 53.6% → 70.6%。

OnEvoMemory：价值引导的记忆模块，维护近期上下文、高价值经验与显著转移，从轨迹结果学"该保留什么"；离线示范初始化记忆先验，在线成功/失败 rollout 精炼选择，帮助策略识别任务阶段转移、避免重复已完成子任务。

Memory Anchors：持续学习视角。回放缓冲通常从全部旧经验随机采样，但一小部分经验对锚定旧性能贡献极大——它们位于新任务观测的表征塌缩到旧任务观测、而两个任务需要相反动作的区域 (熟悉物体要以新方式操作)。约 10% 的锚点经验决定是否遗忘，去掉它们遗忘增加 4.5 倍 (报告第 4 章 T5 转述)。

Analytic Concept-Centric Memory：外部记忆但结构化——物体由语义部件、参数模板、接地位姿、affordance、操作状态表示；物体/场景记忆与转移记忆 (动作引起的状态变化)、技能记忆 (模板接地的可执

行技能) 相连。它介于"权重内"与"纯文本记忆"之间, 是可检索、可解释的中间形态。

3. 口径与局限

- 32.4→84.0、53.6→70.6 分别来自不同基准与基座, 不可并排; 与 PonderPounce 的 60.83% (RoboMME) 也不在同一协议。
- 权重内记忆都需要(再)训练, 与"冻结 VLA"路线的前提不同; 训练数据里的长程历史覆盖决定上限。
- Memory Anchors 讲的是"不忘旧任务", 与"记住本 episode 历史"是不同意义的记忆, 放在一起是因为都涉及"该保留什么经验"。

4. 关系定位与延伸批判

报告 I4 的选择规则来自这组工作与 RoboMME-Interference 的对照: 需要精确时序、计数、低延迟的记忆放权重内; 需要跨会话、抗干扰、可解释、可编辑的记忆放结构化外部记忆。NativeMEM 的"复用自身视觉编码器"与 PonderPounce 的"复用 MLLM 上下文"是同一思想(不造新模块)的两个方向。最缺的实验是**同一基座、同一基准上权重内 vs 权重外的直接对照**——RoboMME 的 14 种变体已经包含若干权重内设计, 把 AGM/HyMeS 接进同一套评测是最短路径。Memory Anchors 的发现对 Fleet Learning (T21) 有直接含义: 群体回流的数据若淹没了锚点, 会让共享策略在旧任务上退化——LWD 的 16 台机器人是否遇到过这个问题, 论文没有报告。

16 · 反思与纠错合评：REMAC、PhysReflect-VLA、Agentic RAG-VLM、PhysiAgent

覆盖：REMAC (arXiv 2503.22122, 2025-03) · PhysReflect-VLA (2606.27146, 2026-06) · Agentic RAG-VLM (2606.31200, 2026-06) · PhysiAgent (2509.24524, 2025-09) 所属主线：T6、T4、T19、T23 · 材料来源：检索地图 T6 代表工作；小红书长文 02 节的 L1-L7 分层

1. 一句话定位

小红书长文给了一个判断标准："Retry ≠ Learning"——反思的价值取决于它能否被写进持久对象。这一组工作展示了反思在 L2-L3 的四种形态：REMAC 用前置/后置条件检查与场景推理做多机器人长程重规划（成功率 +40%）；PhysReflect-VLA 在执行时给 VLA 加物理可行性评估与结构化自反思（接触密集真机任务平均 +5.4%）；Agentic RAG-VLM 用 14 类失败分类与三级自适应重试把杂乱场景抓取从 25% 提到 78.3%；PhysiAgent 让 VLM 根据 VLA 的实时熟练度反馈组织 monitor、memory、reflection 组件。

2. 各篇要点

REMAC：面向多机器人长程操作的自适应规划：前置条件与后置条件检查让每一步执行前后都被验证，场景特定的自进化让规划随环境反馈改进；报告数字为成功率 +40%（相对基线）。它同时属于 T4（长程）、T19（多机器人）、T23（验证器）——前/后置条件是验证器最古典的形式。

PhysReflect-VLA：即插即用的执行时可靠性框架。可行性算子评估候选动作是否产生动力学一致的状态转移；动作解释算子核查转移一致性；LLM 反思模块分析状态差异、生成纠正指令回灌控制回路。数字：接触密集真机任务平均 +5.4%。它把"反思"细化到了单步动作层面，比 episode 级反思更接近控制。

Agentic RAG-VLM：面向杂乱环境抓取。诊断：VLM 抓取方法靠视觉相似度匹配物体，忽略把手可抓性、材料脆性等物理 affordance，且开环、无空间推理与失败恢复。方法：分层 affordance 感知的 RAG + VLM 语义理解 + agentic 自反思规划，14 类失败分类与三级自适应重试。数字：杂乱场景抓取 25% → 78.3%。

PhysiAgent：诊断 VLM + VLA 的"僵硬串联"（VLM 只做高层理解与规划、VLA 只当执行器）导致协作低效与接地差。方法：monitor、memory、self-reflection 机制 + 轻量现成工具箱构成自主脚手架，VLM 根据 VLA 的实时熟练度反馈组织这些组件。它是 2025 年"Agent 围绕 VLA"配方的一个早期完整实例。

3. 口径与局限

- 四篇的数字分别是相对提升（REMAC +40%、PhysReflect +5.4pp）与绝对成功率（Agentic RAG-VLM 25→78.3%），任务集与判定方式各不相同，不能并排。
- PhysReflect-VLA 的可行性算子需要某种动力学一致性模型，其来源与泛化性摘要未说明。
- Agentic RAG-VLM 的 14 类失败分类是人工定义的，覆盖面限于抓取。
- 四篇的反思结果是否跨 episode 持久化（写进记忆或技能）：REMAC 的自进化与 PhysiAgent 的 memory 有此意图，PhysReflect 与 Agentic RAG-VLM 主要是 episode 内纠正。

4. 关系定位与延伸批判

按 L1–L7 分层，PhysReflect–VLA 与 Agentic RAG–VLM 落在 L3（反思产生纠正指令），REMAC 与 PhysiAgent 触及 L4（记忆演化）。真正把反思写进持久对象的是 Harness VLA（学 VLA 的适用范围，notes/20）、ASPIRE（修复即技能，notes/04）与 AGM（进度指针，notes/12）。这组工作最值得追问的是**反思的成本–收益**：PhysReflect 的 +5.4pp 来自每步都跑三个算子，控制频率受多大影响未报告；Agentic RAG–VLM 的 78.3% 用了三级重试，重试次数计入成本后与单次成功率如何比较（同一问题见 Zeva 的 CSR@K 讨论）。反思类方法应当同时报告“每次成功的平均尝试数”，否则与更贵的 retry 无法区分——这一点应成为 T6 主线的口径规范。

17 · PRIMO R1 深度解读：从"观察者"到"批评者"的过程级视频验证

From Passive Observer to Active Critic: Reinforcement Learning Elicits Process Reasoning for Robotic Manipulation (PRIMO R1) arXiv 2603.15600 (2026-03) · Yibin Liu, Yaxing Lyu, Daqi Gao et al. (共 8 位作者) 所属主线：T23、T24、T6 · 材料来源：具身纪元文章"评估与仿真"节（点名 PRIMO R1）

1. 一句话定位

PRIMO R1 (Process Reasoning Induced Monitoring) 是一个 7B 的视频 MLLM 框架，把只会"识别正在发生什么"的被动观察者变成"评估当前状态相对最终目标进展到哪一步"的主动批评者：用基于结果的强化学习激励显式的思维链做进度估计，并把视频序列显式锚定在初始状态图与当前状态图之间构成结构化时序输入。具身纪元文章转述的例子：叠短裤时它能看出下摆已处理、上沿仍未完成，而不只判断最终画面像不像成功；RoboFail 失败检测准确率 67%。

2. 要解决的问题

长程操作的过程监督是关键瓶颈：当前视频 MLLM 主要以 SFT 训练，是被动的"观察者"，识别事件却不相对任务目标评估当前状态；于是它们能说"机器人在叠衣服"，却说不出"叠到了第几步、从哪一步开始错了"。没有过程级判断，自进化循环只能拿到二元的终态成功信号——正是 Let's Verify Step by Step 在 LLM 侧解决的同一问题。

3. 方法

- **结构化时序输入**：视频序列被显式锚定在初始状态图像与当前状态图像之间，让模型对"从哪里开始、现在到哪里"有明确参照。
- **结果驱动的 RL**：以最终结果为奖励，激励模型生成显式 CoT 来估计进度——推理过程不需要逐步标注。
- **PRIMO 数据集与基准**：配套数据与评测支持跨域内环境与域外环境的实验。

4. 实验结果与口径

- RoboFail 失败检测准确率 67%（具身纪元文章转述）。
- 摘要称在多样的域内与域外环境上有广泛实验；具体进度估计误差等指标见正文。
- 口径：判断对象是视频 + 两张锚定图，输出是进度估计与失败位置；67% 是失败检测准确率而非任务成功率。

5. 局限

1. 67% 意味着三分之一的失败没被检出或误报——作为自进化循环的 r 来源，噪声仍然很大；把它当选择门时需要 SkillOpt 式的"严格提升 + 拒绝缓冲"来抵消噪声。
2. 7B 视频模型的推理延迟决定它只能做低频复核，不能做 AGM 式的高频门控。

3. RL 只用结果奖励，CoT 的"过程推理"是否真的对应物理进度（还是语言上合理的叙述）需要像立场论文（notes/19）要求的受控变量评测。

6. 关系定位

PRIMO R1 补上了本仓库 T23 里缺席的一类验证器：过程级视频批评者。与 AGM（2.43M 小验证头，高频，几何证据）形成天然的分层；与 LLM-as-a-Verifier（连续评分、可作 RL 密集奖励）比较，PRIMO R1 专注机器人视频且经过 RL 训练。在具身纪元的矩阵里它属于"自我评估"环节，与 Let's Verify Step by Step（过程奖励模型）是同一思想在两个领域的版本。

7. 延伸批判

把评估器训练成"批评者"之后，下一个问题是谁来评估批评者。PRIMO R1 用 RoboFail 报告 67%，但一旦它进入自进化循环成为 r 的来源，策略就会向它的盲区优化（Goodhart）。Lil'Log 与 AHE 的答案是评估器只读、放在循环之外；Meta-Rewarding 的答案是让"评审的评审"也进循环。哪一条适用于机器人，取决于验证器的错误是否与策略可利用的模式相关——这个问题没有人在机器人上测过，是报告开放问题第 1 条的核心。

18 · VERITAS 深度解读：推理时视觉验证，验证过的 rollout 直接成为训练数据

Visual Verification Enables Inference-time Steering and Autonomous Policy Improvement (VERITAS) arXiv 2606.18247 (2026-06) · Princeton · Mingtong Zhang, Dhruv Shah · 项目页 veritas-improvement.github.io 所属主线：T23、T24、T9、T10 · 材料来源：具身纪元文章"评估与仿真"节

1. 一句话定位

VERITAS 是通用机器人策略的生成器-验证器框架：预训练的通用策略当"生成器"，配一个无梯度的"视觉验证器"在推理时评估候选动作，实现不训练的推理时引导；验证过的 rollout 又能作为离线策略改进的监督——在验证过的自生成轨迹上微调的策略持续提升，且后训练效率与专家示范相当而不需要人工干预。具身纪元文章的例子："把胡萝卜放到盘子上"，50 条自主轨迹训练后成功率 70%，同量人工示范 65%（每组评测 20 次）。

2. 要解决的问题

部署中的机器人应当从经验里学并随时间变好，这需要一个"练习并从反馈里学"的机制。通用策略的能力在，但缺少两样东西：推理时选出好动作的手段，以及不靠人工标注地把自己的 rollout 变成训练数据的手段。两者的共同前提是一个可靠的验证器。

3. 方法

- **生成器-验证器**：策略提出多个候选动作（或动作序列）；视觉验证器（无梯度、不训练）评估候选与目标的一致性——具身纪元文章的描述是 VLM 先画出目标路径，验证器选出最符合路径的一项。
- **推理时引导**：直接用验证器选择，不改权重，即可超过原始通用策略。
- **自主改进**：验证通过的 rollout 作为离线数据微调策略；无需人类示范或干预。

4. 实验结果与口径

- 推理时验证持续优于不加训练的原始通用策略（摘要）。
- 验证过的自生成轨迹微调带来持续增益；后训练效率与专家示范相当。具身纪元转述：50 条自主轨迹 70% vs 50 条人工示范 65%，每组 20 次评测。
- 口径：真机任务、二元成功率、每格 20 次试验——样本量小，70 vs 65 的差异在 20 次评测下不显著，应读作"相当"而非"更好"。

5. 局限

1. 验证器是 VLM 画路径 + 一致性匹配，对"路径正确但接触失败"（抓滑、插不到底）的判断力有限——与 PRIMO R1 的过程级判断互补。
2. 候选动作数与验证器调用次数决定推理时延迟；摘要未给频率。

3. 自主改进只用"验证通过"的轨迹，失败轨迹被丢弃——ASPIRE、Zetta 强调失败轨迹最有价值，Q-Planning (notes/29) 证明从失败学效果更好。

6. 关系定位

VERITAS 横跨 T23 (验证器)、T9 (VLA 后训练)、T10 (部署数据回流)：它是"验证器 → 数据筛选 → 权重更新"这条链最短的实现，也是 Q-Planning (只微调 Q 函数、从失败学) 的对照组——两者都用部署 rollout 改进策略，VERITAS 靠验证器筛成功样本做 SFT，Q-Planning 靠价值函数从成败中都学。在具身纪元矩阵里它同时属于"自我评估"与"训练时自迭代"。

7. 延伸批判

"50 条自主轨迹 $\approx$ 50 条专家示范"如果成立，意味着示范采集的人力可以被验证器替代——这是 RoboClaw (-53.7% 人工时间) 与 ENPIRE (自动复位) 追求的另一目标。但两个条件必须补上：任务数与试验数放大后的统计显著性，以及验证器的选择偏差 (只保留验证器"看得懂"的成功方式) 是否让策略多样性塌缩——这正是 Lil'Log RSI 七挑战里"多样性塌缩"在数据层面的形态。把 VERITAS 的验证器换成 PRIMO R1 或 LLM-as-a-Verifier，看筛出的数据分布如何变化，是一个便宜而有信息量的实验。

19 · 验证器谱系：LLM-as-a-Verifier、立场论文、Consilience、Agentic Harnesses

覆盖：LLM-as-a-Verifier (arXiv 2607.05391, 2026-07) · Position: VLA Cannot Be Verified to Perform Physical Reasoning (2606.30686, 2026-06) · Consilience for Verifier-Free Test-Time Scaling (2608.09898, 2026-08) · Agentic Harnesses: LLM-Driven Verification Layers (2608.09857, 2026-08) 相关解读：AGM (notes/12) · PRIMO R1 (notes/17) · VERITAS (notes/18) · Thea 的 Evaluation as Exit Codes (notes/22) · PhyAgentOS 的 SessionVerifier (notes/21) · VASO (notes/09) 所属主线：T23、T24 · 材料来源：报告第 4 章 T23、第 7 章 I3

1. 一句话定位

2026 年 T23 主线的密度说明验证器已成为自进化循环的瓶颈。这一组工作从四个角度处理它：LLM-as-a-Verifier 把验证当作新的 scaling 轴（对评分 token 的 logits 取期望得到连续分数，沿评分粒度、重复评估、标准分解三个维度扩展，RoboRewardBench 87.4%，可作 RL 密集奖励）；立场论文指出成功率无法区分语义匹配与物理泛化，因此现有评测协议无法验证 VLA 是否在做物理推理；Consilience 讨论没有验证器时如何用置信度轨迹选 rollout；Agentic Harnesses 在规划与执行之间放 LLM-as-a-Judge 集成验证层，对抗攻击 97% 拦截。

2. 各篇要点

LLM-as-a-Verifier：不训练的通用验证框架。与让 LLM 输出离散分数的标准 judge 不同，它对评分 token 的 logits 分布取期望得到连续分数；这个概率形式让验证沿三个维度扩展——分数粒度、重复评估、标准分解。在 RoboRewardBench 上 87.4%，并能作为 RL 的密集奖励（报告第 4 章转述）。它把 Let's Verify Step by Step 的"过程奖励"思想推广到了不训练的通用 judge。

立场论文 (VLA Cannot Be Verified to Perform Physical Reasoning)：把 VLA 策略分解为语义映射与物理动作决策两部分，论证任务成功率无法区分收益来自哪一部分；"互联网规模语义表征迁移到物理执行泛化"这一流行解释从未被独立验证，且在当前评测协议下不可检验。它要求受控变量的评测设计——对本仓库所有 T5/T9/T15 的成功率数字都是一句提醒。

Consilience：测试时扩展通常依赖外部验证器（编译器、测试用例、训练好的价值函数），但很多真实应用没有高质量验证器；基于置信度的无验证器测试时扩展几乎零开销。论文研究置信度轨迹的性质（报告第 4 章转述为"时间不对称性"）来改进 rollout 选择。对机器人的含义：当验证器缺席时，策略自身的置信度是最后的选择信号——但也是最容易被过度乐观污染的信号。

Agentic Harnesses：诊断机器人自主系统重执行、轻验证；规划模型偏向用户目标、可能违反科学伦理、记不住先前的安全风险、易受生态系统攻击。方法：在规划与执行之间加 LLM 驱动验证层评估动作可否被允许，用 LLM-as-a-Judge 集成；报告第 4 章转述的数字是对抗攻击 97% 拦截。它把验证器从"成没成"扩展到"该不该做"，与 T17 安全主线相接。

3. 口径与局限

- 87.4% (RoboRewardBench) 与 97% (对抗拦截) 测的是验证器自身的准确率，不是被验证策略的成功率；两者不可与任务成功率并排。
- LLM-as-a-Verifier 与 Agentic Harnesses 都是不训练的 LLM 判断，延迟决定它们只能低频运行；与 AGM 的 2.43M 高频验证头是不同层级。
- 立场论文没有提出替代指标，只提出评测设计原则；Consilience 的机器人实验范围摘要未说明。

4. 关系定位与延伸批判

把 T23 的全部工作按"验证什么、多快、多贵"排列，可以得到一个三层验证器的雏形：高频门控（AGM 的几何证据、Thea 的 exit codes）→ 低频过程复核（PRIMO R1 的视频批评者）→ 连续评分与准入判断（LLM-as-a-Verifier、Agentic Harnesses）；形式验证（VASO）在部署前过滤，数字孪生（notes/30）在部署前预演。报告开放问题第 1 条就是把这个雏形做成可评测的系统。立场论文警告应当反过来用于验证器本身：验证器判断"成功"时，判断的是语义上像成功还是物理上成功？PRIMO R1 的 67% 与 LLM-as-a-Verifier 的 87.4% 都需要这个拆分。最后，Lil'Log 与 AHE 的"评估器在演化循环之外"原则与 MEMENTO"先演化评估器"、Meta-Rewarding"评审的评审"直接冲突，机器人侧目前没有任何一篇工作正面处理这个冲突。

PART C

Harness、Runtime、双系统与端侧

Harness VLA、PhyAgentOS、Thea、Harness Engineering for Physical AI、其他 harness 框架、治理与运行时、快慢双系统、端侧部署。Harness 从软件名词变成实时系统问题。

1. 20 · Harness VLA 深度解读：不改权重、不扩技能库，学冻结 VLA 的"使用说明书"
2. 21 · PhyAgentOS 深度解读：把 Harness 做成操作系统
3. 22 · Thea 深度解读：物理世界不白送的两样东西——读状态、判结果
4. 23 · Harness Engineering for Physical AI 深度解读：机器人中间件就是 harness 层
5. 24 · Harness 框架合评：RoboHarness、Guava、Cortex 与 Agentic Harnesses
6. 25 · 治理与运行时合评：Runtime Governance、EmbodiedGovBench、ICAN-Deploy、FSAR
7. 26 · 快慢双系统合评：Fast-in-Slow、OneTwoVLA、StreamVLA、LaST0、RationalVLA 与 2026 年的频率分层
8. 27 · 端侧部署合评：PhyAI、EcoVLA、CloudEdgeVLA、ARLI、ST-Merge、Habilis-β

20 · Harness VLA 深度解读：不改权重、不扩技能库，学冻结 VLA 的"使用说明书"

Harness VLA: Steering Frozen VLAs into Reliable Manipulation Primitives via Memory-Guided Agents arXiv 2607.08448 (2026-07-09) · Yixian Zhang, Huanming Zhang, Feng Gao et al. (共 16 位作者) 所属主线：T15、T6、T20、T23 · 材料来源：小红书长文 02 节；检索地图 T15/T20/T23 代表工作；报告第 2.2 节 · 本仓库有原文与首两页中译 (papers/pdf_zh/2607.08448_zh.pdf)

1. 一句话定位

Harness VLA 把冻结的 VLA 当作一个需要"使用说明书"的原语：从任务专属的执行轨迹、全局成功规则和失败模型里，学出这个 VLA 什么时候可靠、什么时候先用解析原语 MOVE_TO、什么时候重新 grounding、什么时候交给解析原语处理——既不修改 VLA 权重，也不扩张技能库。LIBERO-Pro +38.6 个百分点、RoboCasa365 +27.1 个百分点、RoboTwin C2R 58.4% (v4, 2026-09-02；小红书长文引用的早期版本为 RoboCasa365 +25.4)。它是小红书长文 L4 Memory Evolution 的代表，也是"冻结 VLA + 外围学习"范式最典型的一篇。

2. 要解决的问题

语言条件操作同时需要精确的接触控制与对语言、场景、长程的鲁棒推理。端到端 VLA 有强的局部视觉运动技能，但在部署扰动下失效——语义重定向、目标重绑定、空间布局变化、不稳定的局部接触；LLM coding agent 有互补的语义与组合推理，但纯解析原语搞不定不规则抓取、受限放置与铰接物体。两者都不够，简单串联也不够：agent 需要知道 VLA 的能力边界。

3. 方法

记忆引导的 agent：记忆里存三类东西——任务专属执行轨迹（这个 VLA 在这类任务上的表现记录）、全局成功规则（什么条件下调用 VLA 会成功）、失败模型（什么情境下会失败、失败后该怎么办）。agent 据此决定每一步是调用 VLA、先用 MOVE_TO 把末端移到 VLA 熟悉的起始分布、重新 grounding 目标，还是交给解析原语。VLA 权重与技能库都不变；被学习的对象是"何时用、怎么用"的规则。

4. 实验结果与口径

- LIBERO-Pro: +38.6 个百分点（相对同一冻结 VLA 直接执行）。
- RoboCasa365: +27.1 个百分点 (v4; v1 摘要为 +25.4)。
- RoboTwin C2R (跨本体): 58.4%。
- 口径：仿真基准、脚本判定；提升为绝对百分点，基座 VLA 与基线协议需看正文；58.4% 是绝对成功率。与 RHO 的 45.0% (LIBERO-PRO, 代码策略)、Zetta 的 90.8% (LIBERO-Pro, 不同基座与 rollout 预算) 不可并排。

5. 局限

1. 规则从"任务专属执行轨迹"里学，意味着新任务需要先积累轨迹；零样本到全新任务族的表现摘要未说明。
2. "何时可靠"的判断依赖失败模型的覆盖面；没见过的失败模式（例如新物体的接触失效）会被误判为可靠。
3. 记忆是语言/规则形式，BATON 指出的"进入条件"在这里以 MOVE_TO 预定位的形式隐含存在，但没有被显式建模为几何分布。
4. 三个基准全部是仿真；真机结果摘要未提及。

6. 关系定位

Harness VLA 与 RHO 是 T15 的两半：RHO 优化代码策略仓库，Harness VLA 优化 VLA 的调用规则；两者都在 LIBERO-Pro 家族基准上报数，却从未在同一原语与同一 VLA 上组合。它与 BATON (notes/14)、RoboHarness (notes/24)、AtomBridge 共同回答"技能交接"问题，与 AGM (notes/12) 共享"记忆治理 VLA 调用"的思路。ENPIRE 的 RoboCasa365 混合方案（代码做 hover pose、VLA 做接触）是同一分工的 autoresearch 版本。

7. 延伸批判

+38.6pp 这个数字最需要的对照是"同样的记忆预算给一个简单的重试策略能拿多少"——如果 VLA 在扰动下的失败大多是可重试的随机失败，重试就能拿到一部分收益；Harness VLA 的贡献应当以"超过重试的部分"计。另一个问题是规则的可迁移性：从 LIBERO-Pro 学到的"使用说明书"能否直接用于 RoboCasa365 上的同一 VLA？如果不能，那么"不扩张技能库"只是把技能库换成了"规则库"，规模问题并未消失，只是换了对象——这正是 SkillGLoW (notes/07) 对文本技能"文档塌缩 vs 条目膨胀"两难的诊断。

21 · PhyAgentOS 深度解读：把 Harness 做成操作系统

PhyAgentOS: A Self-Evolving Operating System for Embodied Agents with Decoupled Cognitive and Physical Execution arXiv 2607.16636 (2026-07-18) · Yang Liu, Weixing Chen, Xinshuai Song et al. (共 11 位作者) 所属主线: T7、T15、T16、T17、T23 · 材料来源: 检索地图 T7/T15/T16/T23 代表工作; 报告第 4 章、第 5.2 节

1. 一句话定位

PhyAgentOS 把调度、验证、记忆、评测、安全做成系统级服务：以会话（session）而不是动作作为调度、兼容性预检、监督执行、证据收集与验收的最小单元；用 State-as-a-File 把跨层状态物化为 Markdown + YAML 以解耦认知与物理执行；SessionVerifier 区分"执行终止"与"语义完成"；验证过的结果经 epistemic memory 沉淀为可复用知识与纠正性教训；在 19+ 个仿真与真实本体上验证。它是 Lil'Log "harness 像 OS"这一类比在机器人侧的字面实现。

2. 要解决的问题

VLA、世界模型、agentic 规划器各自推进物理智能，但它们的组合缺少共同的执行抽象、共享状态、语义验证与跨异构本体的持久经验。每个项目重写一遍"调用—验证—记忆—恢复"的胶水，且这些胶水无法跨本体复用、无法审计。

3. 方法

- **Session-Centered Runtime**：会话是最小调度单元，包含兼容性预检（本体、传感器、技能是否匹配）、监督执行、证据收集、验收。
- **认知-物理边界 + State-as-a-File**：认知层与执行层通过文件化的状态（Markdown + YAML）通信，任何一层都可以读、写、审计、回滚——这就是 Lil'Log 的"文件系统即持久记忆"。
- **SessionVerifier**：把"程序跑完了"与"任务在语义上完成了"分开判断。
- **Epistemic memory**：验证过的结果沉淀为可复用知识与纠正性教训，供后续会话检索。
- **自进化**：以上服务使系统能在会话之间积累与修正，形成"自进化操作系统"。

4. 实验结果与口径

- 摘要给出的是覆盖范围（19+ 仿真与真实本体）与系统能力，而非单一成功率数字；具体任务级结果见正文。
- 口径：系统论文，评测重点是跨本体的可复用性与治理能力，与单任务成功率论文不同类。

5. 局限

1. "OS"的抽象越通用，每个本体上的性能调优空间越小；PhyAgentOS 在单一基准上与专用系统（如 Harness VLA）的对比摘要未给出。

2. State-as-a-File 让状态可审计，但文件读写的延迟决定它只适用于会话级/子任务级，不适用于动作频率的治理——Zetta (notes/07) 与 Harness Engineering for Physical AI (notes/23) 处理的是那一层。
3. SessionVerifier 的"语义完成"判断依赖什么模型、准确率多少，摘要未说明。

6. 关系定位

PhyAgentOS 同时落在五条主线上，是本仓库连接度最高的系统工作。对 T16 它是 Runtime 主线的"内置治理"代表（与 Runtime Governance 的"外置治理"形成对照，notes/25）；对 T23 它的 SessionVerifier 与 Thea 的 Evaluation as Exit Codes (notes/22) 是同一问题的两种接口；对 T7 它把自进化放在会话粒度；对 Lil'Log 的三模式，它同时实现了工作流自动化与文件系统记忆。

7. 延伸批判

PhyAgentOS 最大的贡献是命名与分层——它让"验证、记忆、调度、安全"从每个项目的私有代码变成可讨论的服务接口。但 OS 的价值来自生态：如果没有第二个团队在 PhyAgentOS 上跑自己的策略，它就只是一个更大的项目脚手架。因此对它最公平的检验不是任务成功率，而是 EmbodiedGovBench 式的七维治理评测（越权、漂移、恢复、可移植、升级安全、人工接管、审计），以及"一个新本体接入需要多少行配置"这类工程指标——两者论文都没有给。另一个未走之路：State-as-a-File 与 ENPIRE 的 Markdown 研究总结 (notes/06) 是同一机制，两者结合可以让"研究经验"也成为 OS 的一等状态。

22 · Thea 深度解读：物理世界不白送的两样东西——读状态、判结果

Towards the Harness of Embodied Agents (Thea) arXiv 2608.11246 (2026-08-03) · Qi Wang, Tianyi Wang, Chengyang Li et al. (共 9 位作者) 所属主线：T15、T23 · 材料来源：报告第 4 章 T15/T23、第 5.3 节

1. 一句话定位

Thea 直接把 coding agent 的 harness 范式搬到机器人：一个 agentic 循环编排机器人能力，每个能力包装成可调用的工具，继承 coding agent harness 的核心组件并按物理世界的要求修改。它的洞见是：物理世界拒绝提供软件白送的两种能力——读取世界状态与判断动作结果——于是引入 **Scene Graph as Context**（持久的符号化世界表示）和 **Evaluation as Exit Codes**（检测动作何时应终止、判断是否成功、失败时诊断原因）。

2. 要解决的问题

coding agent 的成功确立了一个范式：agent 的成就取决于模型周围的基础设施而不只是模型。这个范式能否延伸到具身 agent？软件 harness 依赖两个廉价前提：状态可读（文件、变量、日志）与结果可判（编译、测试的退出码）。机器人两者都没有——相机不是文件系统，动作没有退出码。

3. 方法

- **Scene Graph as Context**：用持久的符号化场景图充当“文件系统”——物体、关系、状态可被 agent 读取、引用与更新，是 agent 上下文的一部分而不是每步重新感知。
- **Evaluation as Exit Codes**：给每个动作一个类似退出码的评估——何时该终止（动作完成或卡住）、是否成功、失败原因诊断——让 agentic 循环能像处理编译错误一样处理物理失败。
- 其余组件（工具调用、上下文管理、记忆）继承 coding agent harness 并做物理修改。

4. 实验结果与口径

- 摘要定位为框架论文，未给单一头条数字；评测范围与任务需看正文。
- 口径：系统层贡献；与 Guava (notes/24) 同属“什么构成有效的具身 harness”的探索，与 PhyAgentOS (notes/21) 同属 harness 系统化。

5. 局限

1. 场景图的构建本身依赖感知模型；场景图错误会以“可读状态”的形式被 agent 信任——这是把感知不确定性藏进符号表示的经典风险。
2. Exit codes 的诊断粒度决定 harness 的有效性；接触失效（抓滑、插不到底）如何被编码为退出码，摘要未说明。
3. 没有时间维度：Harness Engineering for Physical AI (notes/23) 指出物理 harness 必须处理延迟与调度，Thea 的两个补丁不涉及这一点。

6. 关系定位

报告第 5.3 节把机器人 harness 与软件 harness 的本质差异概括为三点——读状态难、判结果难、有时间——前两点来自 Thea，第三点来自 Harness Engineering for Physical AI。Thea 的 Evaluation as Exit Codes 与 PhyAgentOS 的 SessionVerifier、AGM 的证据门控进度指针、ENPIRE 的验证工具 T23 主线上四种"把'成没成'做成显式接口"的方式。Scene Graph as Context 与 Analytic Concept-Centric Memory (notes/15) 在"结构化符号世界表示"上同向。

7. 延伸批判

"退出码"是一个极好的比喻，也是一个危险的比喻：编译器的退出码是确定的，物理动作的"退出码"是一个概率判断。Thea 把这个判断做成接口，却没有说明接口背后的判断器有多准——这正是 PRIMO R1 (67%) 与 LLM-as-a-Verifier (87.4%) 报告的那类数字。一个自然的扩展是让 exit code 带置信度与证据（AGM 式的物理证据、Consilience 式的置信度轨迹），让 agentic 循环能对低置信退出码选择复核而不是直接相信。另一个未走之路：场景图作为持久上下文，天然是多机器人共享世界知识（T19/T21 的空缺层）的候选载体。

23 · Harness Engineering for Physical AI 深度解读：机器人中间件就是 harness 层

Harness Engineering for Physical AI: Robot Middleware Is the Harness Layer arXiv 2606.09416 (2026-06-08) · ACM/IFIP Middleware 2026 (Big Ideas) · Sanghoon Lee, Jiyeong Chae, Kyung-Joon Park 所属主线：T14、T15、T16 · 材料来源：小红书长文 03 节 (Real-time-aware Harness)；检索地图 T14/T15 代表工作；报告第 2.3 节、第 7 章 15

1. 一句话定位

这篇短文提出一个命名：学习策略、规划器与 VLA 已经作为因果参与者进入部署机器人的控制路径，但把它们与时序、调度、网络整合起来的那一层还没有名字；语言 agent 社区叫它 harness——机器人的中间件就是这个 harness。物理 AI 的 harness 与软件 harness 的区别在于**介入的位置**：软件 harness 在工具调用边界介入，物理 harness 必须同时在控制、计算、通信三处介入，因为学习策略的输出同时改变轨迹、日程与带宽。它提出三个缺失的强制功能——Projection、Isolation、Transfer——并建议做成 ROS 2 Harness Profile。

2. 要解决的问题

VLA 与学习策略进入机器人后，一次推理花 2 秒不再只是“慢”：它改变了控制回路的时序，可能让下游控制器读到过期命令，可能挤占通信带宽，可能让安全反射来不及介入。现有中间件（ROS 2 等）把这些模型当普通节点处理，没有为“输出不可预测、延迟不可预测、失败不可预测”的参与者设计强制机制。

3. 方法（提案）

- **Projection**：在输出处约束动作——把学习策略的输出投影到安全/可行集合内，而不是相信它。
- **Isolation**：限定模型推理的执行与传输时隙——推理不能无限占用计算与网络，deadline 必须被强制。
- **Transfer**：检查失败时回退到经过验证的基线——策略、规划器不可用或输出不可接受时，控制权移交给已验证的备份。
- 落地建议：把三者做成 ROS 2 Harness Profile，使它们成为声明式配置而非每个项目重写的代码。

4. 实验结果与口径

- 这是立场/大想法论文 (Big Ideas track)，没有实验数字。
- 与之相关的系统证据来自同期工作：PhyAI 的 control-time Roofline (区分推理受限与环境受限的控制)、CloudEdgeVLA 在 40 步延迟下 63.8–78.0% vs 对比方法最高 6.4%、EcoVLA 在 20 Hz 约束下能效 +236%、ARLI 证明延迟破坏马尔可夫假设 (notes/27)。

5. 局限

1. 三个功能的形式化定义（投影到什么集合、时隙如何分配、回退的触发条件）留给后续工作。

2. "中间件就是 harness"的命名如果被接受，会把 Lil'Log 意义上的 harness（工具、记忆、评估、 workflow）与实时系统意义上的 harness（调度、投影、回退）合在一个词下——两者的设计目标不同，可能需要分层命名。
3. 没有讨论多机器人（通信介入的自然延伸）。

6. 关系定位

这篇论文是报告 I5（harness 从软件名词变成实时系统问题）的直接来源，也是小红书长文"Real-time-aware Harness"（知道模型最大延迟、技能 deadline、断网 fallback、哪些动作必须本地完成）的学术版本。Zetta 的"动作频率治理"（notes/07）是 Isolation 的一个实例；Runtime Governance 的回滚与人工接管（notes/25）是 Transfer 的治理层版本；PonderPounce 的认知"年龄"字段（notes/11）是 Isolation 在接口上的一个微观体现。与 Thea（notes/22）合起来构成机器人 harness 与软件 harness 的三点差异。

7. 延伸批判

这篇论文最有力的地方是它把 harness 问题交还给了一个成熟的工程社区（中间件、实时系统），而不是留在 prompt 工程里。最值得做的后续是把 Projection / Isolation / Transfer 写成 ROS 2 profile 并与 MHS（notes/36）的设备发现协议对接，让"延迟预算、deadline、fallback"成为机器人描述文件的一部分——报告开放问题第 4 条。一个未被讨论的张力：Projection（约束输出）与 Harness VLA（学 VLA 何时可靠）是同一问题的两种解法，前者在毫秒级硬约束，后者在秒级软决策；两者的分层关系没有人正式写出来。

24 · Harness 框架合评：RoboHarness、Guava、Cortex 与 Agentic Harnesses

覆盖：RoboHarness (arXiv 2607.18060, 2026-07, 华为诺亚 · UBC · 多伦多) · Guava (2606.18363, 2026-06) · Cortex (2607.05377, 2026-07) · Agentic Harnesses (2608.09857, 2026-08, 另见 notes/19) 所属主线：T15、T1、T4、T20 · 材料来源：具身纪元文章点名 RoboHarness；报告第 4 章 T1/T15

1. 一句话定位

2026 年 6-8 月出现了一批"围绕冻结模型的机器人 harness 框架", 各自回答一个不同的问题: RoboHarness 问异构策略 (VLA、RL、TAMP) 怎么被当作可复用的 agentic 技能来调度——用执行记忆刻画能力边界, 用 Memory Bridge 在切换前把机器人带到下一策略熟悉的状态 (135 次真机实验, 使用 GPT-5.5 改策略编排代码); Guava 问什么构成有效且通用的具身 harness——系统探索设计空间得到三要素 (迭代感知-推理-动作循环、语义动作抽象、多模态观测) 并蒸馏进 4B 模型; Cortex 问高层规划语义与低层执行运动学之间的缝怎么补——32 个规范技能原语的双向对齐规划接口; Agentic Harnesses 问规划出的动作该不该执行——LLM 验证层。

2. 各篇要点

RoboHarness: 长程任务需要没有单一策略能全部提供的多样能力; 异构策略互补, 但编排它们要推理不确定的能力边界与跨策略的分布不匹配——现有规划方法建立在同质、预定义、适用性固定的技能上, 忽略了这一点。方法: 把独立开发的机器人控制系统 (VLA、RL 策略、TAMP) 封装为可复用 agentic 技能, 用多模态执行记忆刻画每个策略的能力边界, 用 Memory Bridge 引导机器人进入下一策略的分布内区域。具身纪元文章的例子: 先让 VLA 打开柜门取出积木, 再交给 TAMP 精确搭桥; 135 次真机实验。

Guava: 通过系统探索 harness 设计空间得到有效具身 harness 的三个要素——迭代的感知-推理-动作循环、语义动作抽象、多模态观测; 并把 harness 释放的能力蒸馏进一个 4B 模型, 说明 harness 既是运行时也是训练数据的来源。

Cortex: VLA 的马尔可夫性质使其在长程上失效; 分层双系统方法有规划语义与执行运动学之间的缝。方法: 把操作子任务标准化为 32 个规范技能原语, 构建双向对齐的规划接口, 让高层 VLM 输出可执行、可处理的子任务计划给低层 VLA。它是"用有限原语集合约束规划输出"的代表。

Agentic Harnesses: 见 notes/19; 此处强调它把 harness 的职责从"完成任务"扩展到"判断允许"。

3. 口径与局限

- RoboHarness 的 135 次真机实验是本组唯一的真机规模数字; 成功率与任务集见正文。
- Guava 的三要素来自设计空间探索, 探索的基准范围决定结论的普适性; 4B 蒸馏模型的性能与前沿模型 harness 的差距摘要未说明。
- Cortex 的 32 个原语是人工标准化的——这正是 CaP-X 警告的"设计者脚手架"。

4. 关系定位与延伸批判

这四篇与 Harness VLA (notes/20)、Thea (notes/22)、PhyAgentOS (notes/21) 一起构成 T15 的密集带。把它们按"harness 治理什么"排列：Cortex 治理规划输出的形式、Harness VLA 治理 VLA 的调用时机、RoboHarness 治理异构策略间的交接、Agentic Harnesses 治理动作的许可、PhyAgentOS 治理整个会话、Thea 治理状态读取与结果判断、Guava 则回答"以上哪些是必需的"。RoboHarness 的 Memory Bridge 是 BATON"进入条件"问题 (notes/14) 在异构策略上的解法，与 Harness VLA 的 MOVE_TO 预定位同源。未走之路：这些 harness 各自定义了自己的技能接口（RoboHarness 的 agentic skill、Cortex 的 32 原语、Guava 的语义动作抽象），没有一个与 MHS / ROS 2 Harness Profile (notes/23、36) 这类标准接口对接——harness 层的碎片化正在重演策略层的碎片化。

25 · 治理与运行时合评：Runtime Governance、EmbodiedGovBench、ICAN-Deploy、FSAR

覆盖：Harnessing Embodied Agents: Runtime Governance for Policy-Constrained Execution (arXiv 2604.07833, 2026-04) · EmbodiedGovBench (2604.11174, 2026-04) · ICAN-Deploy (2605.28097, 2026-05) · FSAR (federated single-agent runtimes; 见 README T16) 所属主线：T16、T17、T21 · 材料来源：检索地图 T16 代表工作；报告第 4 章 T16、第 7 章 I7

1. 一句话定位

一旦 agent 获得执行权，核心问题从"怎么让它行动"变成"怎么让它的行动在运行时可治理"。这组工作把治理从 agent 循环里拿出来做成独立层：Runtime Governance 外置策略检查、能力准入、执行监控、回滚与人工接管——1000 次随机试验 96.2% 越权拦截，运行时漂移下不安全继续从 100% 降到 22.2%，恢复成功 90.7%，且是与 AutoRT 式宪法过滤、RoboGuard 式两阶段护栏比较中唯一提供持续运行时检测（RVDR 61.3% 对 0%）与结构化恢复的方法；EmbodiedGovBench 把治理做成七维评测；ICAN-Deploy 用 TLA+ 验证身份稳定的金丝雀部署；FSAR 主张 fleet 层联邦单 agent 运行时而非机器人内部的多 agent 碎片化。

2. 各篇要点

Runtime Governance：现有方法把安全、恢复、决策约束嵌入 agent 循环，使执行控制难以标准化、审计与跨环境适配。方法：把治理外置为专门运行时层——策略检查、能力准入、执行监控、回滚、人工接管——把 agent 认知与执行监督分离。数字：96.2% 越权拦截（1000 次随机试验）；漂移下不安全继续 100%→22.2%；恢复成功 90.7%；预执行过滤各方法相当，持续运行时检测 RVDR 61.3% vs 0%。

EmbodiedGovBench：当前评测被任务成功率主导，不测系统是否可治理——是否尊重能力边界、执行策略、安全恢复、保留审计、响应人工监督。七个维度：越权调用、运行时漂移、恢复、策略可移植、升级安全、人工接管、审计完整性。

ICAN-Deploy：金丝雀部署（把一部分流量导到新版本、监控、回滚）的主流控制器会在金丝雀窗口内改变系统的密码学身份，对无状态微服务无害，但对安全关键的具身 agent 会打破"你认证的 agent 仍是你现在的 agent"，迫使每次金丝雀都重新认证。方法：一个中间件构造，其状态机在金丝雀窗口内保持身份哈希不变（分离能力与身份），以 TLA+ 验证。

FSAR：多机器人协调不需要在机器人内部把 agent 碎片化为多 agent 社会，而应在 fleet 层联邦一致的单 agent 运行时；讨论了治理与恢复的边界应放在哪一层。

3. 口径与局限

- 96.2%、22.2%、90.7% 来自随机试验与仿真化的越权场景，摘要未说明真机比例；"越权"由策略规则定义，规则之外的危险行为不在统计内。
- 七维治理指标目前主要在仿真验证（报告第 7 章 I7 的判断：治理被观点文章低估、被论文高估）。
- ICAN-Deploy 的贡献是形式化与中间件构造，没有部署规模数据。
- FSAR 是立场性论述。

4. 关系定位与延伸批判

这组工作填补了小红书长文与具身纪元文章共同的盲区：两篇观点文章都描绘了自进化闭环（包括 closed loop 这一格），却没有说谁来授权闭环。Runtime Governance 的"外置治理"与 PhyAgentOS 的"内置治理服务"（notes/21）是同一目标的两种架构；Lil'Log 与 AHE 的"评估器与权限在演化循环之外"原则在这里得到机器人侧的实现——治理层只读、不可被 agent 演化。最值得追问的是**治理与自进化的接口**：当 Zetta（notes/07）在线演化恢复技能、ASPIRE（notes/04）把修复存成技能时，新技能的能力准入由谁批准？EmbodiedGovBench 的"升级安全"维度与 ICAN-Deploy 的身份稳定正是为此准备的，但目前没有任何自进化系统接入了它们。报告开放问题第 7 条（治理进入 LWD / ENPIRE 规模的真机集群）是检验这条线的下一步。

26 · 快慢双系统合评：Fast-in-Slow、OneTwoVLA、StreamVLA、LaST0、RationalVLA 与 2026 年的频率分层

覆盖：Fast-in-Slow (arXiv 2506.01953, 2025-06) · OneTwoVLA (2505.11917, 2025-05) · StreamVLA (2602.01100, 2026-02) · LaST0 (2601.05248, 2026-01) · RationalVLA (2506.10826, 2025-06) · UniFS (2606.22794) · Latent Bridge (2605.02739) · VisualThink-VLA (2605.30011) · OpenHelix (2505.03912, 综述与开源实现) 所属主线：T13、T14 · 材料来源：小红书长文 03 节 (两个时间尺度)；检索地图 T13 代表工作

1. 一句话定位

小红书长文把 Agent 负责秒、Controller 负责毫秒的划分称为"两个时间尺度"；双系统 VLA 是这一划分在单个模型内部的版本。2025 年的工作确定了三种耦合方式——共享参数 (Fast-in-Slow, System 1 嵌入 System 2, action chunk 为 8 时 117.7 Hz)、单模型自适应切换 (OneTwoVLA 在关键时刻显式推理、其余时刻直接出动作)、拒绝缺陷指令的理性层 (RationalVLA 与 RAMA 基准, 14,000+ 样本含六维缺陷指令)；2026 年的工作解决"何时切换"与"以什么频率切换"——StreamVLA 只在子任务切换时触发慢思考并想象"完成态"作为时间不变的目标锚点 (72% 时间步跳过自回归解码, 延迟 -48%, LIBERO 98.5%), LaST0 把推理搬进潜空间、MoT 双专家异频运行 (10 个真机任务平均 +13-14%), UniFS 按更新频率分层 VLM 各层 (36.5 ms → 17.8 ms), Latent Bridge 预测 VLM 输出的帧间 delta (减少 50-75% VLM 调用), VisualThink-VLA 用视觉中间推理替代文本 CoT (步延迟 8.377 s → 0.367 s)。

2. 各篇要点

Fast-in-Slow：现有双系统把两个系统做成独立模型，System 1 不能充分利用 System 2 的丰富表征。方法：把 System 1 嵌入 System 2 共享参数，异构模态输入、异步频率。117.7 Hz (chunk=8)。

OneTwoVLA：单一统一模型同时做 acting 与 reasoning，自适应切换：任务执行的关键时刻显式推理，其余时刻基于最近的推理生成动作。

StreamVLA：VLA 在每个时间步做冗余的多模态推理导致高延迟与目标不稳。方法："Lock-and-Gated"机制——只在检测到子任务转移时触发文本任务分解与视觉目标想象，把"完成态"锁定为时间不变的目标锚点，连续动作生成在同一参数高效主干上。72% 时间步跳过自回归解码。

LaST0：显式推理带来不可忽略的推理延迟，且局限于语言空间形成表征瓶颈。方法：潜空间时空 CoT，低频推理专家给高频动作专家提供物理表征。

RationalVLA：现有语言条件任务假设指令与环境完全对齐；RAMA 基准包含未见可执行指令与应当拒绝的缺陷指令 (六维缺陷)；双系统里的理性层判断是否执行。它把"拒绝"变成了 VLA 的一等输出，与 T17 安全相接。

UniFS / Latent Bridge / VisualThink-VLA：三种降低慢系统开销的路径——按频率分层 (浅层快、深层慢)、预测帧间特征差 (VLM 只周期性调用, GR00T-N1.6 与 π 0.5 上验证)、用紧凑的视觉证据接口替代文本 CoT。

3. 口径与局限

- 117.7 Hz、72% 跳过、36.5→17.8 ms 是各自设置下的系统指标；LIBERO 98.5% 是仿真成功率；LaSTO 的 +13–14% 是真机相对提升。彼此不可并排。
- "何时切换"的判据（子任务转移检测、关键时刻判断）本身是一个验证问题：误判切换时机会导致慢系统在错误时刻介入或缺席。
- 双系统的收益在长程、需要重规划的任务上最明显；在短程反应式任务上慢系统只是开销。

4. 关系定位与延伸批判

OpenHelix 已经系统整理了双系统的设计空间并给出开源实现，所以 T13 的问题不再是"怎么切"而是"何时切"——StreamVLA 的完成态门控与 RL^2 -VLA 的失败预测门控 (notes/32) 是两种答案。把这条线接回 harness (notes/23)：双系统是模型内部的 Isolation（限定慢推理的时隙），PonderPounce 的认知"年龄" (notes/11) 与 Latent Bridge 的 delta 预测都是让快系统在慢系统缺席时"知道自己在用多旧的信息"。未走之路：没有一篇双系统工作报告在网络延迟或算力抖动下的行为——CloudEdgeVLA 与 ARLI (notes/27) 从系统侧接近了这个问题，但尚未与双系统架构合流。

27 · 端侧部署合评：PhyAI、EcoVLA、CloudEdgeVLA、ARLI、ST-Merge、Habilis-β

覆盖：PhyAI (arXiv 2608.03682, 2026-08) · EcoVLA (2608.15502, 2026-08, APPT 2026) · CloudEdgeVLA (2608.00569, 2026-08) · ARLI / Learning to Act While Waiting (2608.23831, 2026-08) · ST-Merge / Fast Enough to Act (2606.29350, 2026-06) · Habilis-β (2602.18813, 2026-02) 所属主线：T14、T9 · 材料来源：小红书长文 03 节 ("Cloud can think. Edge must survive."); 报告第 4 章 T14、第 7 章 I5

1. 一句话定位

大模型上云、控制在端，延迟、抖动与断网怎么办——这组系统工作给了 2026 年的答案：PhyAI 用一个运行时统一 VLA/WAM 在板载、边缘、云端的推理（对 π_0 、 $\pi_{0.5}$ 、GR00T N1.7、MiniCPM-Robot 提速 1.40–4.65 倍）并提出 control-time Roofline 区分推理受限与环境受限的控制；EcoVLA 做设备-边缘协同推理的能效优化（20 Hz 约束下能效 +236%）；CloudEdgeVLA 把时间错位当作表示学习问题——云端编码慢变任务特征、边缘头结合最新本地视觉，40 步统一延迟窗口下仍 63.8–78.0%，对比方法最高 6.4%；ARLI 说明推理延迟会改变有效环境动力学、破坏马尔可夫假设，标准 RL 在延迟下完全失效，需要带中间信息的异步 RL；ST-Merge 训练无关地合并时空视觉 token（ $\pi_{0.5}$ 在 1024^2 下提速 8.3 倍）；Habilis-β 提出生产力-可靠性平面（Tasks per Hour × Mean Time Between Intervention），在一小时连续运行协议下评测端侧 VLA。

2. 各篇要点

- PhyAI**：物理 AI 策略在生命周期各阶段（评测、云端 RL rollout、边缘 GPU 服务、板载部署）共享同一 checkpoint 与动作语义，却依赖不同的推理程序。方法：单一运行时，架构特定的条件化/求解器/缓存/输出逻辑放在模型适配器里，共享图执行、kernel、内存管理与并行服务；同一代码库在多种硬件上跑 VLA 与 WAM。control-time Roofline 是它给"控制受限于什么"的诊断工具。
- EcoVLA**：板载推理受算力与能量预算限制，难以同时满足实时与能效；卸载到边缘服务器又受系统条件波动影响带来不可预测延迟。方法：面向 VLA 的设备-边缘协同推理的系统性设计与能效优化。
- CloudEdgeVLA**：在移动机器人上部署十亿参数 VLA 有系统性冲突——语义推理受益于云端 GPU，闭环控制必须本地响应网络延迟与抖动。方法：云端 VLA 把延迟的观测编码为慢变的任务特征，轻量边缘头把它与最新本地视觉结合；"涌现表征"意味着不需要显式的调度或延迟线索。
- ARLI**：大模型的推理延迟导致停顿或抖动，改变有效环境动力学；若不正确处理，标准 RL 算法完全失败。方法：延迟感知的异步 RL，用中间信息增广状态。它是 T9 与 T14 的交点——部署时 RL 必须知道自己的延迟。
- ST-Merge**：视频与高分辨率图像产生海量视觉 token；在视觉编码阶段用 3D 时空坐标做多队列并行匹配与加权聚合融合冗余 token，即插即用、无需训练。
- Habilis-β**：单次成功率在精心复位下的评测不反映实用能力；PRP 平面用 TPH 与 MTBI 在连续运行协议下同时要求高速执行与持续鲁棒。

3. 口径与局限

- 提速倍数 (1.40–4.65×、8.3×)、能效 (+236%)、延迟窗口下的成功率 (63.8–78.0%)、TPH/MTBI 是四种不同类型的指标，与任务成功率不可并排。
- CloudEdgeVLA 的 40 步延迟是仿真中的统一延迟窗口，真实网络的抖动分布未必匹配。
- Habilis-β 的一小时连续运行是本仓库少见的"长期运行"口径，但任务与平台单一。

4. 关系定位与延伸批判

这组工作是报告 I5 (harness 变成实时系统问题) 的实证基础，与 Harness Engineering for Physical AI (notes/23) 的 Projection/Isolation/Transfer 提案——对应：ARLI 与 CloudEdgeVLA 处理 Isolation 失效后的补救，PhyAI 的 Roofline 是诊断 Isolation 是否被违反的工具。Habilis-β 的 TPH × MTBI 是本仓库推荐的"Experience Scaling 度量"候选之一 (报告开放问题第 8 条)：它衡量的正是"机器人能否长期工作"。未走之路：没有一篇把双系统架构 (notes/26) 与云边分工放在同一实验里——慢系统在云、快系统在端是最自然的映射，CloudEdgeVLA 最接近，但它的"边缘头"不是一个完整的 System 1。

PART D

学习闭环：RL、数据回流、数字孪生与世界模型

LWD 的 16 台机器人、Q-Planning 的小 Q 函数、TwinRL 与数字孪生谱系、RoboClaw 的自复位、VLA + RL 的信用分配、世界模型的角色。

1. 28 · LWD 深度解读：16 台机器人的 fleet-scale 离线到在线 RL
2. 29 · Q-Planning 深度解读：只微调一个小 Q 函数，就能从部署失败里学
3. 30 · TwinRL 与数字孪生谱系：Sim 是 sandbox，不是训练场
4. 31 · RoboClaw 深度解读：让机器人同时学会"做任务"与"恢复现场"
5. 32 · VLA + RL 合评：信用分配、免外部奖励与测试时 RL
6. 33 · 世界模型的角色合评：H-WM、Motus2、ContactGuard、SafeDojo、Online Continual RL、RoboGene

28 · LWD 深度解读：16 台机器人的 fleet-scale 离线到在线 RL

Learning While Deploying: Fleet-Scale Reinforcement Learning for Generalist Robot Policies (LWD) arXiv 2605.00416 (2026-05-01) · Yi Wang, Xincheng Li, Pengwei Xie et al. (共 16 位作者) 所属主线：T9、T10、T21
· 材料来源：小红书长文 04/06 节；检索地图 T9/T10/T21 代表工作；报告第 2.4 节、第 7 章 I2/I8

1. 一句话定位

LWD 是"部署 → 经验 → RL → 更新策略 → 重新部署"这条数据回流闭环目前最完整的真机实证：从预训练 VLA 出发，用 16 台双臂机器人、8 个真实任务做 fleet-scale 的离线到在线 RL，单一通用策略随群体经验积累提升到平均 95% 成功率，长程任务提升最大。它同时是小红书长文 L6（策略权重被更新）与 L7（群体共享策略改进）的证据，也是报告 I2"外围学习有天花板、真实反馈终要写回权重"的主要支撑。

2. 要解决的问题

通用机器人策略受益于大规模预训练，但离线数据不足以支撑鲁棒部署：部署中的分布偏移、长尾失败、任务变体与人类纠正机会都不是固定示范数据集能覆盖的。问题是如何让部署本身成为持续后训练的数据来源，并让多台机器人的经验汇入同一个策略。

3. 方法

从预训练 VLA 出发，闭合部署、共享经验缓冲与持续 RL 后训练之间的循环：多台机器人并行部署同一策略、把 rollout（含成功、失败与人类纠正）汇入共享数据、离线到在线 RL 更新策略、再统一重新部署。具体算法细节（离线阶段的正则化、在线阶段的探索控制）见正文，摘要强调"fleet-scale"与"offline-to-online"两个属性。

4. 实验结果与口径

- 16 台双臂机器人、8 个真实任务；单一通用策略平均成功率提升到 95%；长程任务提升最大。
- 口径：真机、多任务平均、二元成功率；起点（预训练 VLA 的成功率）与每任务试验数需看正文；"95%"是多任务平均而非单任务。
- 群体规模与提升速度的关系（scaling 曲线）摘要未给出——这是与 ENPIRE 集群实验（8 倍机器人 2-3 倍加速）最值得对照的空缺。

5. 局限

1. 8 个任务是否覆盖了策略预训练时未见的任务族，决定"95%"是适应还是泛化。
2. 人类纠正在数据里的比例与作用未在摘要中拆分；"无人干预"程度不清楚。
3. 共享经验意味着一台机器人的坏数据会进入全体策略——Memory Anchors (notes/15) 提示回流数据可能淹没旧任务的锚点，多机器人通信攻击 (notes/37) 提示共享经验会共享污染；LWD 摘要没有讨论数据准入。
4. 16 台同型双臂机器人：跨本体的 fleet 学习不在范围内。

6. 关系定位

LWD 是 T10（数据回流）的锚点工作。与 Q-Planning (notes/29) 比较：LWD 更新整个策略、需要 fleet 与 RL 基础设施，Q-Planning 只微调小 Q 函数、单机可行；两者都证明从部署失败中学优于只用成功样本 SFT (VERITAS 的路线，notes/18)。与 ENPIRE (notes/06) 比较：LWD 的群体共享的是策略数据，ENPIRE 的群体共享的是研究假设；前者线性扩展 rollout，后者亚线性扩展假设吞吐量。与 RoboClaw (notes/31) 比较：RoboClaw 解决的是自复位让数据回流不需要人，LWD 假设了数据回流的基础设施已存在。

7. 延伸批判

LWD 的 95% 是本仓库最常被引用的头条数字之一，但它需要三个补充才能成为"Experience Scaling"的证据：(1) 随机器人数量 N 的 scaling 曲线——8 台能否达到同样水平、32 台是否更快；(2) 数据准入机制——哪些 rollout 被允许进入共享缓冲，谁来判断（回到验证器问题）；(3) 对旧任务的保留率——群体经验主要来自 8 个任务，预训练能力是否被吃掉（Memory Anchors 的问题）。这三项正是报告开放问题第 5、1、3 条在 fleet 尺度上的形态。

29 · Q-Planning 深度解读：只微调一个小 Q 函数，就能从部署失败里学

Beyond Imitation: Self-Improving Robot Policies via Off-Policy Q-Planning arXiv 2608.21204 (2026-08-21)

· Varun Giridhar, Anant Khandelwal, Jeremy A. Collins et al. (共 5 位作者) 所属主线: T9、T10、T7 · 材料来源: 报告第 2.4 节、第 4 章 T9/T10、第 7 章 I2

1. 一句话定位

Q-Planning 给一个大的视觉运动 BC 策略配一个小的 off-policy Q 函数：因为 Q 函数估计价值而不是模仿动作，它可以在策略自己的失败 rollout 上训练；部署时用 Q 函数在 BC 策略的候选动作里做规划/选择。只微调 Q、BC 冻结、无人干预，真机 stack-cups 从 40% 到 90%、insert-wallet 从 25% 到 80%，而只用成功 rollout 做 SFT 分别停在 55% 和 30%。

2. 要解决的问题

BC 无法自我改进：失败的策略不能从失败里学，除非有新的人类示范；RL 微调提供了自改进的路，却难以扩展到支撑现代机器人策略的数十亿参数模型。需要一条"不改大模型也能从失败学"的路。

3. 方法

- 大 BC 策略（冻结）负责提出动作候选；小 off-policy Q 函数负责评估候选的价值。
- Q 函数在部署 rollout（成功与失败都用）上以 off-policy 方式训练——失败样本给出低价值，成功样本给出高价值。
- 部署时 Q-Planning：从 BC 采样多个候选动作，选 Q 值高者执行。
- 自改进循环：部署 → 收集 rollout → 更新 Q → 再部署，无人干预、无需新示范。

4. 实验结果与口径

- 真机：stack-cups 40% → 90%，insert-wallet 25% → 80%。
- 对照：只用成功 rollout 做 SFT，分别 55% 与 30%——说明失败样本的信息被 Q 函数用上了，而 SFT 用不上。
- 口径：真机、两个任务、二元成功率；每次评测的试验数与自改进轮数见正文。

5. 局限

1. Q 函数的能力上限由 BC 策略的候选覆盖决定：如果 BC 从不提出正确动作，Q 无法创造它——这是"选择器"而非"生成器"的固有边界（与 VERITAS 的推理时引导同样的边界，notes/18）。
2. 两个任务、单机；跨任务的 Q 函数是否可共享、是否随任务数增长而失效未评估。
3. 候选采样数与 Q 评估的延迟决定控制频率，摘要未给。

6. 关系定位

Q-Planning 是报告 I2 的关键证据之一：外围学习（这里是一个小价值函数）能拿到大部分收益，但收益来自真实失败被写进了一个可训练对象——这与 HyMeS“技能在权重里、记忆在代码里”（notes/13）形成有趣的第三条路：“技能在大权重里、价值在小权重里”。与 LWD（notes/28）相比它便宜得多；与 VERITAS 相比它用失败样本而不只用成功样本；与 TT-VLA、T²VLA（notes/32）相比它不需要修改 VLA 本身。在具身纪元的 RSI 矩阵里它属于“训练时自迭代”，但更新的对象很小。

7. 延伸批判

“从失败学”的对照实验（Q-Planning 90/80 vs 成功样本 SFT 55/30）是本仓库最干净的一组数字，因为两者用的是同一批 rollout。它反过来给 VERITAS 与 RoboClaw 这类“只回收成功数据”的流水线提了一个问题：丢掉失败样本丢掉了多少信息？一个便宜的后续实验是把 Q-Planning 的 Q 函数当作验证器接进 ENPIRE 的 EN 模块或 AGM 的门控——价值函数与验证器在数学上是同一类对象（对状态-动作的评估），把它们统一起来是 T9 与 T23 合流的自然路径。

30 · TwinRL 与数字孪生谱系：Sim 是 sandbox，不是训练场

覆盖：TwinRL (arXiv 2602.09023, 2026-02) · RoboSnap (2607.06699, 2026-07) · Agentic Real2Sim (2607.19190, 2026-07) · ConCent (2606.30268, 2026-06) · PerceptTwin (2606.04226, ICRA 2026) · Real2Sim via Active Perception (2601.08454, 2026-01) · BestMan (2606.18646) · SafeDojo (2606.20698)
所属主线：T11、T9、T12 · 材料来源：小红书长文 05 节 (Real → Sim → Learn → Verify → Real)；检索地图 T11 代表工作；报告第 2.5 节、第 7 章 I6

1. 一句话定位

小红书长文的判断是 Sim 的角色是 Physical Agent 的 sandbox 而不是回到"全部在仿真里训练"。TwinRL 是这一判断的实证：手机拍摄重建数字孪生，SFT 阶段扩展轨迹分布支撑，twin 内并行 RL 预热真机 RL 并定位易失败配置，四个任务接近 100% 成功且只用 20 分钟真机交互。2026 年的其余工作让"建 twin"的成本快速下降：RoboSnap 从单张 RGB 图生成可交互场景（发布 DROID-Sim 564 个场景），Agentic Real2Sim 让 VLM agent 把真实交互录像转成可仿真的 episodic twin，ConCent 以接触事件序列为学习目标做 real-to-sim-to-real，PerceptTwin 从机器人感知栈自动构建交互仿真来验证与精炼计划（GPT-5 系列规划器的计划成功率平均 +39%），Real2Sim via Active Perception 让 VLM 生成行为树主动获取缺失的物理参数，BestMan 提供自动场景生成 + 硬件无关中间件的平台，SafeDojo 在交互式视频世界模型上做安全约束的 RL。

2. 各篇要点

TwinRL：系统真机实验发现在线 RL 的有效探索空间基本被 SFT 诱导的轨迹分布约束。方法：数字孪生 → SFT 阶段用 twin 扩大轨迹分布 → twin 内并行 RL 预热真机 RL 并识别易失败配置 → 真机 RL。四任务接近 100%，20 分钟真机交互。

RoboSnap：分层设计把物理关键的交互区域与周围视觉上下文分开——碰撞感知的前景资产为稳定交互而精炼，3D 高斯泼溅提供视觉背景；单张 RGB 图 → 可仿真场景；发布 DROID-Sim 564 个场景并给出 sim-real 相关性。

Agentic Real2Sim：real2sim 需要的不只是视觉重建——恢复几何与物体状态、推断物理参数、把 actor/物体/相机/姿势/轨迹装配成可运行仿真；现状依赖手工调视觉基础模型、清理网格、对齐坐标系与脆弱的工作流胶水。方法：VLM agent 驱动的物理世界建模，开源模型即可。

ConCent：sim-to-real 的差异集中在接触动力学；把接触事件序列作为学习目标，避免策略利用不真实的仿真接触。

PerceptTwin：从感知栈的语义场景表示（开放词汇物体地图 + 3D 资产生成 + affordance 预测 + 常识）自动构建交互仿真，用于 LLM 规划的迭代验证与精炼；GPT-5 系列规划器 +39%。

Real2Sim via Active Perception：给定语言请求、不完整的仿真描述与视觉观测，VLM 做语义任务分解并生成行为树，主动交互获取缺失的物理参数，避免任务无关的穷举探索。

SafeDojo：安全 RL 要么需要昂贵的真机探索、要么依赖手工安全函数；SafeDojo 在世界模型的想象中学安全动作，是"twin 作为安全验证器"的 RL 版本。

3. 口径与局限

- TwinRL 的"接近 100%、20 分钟"是四个任务上的真机数字，任务复杂度与试验数见正文。
- RoboSnap 的 sim-real 相关性是评测层指标，不是策略成功率；DROID-Sim 564 场景的多样性来自 DROID 数据集。
- PerceptTwin 的 +39% 是"计划成功率"（在 twin 中验证通过），不是真机执行成功率。
- 所有 twin 都继承感知重建的误差；ConCent 提醒接触参数的偏差可以完全改变任务结果。

4. 关系定位与延伸批判

这组工作把 T11 的角色从"训练场"改成"验证器的一部分"：agent 生成的新技能或计划先在物理仿真里跑一遍（PerceptTwin），RL 先在 twin 里预热并找出易失败配置（TwinRL），安全约束先在想象里学（SafeDojo）。小红书长文描绘的流水线（生成技能 → 静态检查 → 数字孪生 rollout → 域随机化 → 失败挖掘 → 安全验证 → 硬件在环 → 小规模真机 → 真实反馈 → 更新仿真）在 2026 年每一步都有工具，缺的是把它们串起来的 harness（报告开放问题第 6 条）。具身纪元文章说"Robot RSI 缺一个虚拟世界"并寄望世界模型比传统仿真更可扩展——这组工作给出的更谨慎的答案是：可扩展性来自"建 twin 的成本"下降（单张图、一段录像、感知栈直接生成），而 twin 的可信度仍取决于接触建模（ConCent）。未走之路：没有一篇把 twin 的保真度与它作为验证器的假阳性/假阴性率联系起来——"twin 里过了、真机上挂了"的比例才是衡量 sandbox 价值的指标。

31 · RoboClaw 深度解读：让机器人同时学会"做任务"与"恢复现场"

RoboClaw: An Agentic Framework for Scalable Long-Horizon Robotic Tasks arXiv 2603.11558 (2026-03-12) · 智元机器人 · 新加坡国立大学 · 上海交通大学 (具身纪元文章注) · Ruiying Li, Yunlang Zhou, YuYao Zhu et al. (共 18 位作者) 所属主线: T4、T10、T7、T24 · 材料来源: 小红书长文 04 节; 具身纪元文章"训练时改进"节; 小红书 Robot RSI 笔记

1. 一句话定位

RoboClaw 用一个 VLM 驱动的统一数据采集、策略学习与任务执行，并在策略层引入 Entangled Action Pairs (EAP)：把正向技能与逆向恢复技能绑定，让机器人在完成任务后自己把场景复位——不是倒放录像，而是单独学会反向任务。正反任务交替运行并回收成功数据，长程成功率 +25%，人工投入时间 -53.7%。它补上的是自动复位与持续数据采集，也就是机器人 RL 里"最贵的是人"这一成本结构。

2. 要解决的问题

VLA 在语言驱动的操作上潜力大，但扩展到长程任务困难：现有流水线把数据采集、策略学习、部署分开，导致严重依赖人工复位环境，以及脆弱的多策略执行。每一次真机 rollout 之后都要有人把场景摆回去，是数据回流最大的隐性成本。

3. 方法

- **单一 VLM 控制器**：负责调度——采集什么、训练什么、执行什么。
- **Entangled Action Pairs**：每个正向技能（把妆前乳放进抽屉）配一个逆向恢复技能（把它取出来放回原处），两者作为一对被学习与调度；正反交替运行使场景自动回到起点，成功数据被回收进训练。
- **持续采集**：Collection ↔ Learning ↔ Deployment 在同一 agent 循环里闭合（Lil'Log 工作流自动化模式的机器人版本）。

4. 实验结果与口径

- 长程任务成功率 +25%（相对基线）；人工投入时间 -53.7%。
- 口径：真机（具身纪元文章的例子是抽屉场景）；+25% 为相对/绝对提升需看正文；-53.7% 是人工时间的相对减少，是本仓库少数直接度量"人力"的数字。

5. 局限

1. 逆向技能并非总是存在：不可逆操作（切割、倒水、剪扎带）没有"恢复现场"的技能，EAP 覆盖不了——ENPIRE 的剪扎带任务用程序化工具复位到"最近一次完整尝试的起点"是另一条路。
2. 逆向技能本身也要学、也会失败；恢复失败时的处理摘要未说明。
3. 只回收成功数据（"回收成功数据"）——Q-Planning (notes/29) 证明失败样本对自改进同样重要。

6. 关系定位

RoboClaw 与 ENPIRE 的 EN 模块 (notes/06) 解决同一问题：自动复位是数据回流闭环的前提。前者把复位学成技能，后者把复位做成工具调用。在小红书长文的三个闭环里，RoboClaw 让 Learning Loop 不再需要人在场；在具身纪元的 RSI 矩阵里它是"训练时自迭代"的机器人侧例子（部署现场自己生产下一版策略的数据）。它与 LWD (notes/28) 互补：LWD 假设数据回流基础设施存在，RoboClaw 提供了这个基础设施的一个实现。

7. 延伸批判

-53.7% 人工时间是一个比成功率更有产业意义的数字，但它的口径需要说明：剩下的 46.3% 花在哪里（异常处理？任务定义？验证？）。如果剩余人力集中在验证与异常处理，那么 RoboClaw 的下一步不是更多 EAP，而是接入 AGM/PRIMO R1 式的验证器 (notes/12、17)。另一个未走之路是把 EAP 的"正反配对"推广为 BATON 意义上的"进入条件" (notes/14)：逆向技能的终态就是正向技能的进入分布，EAP 天然给出了一个可采样的进入条件——这一点论文没有利用。

32 · VLA + RL 合评：信用分配、免外部奖励与测试时 RL

覆盖：Temporal GRPO (arXiv 2608.13026, 2026-08) · TEMPO (2608.07314, 2026-08) · WCM (2607.29613, 2026-07) · Z-1 (2606.31846, 2026-06) · SAC Flow (2509.25756, 2025-09) · TT-VLA (2601.06748, 2026-01) · RL²-VLA (2607.26991, 2026-07) · T²VLA (2606.29892, 2026-06) · CaP-RL (见 notes/02) 所属主线：T9 · 材料来源：检索地图 T9 代表工作；报告第 4 章 T9

1. 一句话定位

VLA + RL 在 2026 年从“能不能”变成了“怎么分配信用、怎么在没有奖励时做、怎么在测试时做”。信用分配：Temporal GRPO 按可检测的任务阶段分配优势，解决轨迹级信用混叠；TEMPO 冻结 VLM 主干，语义投影层低频、动作专家高频的双时间尺度更新；WCM 让 critic 同时预测未来 latent 与价值以匹配部分可观测性（149 个任务上 SOTA）。免外部奖励：T²VLA 发现离散动作 VLA 的生成置信度与成功显著相关，用它作内在奖励做测试时 RL；RL²-VLA 只在预测失败时激活潜空间组合式引导（OOD 最多 +17.3%）。效率：Z-1 在 $\pi 0.5$ 上只用公开 RoboCasa 示范做 SFT 再做任务级 GRPO，24 任务 80.6%；SAC Flow 把 flow rollout 视为残差 RNN，用门控/Transformer 速度网络稳定 off-policy RL；TT-VLA 用逐步任务进度的密集奖励做测试时 RL。

2. 各篇要点

Temporal GRPO：GRPO 式后训练把一个 rollout 级优势加到轨迹里每个动作上，完成了几个有效阶段却在后面失败的 rollout 会惩罚产生早期进度的动作——“轨迹级信用混叠”。方法：构造可检测的任务阶段，把每个 rollout 与阶段特定的动作区间对齐，只比较同阶段的 rollout。它把“阶段检测”（一个验证器问题）变成了 RL 信用分配的前提。

TEMPO：SFT 易分布不匹配，现有 RL 对所有组件用同一更新策略。方法：冻结预训练视觉语言主干保留通用语义，适配限于两个组件——语义投影层（慢时间尺度）与动作专家（快时间尺度）。它是双系统思想在训练动力学上的体现。

WCM：critic 基于单帧观测或单帧 VLM latent，与机器人控制的部分可观测性根本不匹配；朴素地加历史会指数复杂且标量回报回归监督不足。方法：World Critic Model，同时预测未来 latent 与价值；149 任务 SOTA（报告转述）。

Z-1：面向 flow-based VLA 的 RL 后训练；基于 $\pi 0.5$ ，只用公开 RoboCasa 示范 SFT，再任务级 GRPO；RoboCasa 24 任务 80.6%（报告转述）。

SAC Flow：flow 策略的 off-policy RL 不稳定，因为 flow rollout 在代数上等价于残差递归计算，会像 RNN 一样梯度消失/爆炸；用现代序列模型原理重参数化速度网络（Flow-G 门控、Flow-T Transformer）。

TT-VLA / T²VLA / RL²-VLA：三种测试时改进——TT-VLA 用逐步进度奖励在部署中做 RL；T²VLA 用生成置信度当内在奖励，架构无关；RL²-VLA 自适应地只在基座可能失败时介入，用潜空间组合式引导打破“动作样本集中在相似行为、继承相关失败模式”的问题。

3. 口径与局限

- 80.6% (RoboCasa 24 任务)、+17.3% (OOD)、149 任务 SOTA 分别来自不同基准与基座，不可并排。
- Temporal GRPO 需要"可检测的阶段"，其检测器的准确率决定信用分配的正确性——阶段检测与 PRIMO R1 的进度估计 (notes/17) 是同一问题。
- 置信度作为内在奖励 (T^2 VLA) 有 Consilience (notes/19) 讨论的过度乐观风险：置信高不等于物理成功。
- 绝大多数在仿真；ARLI (notes/27) 指出真机 RL 还要处理推理延迟。

4. 关系定位与延伸批判

这一组与 LWD (notes/28)、Q-Planning (notes/29)、TwinRL (notes/30) 合起来构成 T9 的完整图景：训练时 (Z-1、TEMPO、WCM、SAC Flow)、部署时 (LWD、Q-Planning、TT-VLA、 T^2 VLA、 RL^2 -VLA)、仿真预热 (TwinRL)。三个观察：(1) 信用分配正在向"阶段"收敛——Temporal GRPO 的阶段、BATON 的子任务、AGM 的子目标、PRIMO R1 的进度是同一对象在 RL、探索、记忆、验证四个视角下的名字；(2) "何时介入"成为共同问题—— RL^2 -VLA 的失败预测门控与 StreamVLA 的完成态门控 (notes/26) 同构；(3) 免外部奖励的代价是信号来自策略自身，Lil'Log 的奖励作弊警告在这里最锋利。未走之路：把 LLM-as-a-Verifier 的连续分数 (notes/19) 当作 Temporal GRPO 的阶段奖励，是把 T23 与 T9 接起来最便宜的实验。

33 · 世界模型的角色合评：H-WM、Motus2、ContactGuard、SafeDojo、Online Continual RL、RoboGene

覆盖：H-WM (arXiv 2602.11291, 2026-02) · Motus2 (2608.30237, 2026-08) · ContactGuard (2608.13438, 2026-08) · SafeDojo (2606.20698, 2026-06) · Online Continual RL with World Model Feedback (2603.04029, 2026-03) · RoboGene (2602.16444, 2026-02) 所属主线：T12、T4、T10、T17 · 材料来源：小红书长文 05 节 ("脑内预演")；检索地图 T12 代表工作；报告第 4 章 T12

1. 一句话定位

小红书长文说世界模型是 Agent"脑内预演"的地方，而不只是动作预测器。2026 年的论文给了这个角色更清楚的分工：预演/引导 (H-WM 用逻辑 + 视觉分层世界模型给 VLA 稳定的中间引导)、闭环学习的统一接口 (Motus2 用一个共享权重的模型同时暴露策略、模拟器、评估器三个接口)、执行监控 (ContactGuard 在机器人真正接触前用潜空间世界模型预测并中止可能的失败)、安全训练 (SafeDojo 在想象中学安全动作)、变化检测 (Online Continual RL 用 DreamerV3 的预测残差检测 OOD 事件并自动触发微调)。RoboGene 虽不是世界模型，但它用多样性驱动 + 自反思物理约束的 agent 自动生成真实操作任务 (18k 轨迹)，属于同一条"用模型生成数据与任务"的线。

2. 各篇要点

H-WM：现有世界模型偏重视频生成或自然语言预测，难以接地到机器人动作且长程误差累积；经典 TAMP 在逻辑空间建模转移、鲁棒但脱离视觉。方法：分层世界模型，逻辑世界模型与视觉世界模型联合预测，为 TAMP 与 VLA 提供同步的符号-视觉状态预测。

Motus2：通用具身 agent 应在统一系统里感知、预测、行动、评估、改进；现有世界模型只是给模拟器接一个动作头，没有耦合成决策-学习闭环。方法：单一共享权重模型暴露三个接口——策略（世界-动作模型）、模拟器、评估器——通过模型与数据 scaling 推进；"自进化"意味着评估器与模拟器为策略改进提供闭环。

ContactGuard：接触密集操作的失败常在机器人已经接触后才被检出，腕部相机设置下尤甚。方法：给定策略计划的动作块，用动作条件的潜空间世界模型预测其短程后果，若预测的未来 latent 指示可能失败则中止。

SafeDojo：安全 RL 要么需要昂贵真机探索、要么依赖手工安全函数；第一个面向 VLA 的基于模型的安全 RL 框架，在交互式世界模型的想象中学安全动作。

Online Continual RL：学习型控制器离线训练、固定参数部署，难以应对运行中的未预见变化。方法：基于 DreamerV3，用世界模型预测残差检测 OOD 事件并自动触发微调，用任务级性能与模型层指标监控适应进度。

RoboGene：机器人数据采集是主动过程、物理成本高，自动化的任务策划 (curation) 未被充分探索；人工方法不可扩展且偏向常见任务，现成基础模型会幻觉出物理上不可行的指令。方法：多样性驱动 + 自反思物理约束的 agentic 框架自动生成真实操作任务，采集 18k 轨迹，用其预训练的 VLA 泛化更好。

3. 口径与局限

- Motus2 的三接口共享权重是架构主张，评估器接口的准确率（作为验证器）摘要未给。

- ContactGuard 的"预测失败并中止"有假阳性代价（该做的接触被中止），摘要未量化。
- SafeDojo 的安全在想象中学，想象与现实的差距（sim2real 的世界模型版本）决定安全是否成立。
- RoboGene 的 18k 轨迹是自动生成任务后采集的真实数据，采集本身仍需机器人与人。

4. 关系定位与延伸批判

把这组工作与数字孪生（notes/30）并排，可以看到两条"虚拟世界"路线正在竞争"Robot RSI 的测试系统"这一角色：几何/物理仿真（twin）保真但建造成本高、接触建模难；学习型世界模型（Motus2、SafeDojo、ContactGuard）可扩展但可信度取决于训练分布。具身纪元文章押注后者更可扩展，本仓库的判断更保守（报告 I6）：世界模型在 Agentic Robot 里最有用的角色是预演器 + 监控器，而不是动作生成器。Motus2 把"评估器"做成世界模型的一个接口，是 T12 与 T23 合流的信号——但也把验证器放进了会被训练的同一组权重里，与"评估器在循环之外"的原则直接冲突。ContactGuard 与 AGM（notes/12）是同一类"接触前/达成后"的门控，二者一前一后，天然可以组合成一个动作块级的验证对。

PART E

总览、接口、多机器人与跨本体

AgenticLab 的规划语言接口、Agent + Robot 总览、ROSClaw 与 MHS 的接口层、多机器人协作、生命周期与主动感知。

1. 34 · AgenticLab / PPlanAR 深度解读：用规划语言定义 VLM 的推理空间
2. 35 · Agent + Robot 总览合评：Agentic Robot、ManiAgent、VoLo、HoloAgent-0 与"灵活但仍脆弱"
3. 36 · 接口层合评：两篇 ROSClaw、MHS、OpenClaw 生态与 MCP
4. 37 · 多机器人协作合评：RoCo、MALMM、REMAC、RoboOS 系列、DynaHMRC、Scale-Plan、MeCo、Tool-RoCo、对话对齐与通信攻击
5. 38 · 生命周期、跨本体与主动感知合评：Arcadia、PhysCaP、ActiveVLA 与相关工作

34 · AgenticLab / PPlanAR 深度解读：用规划语言定义 VLM 的推理空间

PPlanAR: Planning–Language–Grounded Agentic Reasoning for Robot Manipulation (v1 题名；检索地图称 AgenticLab) arXiv 2602.01662 (2026–02–02) · Purdue · Pengyuan Guo, Zhonghao Mai, Zhengtong Xu et al. (共 11 位作者) · 项目页 agentic1ab.github.io 所属主线：T1、T6、T22、T23 · 材料来源：检索地图 T1/T6/T22 代表工作；报告第 4 章 T1

1. 一句话定位

AgenticLab 是 Agent + Robot 总览主线的代表性真机平台：它不让 VLM 用自由自然语言推理，而是用**规划语言接口**定义 VLM 的推理空间——物体谓词表示场景状态，动作 schema 带前置条件与效果，符号计划作为可执行的中间表示；每一步执行后用板载观测检查符号效果是否达成，不达成就重规划。它把 LLM+P (2023) "把问题翻译成 PDDL 交给规划器"的思路推进为"让 VLM 在规划语言里思考并在线验证"。

2. 要解决的问题

非结构化环境里的长程操作要求 VLM 推理不断变化的场景状态、动作约束与执行结果，纯自然语言推理难以做到：状态描述随意、约束隐含、结果无法核对。需要一个既能被 VLM 生成、又能被程序检查的表示。

3. 方法

- **规划语言接口**：谓词（`on(A,B)`、`holding(x)` 一类）描述场景，动作 schema 显式写出前置条件与效果，VLM 的输出被约束为这一语言里的符号计划。
- **在线验证**：每个动作执行后，用板载感知检查 schema 声明的效果是否在世界中成立——这是 T23 主线上最古典的验证器形式（前置/后置条件），也是 T22 主动感知的一个弱形式（为核对效果而观察）。
- **重规划**：效果不成立时触发重规划，而不是继续执行错误计划。

4. 实验结果与口径

- 真机开放词汇长程操作任务；摘要未给单一头条数字，具体任务与成功率见正文与项目页。
- 口径：真机、长程、开放词汇；与 Agentic Robot (LIBERO 79.6%，仿真)、ManiAgent (SimplerEnv 86.8%，仿真) 不同协议。

5. 局限

1. 规划语言的表达力决定上限：几何、接触、连续量（"往左一点"）在谓词里没有自然表示——这正是 Code as Policies (notes/01) 用代码解决的问题。
2. 谓词的真值由感知判定，感知错误会被当作"效果不成立"触发无谓重规划，或反过来漏检。
3. "AgenticLab"作为平台名与论文题名 PPlanAR 不一致，检索时需两名并查（报告 1.2 节已溯源）。

6. 关系定位

AgenticLab 与 Agentic Robot、ManiAgent (notes/35) 共同定义了 2025–2026 年"感知 → 分解 → 执行 → 验证 → 重规划"的闭环骨架；它的独特之处是把验证做进了表示本身。与 Thea 的 Scene Graph as Context (notes/22) 比较，两者都用符号化世界表示当 agent 的上下文，AgenticLab 更强调动作 schema 的前置/后置条件。与 REMAC 的前置/后置条件检查 (notes/16) 同源。

7. 延伸批判

报告第 4 章 T1 的判断——闭环骨架已经标准化，可靠性瓶颈在验证与 grounding 而不在规划——AgenticLab 是这一判断的正面例子：它的规划质量由规划语言保证，成败取决于谓词能否被可靠感知。因此对它最有价值的扩展不是更强的 VLM，而是让谓词的真值带置信度（AGM 式的物理证据，notes/12），并在低置信时主动感知（PhysCaP 式的交互探索，notes/38）。另一个未走之路：规划语言接口天然适合形式验证（VASO 的模型检查，notes/09），把两者接起来可以在执行前就过滤掉违反安全约束的计划。

35 · Agent + Robot 总览合评：Agentic Robot、ManiAgent、VoLo、HoloAgent-0 与"灵活但仍脆弱"

覆盖：Agentic Robot (arXiv 2505.23450, 2025-05) · ManiAgent (2510.11660, 2025-10) · VoLo (2606.07723, 2026-06) · HoloAgent-0 (2606.23565, 2026-06) · Agentic AI for Robot Control: Flexible but still Fragile (2602.13081, 2026-02) · Cortex (见 notes/24) · Guava (见 notes/24) 所属主线：T1、T4、T23
· 材料来源：检索地图 T1 代表工作；报告第 4 章 T1

1. 一句话定位

T1 主线回答"把 LLM/VLM 的推理接到真实机械臂上能否形成闭环"。Agentic Robot 用 Standardized Action Procedure 协调推理模型、VLA 执行器与时序验证器 (LIBERO 长程 79.6%)；ManiAgent 用多 agent 协作做环境感知、任务分解与动作生成 (SimplerEnv 86.8%)，并用 agent 给 VLA 生成训练数据；VoLo 让 VLM 把 VLA/WAM 当作可中断的工具在执行中途干预，并把"决策、动作与工具调用的时序在不会暂停的物理世界里很重要"命名为物理编排 (Physical Orchestration)；HoloAgent-0 用 Embodied AgentOS + 3D 空间记忆把操作、空间理解、导航、人形控制统一进一个执行循环；"Flexible but still Fragile"则是这条线最清醒的真机报告：两台真机平台之间迁移只需改系统 prompt，但非确定性行为、指令遵循错误与对 prompt 措辞的高敏感性普遍存在。

2. 各篇要点

Agentic Robot：长程操作的误差累积与执行中缺少验证机制；脑启发的三角色——推理模型规划、VLA 执行、时序验证器核对——由 Standardized Action Procedure 协调。LIBERO 长程 79.6%。

ManiAgent：VLA 在复杂推理与长程规划上受数据与容量限制；多 agent 通过 agent 间通信完成环境感知、子任务分解与动作生成，从任务描述到动作端到端输出；SimplerEnv 86.8%；还能给 VLA 生成训练数据——agent 系统作为数据引擎的早期例子。

VoLo：闭环 agent 循环中 VLM 编排异构机器人能力为可中断工具；与虚拟 agent 不同，物理世界不为推理暂停，因此何时决策、何时中断、何时调用工具的时序是一等问题。它把 T13 双系统的"何时切换" (notes/26) 提升为 agent 层的编排问题。

HoloAgent-0：LLM agent 在数字环境的循环 (结构化状态 → 调用工具 → 检查反馈 → 修订) 难以直接延伸到连续、依赖本体、不确定且受安全约束的物理执行；用 Embodied AgentOS 与 3D 空间记忆统一多种具身能力。

Flexible but still Fragile：推理型语言模型在迭代的规划器-执行器循环里选择并调用机器人技能，部署在两个平台 (室内移动操作 Mobipick 的桌面抓放与箱体插入；农业自主导航)。发现：跨平台迁移只改系统 prompt；但非确定性、指令遵循错误、prompt 敏感普遍存在。

3. 口径与局限

- 79.6% (LIBERO 长程) 与 86.8% (SimplerEnv) 是仿真基准上的成功率，基座与任务集不同，不可并排；"Fragile"论文是真机定性 + 少量统计。

- 多 agent 架构 (ManiAgent) 的 agent 间通信开销与非确定性未量化——Tool-RoCo (notes/37) 后来发现 agent 很少把彼此当工具用。
- VoLo 的"可中断"依赖 VLA/WAM 支持中途中断与安全停止, 这是硬件与策略层的前提。

4. 关系定位与延伸批判

把这组工作与 AgenticLab (notes/34)、Cortex 与 Guava (notes/24) 放在一起, T1 的骨架已经标准化: 感知 → 分解 → 执行 → 验证 → 重规划, 差别只在验证器 (时序验证器 / 谓词效果检查 / exit codes) 与执行器 (VLA / 代码 / 混合)。
"Flexible but still Fragile"给出的三条脆弱性——非确定性、指令遵循错误、prompt 敏感——恰好对应 ROSClaw 的发现 (不同前沿模型越权动作率差 3.4–4.8 倍, 执行层设计比 prompt 措辞更影响安全, notes/36): 可靠性来自执行层与验证层, 不来自更好的 prompt。这也是本仓库把 T15/T16/T23 视为 2026 年重心的原因。未走之路: VoLo 的"物理编排"时序问题与 Harness Engineering for Physical AI 的 Isolation (notes/23) 是同一问题的 agent 层与中间件层表述, 尚无工作把两层的时序预算统一起来。

36 · 接口层合评：两篇 ROSClaw、MHS、OpenClaw 生态与 MCP

覆盖：ROSClaw: An OpenClaw ROS 2 Framework (arXiv 2603.26997, 2026-03) · ROSClaw: Hierarchical Semantic-Physical Framework for Heterogeneous Multi-Agent Collaboration (2604.04664, 2026-04) · Anthropic Model Hardware Standard (研究预览, 2026-08-27) · OpenClawPi (松灵机器人技能库) · OpenGo (2604.01708) · AgentRob (2602.13591) · ROSBag MCP Server (2511.03497) · Contract-Grounded BT Synthesis (2607.12220) · ChemBot (2604.15671) 所属主线：T3、T18、T17、T20 · 材料来源：检索地图 T3/T18 代表工作；报告第 4 章 T3/T18

1. 一句话定位

接口层回答“任何基础模型怎么接到任何机器人”。两篇同名的 ROSClaw 目标不同：Cardenas 等把 OpenClaw agent runtime 接到 ROS 2，做成**模型无关的执行层**——能力发现与 affordance 注入、观测归一化、安全包络内的预执行验证、审计日志；换模型或换平台只是改配置，并顺带成为一件测量仪器：同一基底上不同前沿模型的越权动作提议率相差 3.4–4.8 倍，且执行层设计比 prompt 措辞更影响任务完成与安全行为。Zhao 等的 ROSClaw 面向异构多机器人，用 e-URDF 物理约束构建 sim-real 拓扑映射，把采集、训练与执行放进统一的 VLM 控制器。Anthropic 的 MHS (2026-08-27) 是标志性事件：一套让 agent 发现、操作、排障任意物理设备的共享规范，被媒体称为“硬件版 MCP”，可通过 MCP、CLI 或代码调用。MCP 正在成为机器人侧的通用接口：ROSBag MCP Server、ChemBot 的 MCP 子 agent 编排、Contract-Grounded BT Synthesis 让 coding agent 先向机器人侧 MCP server 拉取技能契约再合成行为树、AgentRob 通过 MCP 把论坛 agent 连到 Unitree Go2/G1（也展示了被劫持的风险）、OpenGo 是 OpenClaw 驱动的机器狗（技能库 + 调度器 + 反馈自学习）。

2. 各篇要点

ROSClaw (OpenClaw ROS 2)：今天把基础模型接到机器人需要把感知、执行与安全耦合到单一模型和平台的定制集成。方法：模型无关执行层——动态能力发现 + 标准化 affordance 注入、观测归一化、可配置安全包络内的预执行动作验证、审计日志。发现：越权动作提议率跨模型差 3.4–4.8 倍；执行层设计 > prompt 措辞。

ROSClaw (异构)：LLM + 具身 agent 提升了高层推理，但语义理解与物理执行之间仍有缝；VLA/VLN 在长程、时序结构任务上吃力。方法：分层语义-物理框架，e-URDF 物理约束、sim-real 拓扑映射、统一 VLM 控制器串起采集/训练/执行。

MHS：让 agent 标准化地发现、操作与排障显微镜、液体处理器、相机、机械臂、激光系统等真实设备的共享规范；来源是 Anthropic 研究预览与媒体报道（非论文）。

Contract-Grounded BT Synthesis：从自然语言合成可部署行为树要求接地——每个生成的 BT 只引用机器人真能执行的技能；现有方法把接地责任推给 prompt 作者。方法：coding agent 先从机器人侧 MCP server 拉取技能契约（有哪些技能、参数如何、运行时对 BT 结构的约束），再合成 BT。

AgentRob：把在线论坛、LLM agent 与真机通过 MCP 桥接——agent 读帖、抽取自然语言命令并执行；论文同时是一份威胁模型：论坛介导的机器人可被劫持。

ROSBag MCP Server / ChemBot / OpenGo / OpenClawPi: MCP 分析 ROS bag、化学实验室 VLA agent 的 MCP 子 agent 编排与双层记忆、OpenClaw 机器狗的实时技能切换、松灵的 OpenClaw 技能库——生态层的实例。

3. 口径与局限

- ROSClaw 的"3.4–4.8 倍"是越权动作**提议率**的跨模型比值，不是任务成功率；越权由安全包络定义。
- MHS 是研究预览，没有论文与数字；能否成为事实标准取决于采用。
- MCP 相关工作多为系统描述与演示，少有统计评测。

4. 关系定位与延伸批判

报告第 4 章 T3 的判断：模型无关的执行层是 2026 年基础设施层最实用的进展，它把"安全包络"与"审计"做成了配置项。这与 Runtime Governance 的外置治理 (notes/25) 同向，与"Flexible but still Fragile"的发现（可靠性来自执行层，notes/35）互证。接口标准化决定 harness 层的可移植性：Harness Engineering for Physical AI 提议的 ROS 2 Harness Profile (notes/23) 与 MHS 是两条正在逼近的标准线，是否融合值得跟踪（报告开放问题第 4 条）。最值得警惕的是 AgentRob 展示的攻击面：接口越标准、越开放，"任何模型接任何机器人"就意味着"任何被劫持的模型接任何机器人"——T18 与 T17 必须同步推进，而目前 MHS 的预览没有公开其权限模型。

37 · 多机器人协作合评：RoCo、MALMM、REMAC、RoboOS 系列、DynaHMRC、Scale-Plan、MeCo、Tool-RoCo、对话对齐与通信攻击

覆盖：RoCo (arXiv 2307.04738, 2023-07) · MALMM (2411.17636, 2024-11) · REMAC (2503.22122, 见 notes/16) · RoboOS (2505.03673, 2025-05) · RoboOS-NeXT (2510.26536, 2025-10) · DynaHMRC (2606.14882, 2026-06) · Scale-Plan (2603.08814, 2026-03) · MeCo (2601.20577, 2026-01) · Tool-RoCo (2511.21510, 2025-11) · World-Model Alignment Through Dialogue (2605.12920, 2026-05) · Communication Attacks in LLM Multi-Robot (2608.06830, 2026-08) 所属主线：T19、T20、T21、T17 · 材料来源：小红书长文 06 节；检索地图 T19/T20/T21 代表工作；报告第 2.6 节、第 4 章 T19-T21

1. 一句话定位

小红书长文说大规模进化不该发生在一台机器人身上，并列四共层（技能、经验、世界知识、策略改进）。论文覆盖的是两端：RoboOS / RoboOS-NeXT 的共享时空-本体记忆（状态层）与 LWD 的共享策略改进（notes/28）；中间的共享经验与共享世界知识还没有系统工作。语言层的协作方法已经不缺：RoCo (2023) 让机器人用 LLM 对话协商分工与航点再交给多臂运动规划器；MALMM 用 Planner/Coder/Supervisor 三 agent 零样本完成 RL Bench 任务；DynaHMRC 做去中心化角色感知 agent 与领导者竞选；Scale-Plan 用 LLM 引导的动作图搜索裁剪 PDDL 问题；MeCo 用相似任务记忆化避免重复规划。缺的是两样东西：共享记忆的一致性与通信的可信性——Tool-RoCo 发现 LLM agent 很少把其他 agent 当助手调用（协作工具只占 7.09%），对话对齐研究发现对话把动作冲突减少 40-83 个百分点却降低任务成功，通信攻击研究发现三种架构下不安全信息都能变成不安全动作（最高 97.8%），Claim Provenance and Verification Gate 能把违规率从 70.0% 降到 36.6%。

2. 各篇要点

- RoCo**：LLM 同时做高层通信与低层路径规划——机器人各带 LLM 讨论策略、生成子任务计划与任务空间航点，多臂运动规划器加速轨迹规划；环境反馈（碰撞检查）回灌 LLM 重规划。它是 T19 的起点。
- MALMM**：LLM 规划在长程上幻觉、单次生成缺乏实时反馈；三 agent（规划、写代码、监督）零样本完成 RL Bench 操作任务。
- RoboOS / RoboOS-NeXT**：跨本体与多 agent 协作缺少自适应、调度与动态纠错；RoboOS 用 Brain-Cerebellum 分层与实时共享记忆协调多本体；RoboOS-NeXT 指出现有方法依赖有限或个体 agent 记忆，用统一的 Spatio-Temporal-Embodiment Memory 做终身、可扩展、鲁棒的多机器人协作。
- DynaHMRC**：集中式 LLM 调度随团队规模与环境复杂度扩展性差（单模型处理过多上下文、长上下文近似退化）；去中心化角色感知 agent 与领导者竞选。
- Scale-Plan**：异构团队长程规划被大量无关感知信息拖累；LLM 引导的动作图搜索裁剪 PDDL 问题规格，兼顾符号规划的可靠与 LLM 的灵活。
- MeCo**：LLM 多机器人协作每次从头规划；相似任务记忆化复用先前方案。它是共享经验层的雏形。

Tool-RoCo: 在 RoCo 上构建的长期多 agent 协作基准, 把其他 agent 当作工具、引入协作工具, 用工具使用评测自组织; 发现协作工具只占调用的 7.09%——agent 不主动求助。

对话对齐: 部分可观测下无通信的协调是可证明困难的, 通信原则上可以对齐世界模型; 实证发现对话减少动作冲突 40–83 个百分点却降低任务成功, 并给出世界模型对齐的度量。

通信攻击: LLM 多机器人系统的通信风险此前研究不足; 在三种架构 (含 DMAS/HMAS) 下, 不安全信息经通信变成不安全动作, 最高 97.8% 不安全动作成功率; Claim Provenance and Verification Gate 把违规率 70.0% → 36.6%。

3. 口径与局限

- 7.09%、40–83pp、97.8%、70.0→36.6% 分别测的是工具调用比例、冲突减少幅度、攻击成功率与违规率, 没有一个是任务成功率; 它们描述的是协作的**质量与风险**, 不是能力。
- RoCo、MALMM 等的成功率来自各自仿真基准, 任务集不同。
- 共享记忆的一致性 (两台机器人对同一物体状态的记录冲突时如何裁决) 在 RoboOS–NeXT 摘要中未展开。

4. 关系定位与延伸批判

T19–T21 的现状可以概括为: 语言层方法充足、状态层记忆已有、经验层与世界知识层空缺、可信性刚被正视。三个含义: (1) "共享经验会共享污染"——通信攻击的 97.8% 与 Memory Anchors (notes/15) 的锚点淹没问题提示 fleet 学习需要数据准入机制, LWD (notes/28) 没有讨论; (2) 对话减少冲突却降低成功, 说明协作的目标函数不是"少冲突"而是"完成任务", 多 agent 系统的复杂度本身是风险来源 (报告 I10); (3) FSAR (notes/25) 主张 fleet 层联邦单 agent 运行而非机器人内部多 agent 碎片化, 与 Tool-RoCo 的 7.09% 一起构成对"多 agent 一定更好"的反证。未走之路: Thea 的场景图 (notes/22) 与 RoboOS–NeXT 的 STEM 记忆是共享世界知识层的两个候选载体, 把它们与 Claim Provenance 式的来源验证结合, 是填补中间两层最直接的方向。

38 · 生命周期、跨本体与主动感知合评：Arcadia、PhysCaP、ActiveVLA 与相关工作

覆盖：Arcadia (arXiv 2512.00076, 2025-11) · PhysCaP (2608.21031, 2026-08) · ActiveVLA (2601.08325, 2026-01) · Real2Sim via Active Perception (见 notes/30) · 跨本体证据：RoboOS (notes/37)、ASPIRE (notes/04)、Harness VLA (notes/20)、BestMan (notes/30)、PhyAgentOS (notes/21) 所属主线：T7、T10、T11、T20、T22 · 材料来源：检索地图 T7/T20/T22 代表工作；报告第 4 章 T7/T20/T22

1. 一句话定位

三条较小的主线在这里合评。**Arcadia** 主张具身学习是生命周期问题而非单阶段优化：自进化探索与接地（自主采数据）、生成式场景重建与增强、共享具身表征、sim-from-real 评估与演化四阶段紧耦合，去掉任一阶段就退回一次性训练。**PhysCaP** 给 Code-as-Policy agent 加一层物理信息驱动的主动探索：训练无关的物理属性提取模块从交互（本体感知）估计质量、刚度等潜在属性，Planner 决定何时探索与停止，Prioritizer 过滤不合理交互——找隐藏物体、检测空罐、挑成熟的鳄梨。**ActiveVLA** 指出 VLA 依赖静态腕部相机的末端中心视角，无法自适应选视点，方法是关键区域定位 + 主动视点选择与 3D 放大。**跨本体**在本仓库没有独立的代表论文，而是散布在系统工作里：RoboOS 支持异构本体、ASPIRE 的技能跨本体与 API 迁移有初步 sim-to-real 证据、Harness VLA 在 RoboTwin C2R 达 58.4%、ROSClaw（异构）用 e-URDF、PhyAgentOS 在 19+ 本体验证。

2. 各篇要点

Arcadia：只优化一环（采集、仿真、学习或部署）的系统很少能持续改进或泛化。四阶段：（1）自进化探索与接地，在物理环境自主获取数据；（2）生成式场景重建与增强；（3）共享具身表征；（4）sim-from-real 评估与演化。它是小红书长文"Real → Sim → Learn → Verify → Real"流程最早的系统化表述之一。

PhysCaP：VLA 擅长模仿示范但依赖被动观察，无法推断对操作至关重要的潜在物理属性。方法：在 code-as-policy 框架上加物理信息驱动的探索层，显式地通过交互寻求信息；训练无关的属性提取模块；Planner（何时探索/停止）与 Prioritizer（过滤不合理交互）。

ActiveVLA：把主动感知注入 VLA——定位关键区域、选择视点、3D 放大以获得精确的三维操作所需的观测。

3. 口径与局限

- Arcadia 是框架论文，四阶段的收益没有单一数字；"去掉任一阶段就退回一次性训练"是论证而非消融结果。
- PhysCaP 的属性估计是训练无关的物理模型，精度与适用物体范围摘要未给；探索的时间成本（何时停）是关键但未量化。
- 跨本体的证据全部是"系统支持多本体"或"技能在不同 API 上迁移的初步证据"，没有一篇报告跨本体的成功率矩阵。

4. 关系定位与延伸批判

报告第 4 章的两个判断在这里汇合：跨本体在 Agent 层比在策略层容易（代码、技能契约、harness 配置天然可迁移，策略权重不能）；主动感知在 Agent 系统里的形态是“把探索当作可调用的工具”，成本控制是关键。Arcadia 的四阶段与 ENPIRE 的四模块（notes/06）、PhyAgentOS 的系统服务（notes/21）是三种粒度的“生命周期”表述——Arcadia 最宏观、PhyAgentOS 最系统化、ENPIRE 最工程化。PhysCaP 的“何时停止探索”与 AgenticLab 的“效果不成立时重规划”（notes/34）、AGM 的“何时验证”（notes/12）都是同一类元决策：在物理成本下决定要不要再看一眼。未走之路：主动感知与验证器（T23）尚未合流——验证器需要的证据（AGM 的跨视角比较）正是主动感知能提供的东西，让验证器在低置信时调用 ActiveVLA 式的视点选择是一个自然组合。

PART F

安全与治理

护栏模型、形式化、世界模型与部署视角四类安全工作；攻击面已覆盖指令、通信、模型、元数据、接触五个层面。

1. 39 · 安全合评：EMBGuard、SENTINEL、VASO、Same Weights Different Robot、RationalVLA、通信攻击与 AgentRob

39 · 安全合评：EMBGuard、SENTINEL、VASO、Same Weights Different Robot、RationalVLA、通信攻击与 AgentRob

覆盖：EMBGuard (arXiv 2605.30924, ICML 2026) · SENTINEL (2510.12985, 2025-10) · VASO (见 notes/09) · Same Weights, Different Robot (2606.03724, 2026-06) · RationalVLA (见 notes/26) · Communication Attacks (见 notes/37) · AgentRob (见 notes/36) · Runtime Governance / EmbodiedGovBench (见 notes/25) · SafeDojo / ContactGuard (见 notes/33) 所属主线：T17 · 材料来源：检索地图 T17 代表工作；报告第 4 章 T17、第 7 章 I7

1. 一句话定位

T17 在 2026 年从"过滤不安全计划"扩展为四类工作。**护栏模型**：EMBGuard 是第一个 MLLM 安全护栏，把物理风险推理与 agent 策略解耦——对（视觉观测，动作）对识别危险并推理动作条件的风险，2B/4B 模型接近 GPT-5.1 / Gemini-2.5-Pro 且假阳性更低。**形式化**：SENTINEL 在语义解释、计划生成、物理执行三层用时序逻辑做统一形式评测，替代启发式规则与主观的基础模型判断；VASO 把形式验证接进技能自进化。**世界模型**：SafeDojo 在想象中学安全动作，ContactGuard 接触前中止。**部署视角**：Same Weights, Different Robot 指出 VLA 常被当作 checkpoint 定义的对象，但同一归一化输出经动作反归一化与控制器约定后会变成不同的物理动作——安全审查认证了 checkpoint，却漏掉到达控制器的可执行策略；替换一个看似合理的元数据键就能让 LIBERO-Goal 从 28/28 降到 2/28。加上 RationalVLA 的缺陷指令拒绝、多机器人通信攻击（97.8%）与 AgentRob 的劫持风险，安全的攻击面已经覆盖指令、通信、模型、元数据、接触五个层面。

2. 各篇要点

EMBGuard：MLLM 具身 agent 在真实环境中遇到物理危险，现有方法缺少显式的危险识别与动作条件风险推理机制，导致漏判或过度判定。方法：解耦的安全护栏，评估（观测，动作）对；2B/4B 模型，性能接近前沿闭源模型且假阳性更低（报告转述）。

SENTINEL：第一个在语义解释、计划生成、物理执行三层做统一形式安全评测的框架；把实用安全需求接地到形式时序逻辑语义（状态不变量、时序依赖等），而不是启发式规则或主观 FM 判断。

Same Weights, Different Robot：动作反归一化元数据是可执行策略的一部分；只审 checkpoint 会漏掉真实策略。28/28 → 2/28 是一个元数据键替换的后果。

RationalVLA：RAMA 基准的六维缺陷指令（模糊、无关、不可行等），理性层判断该不该执行——"拒绝"作为一等输出。

3. 口径与局限

- EMBGuard 的"接近前沿模型、假阳性更低"是护栏自身的判别指标；SENTINEL 是评测框架；两者都不直接给任务成功率。
- Same Weights 的 28/28 → 2/28 是单一元数据替换的极端案例，说明的是脆弱性存在，不是常见频率。

- 绝大多数安全评测在仿真或受控场景；真机上的持续运行安全数据（Habilis-β 式的 MTBI, notes/27）几乎没有。

4. 关系定位与延伸批判

报告 I7 的判断：安全与治理是 Agent × Robot 里被材料低估、被论文高估的方向——低估是因为观点文章不写它，高估是因为它目前主要在仿真里验证。把 T17 与 T16 (notes/25)、T18 (notes/36) 放在一起，可以看到安全正在从“过滤”扩展到“运行时持续检测 + 恢复 + 可审计 + 升级安全”，并触及动作空间语义这类以前没人看的层面。三个未走之路：(1) 自进化系统的新技能（ASPIRE、Zetta）如何通过 EMBGuard/SENTINEL 式的准入——目前没有任何自进化工作接入护栏；(2) Same Weights 的教训应当扩展到 harness 层：harness 配置（延迟预算、fallback）同样是“可执行策略的一部分”，同样需要被认证；(3) 通信攻击与 AgentRob 提示接口标准化（MHS）必须附带权限模型，而 MHS 预览没有公开它。

PART G

基础：LLM Agent、Harness 工程与 RSI

Lil'Log 两篇、软件侧 RSI 谱系（四个环节）、harness 演化方法（优化阶梯的完整实例集）、AI 研发基准与失败模式。

1. 40 · Lil'Log 两篇深度解读：《LLM Powered Autonomous Agents》与《Harness Engineering for Self-Improvement》
2. 41 · RSI 谱系：从 Good 1965 到 BigBang-V1——软件侧递归自我改进的四个环节
3. 42 · Harness 演化方法合评：ACE、MCE、Meta-Harness、Self-Harness、AHE、Harness Updating ≠ Harness Benefit、DGM、Hyperagents、AlphaEvolve、ShinkaEvolve、ThetaEvolve、GEPA、Promptbreeder、STOP、ADAS、AFlow 与相关工作
4. 43 · AI 研发基准与失败模式合评：PaperBench、RE-Bench、MLE-bench、ScienceAgentBench、CORE-Bench、KernelBench、Early Science Acceleration 与《Why LLMs Aren't Scientists Yet》

40 · Lil'Log 两篇深度解读：《LLM Powered Autonomous Agents》与《Harness Engineering for Self-Improvement》

Lilian Weng, Lil'Log · 2023-06-23 (lilianweng.github.io/posts/2023-06-23-agent/) · 2026-07-04 (lilianweng.github.io/posts/2026-07-04-harness/) 所属主线：T0、T15、T24 · 材料来源：本仓库源材料之四；两文 60 条参考文献全部收入 T0（见 notes/41-43）；报告第 5 章

1. 一句话定位

2023 年那篇给出了 Agent 的经典分解——LLM 大脑 + Planning（子目标分解、反思与精炼）+ Memory（短期 = 上下文，长期 = 外部向量库与快速检索）+ Tool use（外部 API）——是小红书长文那张完整架构图的直系祖先。2026 年那篇把 harness 定义为“围绕基座模型、编排执行的系统：决定模型如何思考与规划、如何调用工具与行动、如何感知与管理上下文、如何存储产物、如何评估结果”，判断近期的递归自我改进不太可能从模型改写自己的权重开始，而更可能首先发生在 harness 层。

2. 2023: LLM Powered Autonomous Agents

- **Planning**：任务分解（CoT、ToT、LLM+P 把问题翻译成 PDDL）与自反思（ReAct 的 Thought/Action/Observation 循环、Reflexion 的反思记忆、Chain of Hindsight、Algorithm Distillation 的上下文内 RL）。
- **Memory**：类比人类的感觉/短期/长期记忆；短期是上下文内学习，长期是外部向量库 + 最大内积搜索（LSH、ANNOY、HNSW、FAISS、ScaNN）。
- **Tool use**：MRKL 的模块化路由、TALM/Toolformer 的自监督工具学习、HuggingGPT 的模型调度、API-Bank 评测、ChemCrow 与 Boiko 等的科学工具 agent。
- **案例与挑战**：Generative Agents 的记忆流 + 反思 + 规划；AutoGPT、GPT-Engineer；三个挑战——有限上下文、长程规划与分解的困难、自然语言接口的不可靠。

对机器人的映射（报告 5.1 节）：Planning → 小红书长文的 Agent/System 2 层；Memory → Memory/Skill/Dataset 三类；Tool use → Harness/Runtime 的 Tool Routing 与 Skill Layer 的四类原语（代码技能、解析原语、VLA 原语、RL 策略）。三个挑战分别对应 T5 记忆、T4 长程、T23 验证器。

3. 2026: Harness Engineering for Self-Improvement

- **三种设计模式**：工作流自动化（plan → execute → observe/test → improve）；文件系统即持久记忆（状态与产物存文件而非塞进上下文）；子代理与后台任务（并行搜索假设、监控作业、合并结果）。
- **优化对象阶梯**：instruction prompts → structured context → workflow → harness code → optimizer code。ACE/MCE 在 context 一级，ADAS/AFlow 在 workflow 一级，Meta-Harness/Self-Harness/AHE 在 harness code 一级，STOP/DGM/Hyperagents 在 optimizer code 一级。
- **两个关键发现**：STOP 的自学优化器在 GPT-4 上有效、在 GPT-3.5 与 Mixtral 上退化；Lin 等（Harness Updating ≠ Harness Benefit）拆出两个轴——写 harness 的能力从 Qwen3.5-9B 到 Claude Opus 4.6 几乎持平，利用 harness 的能力非单调、中等模型受益最大。评估器与权限控制必须

在演化循环之外：AHE 把 runs 目录、tracer、verifier 与 LLM 配置设为只读，才能把收益归因到 harness 编辑而非奖励作弊。

- **自动研究**：AI Scientist、Nature 上的端到端自动化、ScientistOne、Autodata，以及 Trehan & Chopra 总结的六类失败模式（训练数据默认偏置、实现漂移、记忆退化、过度乐观、领域知识不足、科学品味弱）。
- **RSI 七个挑战**：弱而模糊的评估器、上下文与记忆的生命周期、负面结果、多样性塌缩、奖励作弊、长期成功、人的角色。

对机器人的映射（报告 5.2 节）：工作流自动化 → ENPIRE / ASPIRE / RoboClaw；文件系统记忆 → PhyAgentOS 的 State-as-a-File、ENPIRE 的 Markdown 研究总结；子代理 → ENPIRE 的 Agent 团队 × 集群、HARBOR、RATs；阶梯 → SkillOpt (context)、ASPIRE (workflow/库)、RHO/SHAPER (harness code)、Meta-Harness/HarnessOpt (optimizer code)。

4. 两篇没有覆盖的部分

Lil'Log 的 harness 生活在数字世界：状态可读、结果可判、失败可重试一千次。Thea (notes/22) 指出物理世界拒绝白送读状态与判结果；Harness Engineering for Physical AI (notes/23) 指出第三个缺口——时间。这三点是机器人 harness 与软件 harness 的本质差异，也是本仓库 I3、I5、I6 三条洞见的出发点。

5. 延伸批判

两篇文章合起来提供的是一把尺子而不是一个方法：任何自称"自进化"的机器人系统都可以被问三个问题——它在阶梯的哪一级改东西、它的评估器在不在循环之外、它的收益是"harness 更新"还是"harness 收益"。用这把尺子量本仓库的核心工作：ENPIRE 在 harness code 一级、评估器由人写且只读、收益归因未拆分（Codex/Claude/Kimi 的差距未解释）；Zetta 在 harness code 一级、评估器（critic）自己被演化——这正是 Lil'Log 警告的配置；SkillOpt 在 context 一级、评估器是 held-out 分数且外置——最符合纪律。这把尺子应当成为本仓库评价新工作的默认清单。

41 · RSI 谱系：从 Good 1965 到 BigBang-V1——软件侧递归自我改进的四个环节

覆盖：Good (1965) · Yudkowsky (LessWrong 2008) · Gödel Machine (cs/0309048, 2003) · STaR (2203.14465, NeurIPS 2022) · Reflexion (2303.11366, NeurIPS 2023) · Self-Refine (2303.17651) · Let's Verify Step by Step (2305.20050, ICLR 2024) · Self-Rewarding LMs (2401.10020) · SPIN (2401.01335, ICML 2024) · Meta-Rewarding LMs (2407.19594) · Absolute Zero (2505.03335) · Anchored Self-Play for Code Repair (2607.03523, ICML 2026) · The AI Scientist (2408.06292) · Towards end-to-end automation of AI research (Nature 2026) · ScientistOne (2605.26340) · Autodata (2606.25996) · BigBang-V1 (Endless Frontier 技术报告) · HiSME (2605.28390) · Anthropic 《When AI builds itself》(2026) 所属主线：T24、T0 · 材料来源：具身纪元文章第二节；Lil'Log 2026 文的参考文献

1. 一句话定位

具身纪元文章用两条轴线整理 RSI：改进的环节（部署时自演化 / 训练时自迭代 / 自我评估 / 自动研究）× 人的参与程度。这份合评把软件侧的谱系按第一条轴排好，供机器人侧对照（对照表见报告第 6 章）。概念源头是 Good 1965 的“智能爆炸”设想与 Schmidhuber 2003 的 Gödel Machine（只有能证明修改会提高效率时才执行修改）；Yudkowsky 2008 的 LessWrong 长文是概念在 AI 安全社区的系统论述；Anthropic 2026 年的《When AI builds itself》判断路径是“代码建议 → 编程智能体自改代码 → AI 参与设计与训练后继系统”，并认为具身智能可能紧随，但物理制造、实验周期与部署是新的速度瓶颈。

2. 四个环节的代表

部署时自演化（权重不变）：Reflexion——做完任务后把测试/环境反馈写成反思存进记忆，下次带着经验重试，HumanEval 91%（具身纪元转述）；Self-Refine——同一模型生成 → 自我反馈 → 精炼，不训练；HiSME——从执行轨迹学“怎样生成与修改技能”的元技能并反过来整理技能库，MineDojo 0.700 → 0.856。机器人侧对应：ASPIRE、RoboHarness、Zetta。

训练时自迭代（上一版制造下一版）：STaR——生成推理、答对保留、答错看答案重推、筛出的过程微调下一版；SPIN——与自己的历史版本博弈，弱模型变强，无需额外人类数据；Self-Rewarding——模型用 LLM-as-a-Judge 给自己打分并做 DPO，回答与评价能力同时迭代；Absolute Zero——零数据自博弈，模型自己提出可验证任务并求解；Anchored Self-Play——带锚点的自博弈代码修复避免漂移；BigBang-V1——出题者/批评者/元批评者合成约一万条可验证难题更新权重（证据主要来自团队技术报告）。机器人侧对应：RoboClaw、LWD、Q-Planning。

自我评估（改进 judge / reward model / PRM / verifier）：Let's Verify Step by Step——过程奖励模型逐步标出推理出错位置而非只看最终答案；Meta-Rewarding——同一模型既当回答者、评审者和“评审的评审”，四轮后 AlpacaEval 2 长度控制胜率 22.9% → 39.4%。机器人侧对应：PRIMO R1、VERITAS、LLM-as-a-Verifier。

自动研究（提假设、改算法、跑实验、安排下一轮）：The AI Scientist——从代码模板出发提想法、查新颖性、改代码、跑实验、画图、写论文并接受自动评审；其 Nature 版本《Towards end-to-end automation of AI

research》(Nature 651:914–919, 2026)；ScientistOne——以证据链组织自主研究；Autodata——作为数据科学家的 agent 生成高质量合成数据。机器人侧对应：Eureka → DrEureka → HARBOR → ENPIRE。

3. 口径与局限

- 这些数字 (HumanEval 91%、MineDojo 0.856、AlpacaEval 39.4%) 全部是软件/推理基准，与机器人成功率无可比性；它们的意义是展示每个环节"可以被做"，不是"做得多好"。
- Self-Rewarding 与 Meta-Rewarding 把评估器放进了被改进的对象里，与 Lil'Log/AHE"评估器在循环之外"的原则直接冲突——这是 RSI 内部未解决的张力，机器人侧尚未正面处理。
- BigBang-V1 的证据来自技术报告；Anthropic 的文章是路线判断而非实验。

4. 关系定位与延伸批判

把四个环节叠到讲稿公式 $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$ 上：部署时自演化改 z 里的记忆/技能/harness，训练时自迭代改权重，自我评估改 r 的来源，自动研究改整个流程。具身纪元文章的核心判断——前沿 LLM 更可能先成为 Robot RSI 的认知中枢——在这张表上有一个具体形态：LLM 在四个环节里已经有软件侧的成熟实例，机器人侧缺的是让这些实例的 τ 、 r 、 $\log$ 来自物理世界，而这要求可重复、可扩展的验证环境 (notes/30) 与可靠的验证器 (notes/19)。Gödel Machine 的原则 (可证明有益才修改) 在 2026 年以 VASO 的模型检查 (notes/09) 与 SkillOpt 的严格提升门 (notes/05) 两种弱化形式回到了实践中——前者证明安全性质，后者证明验证集上的提升；两者都还证明不了"真机上会更好"。

42 · Harness 演化方法合评：ACE、MCE、Meta-Harness、Self-Harness、AHE、Harness Updating ≠ Harness Benefit、DGM、Hyperagents、AlphaEvolve、ShinkaEvolve、ThetaEvolve、GEPA、Promptbreeder、STOP、ADAS、AFlow 与相关工作

覆盖：ACE (arXiv 2510.04618, ICLR 2026) · Meta Context Engineering (2601.21557) · Meta-Harness (2603.28052) · Self-Harness (2606.09498) · Agentic Harness Engineering / AHE (2604.25850) · Harness Updating Is Not Harness Benefit (2605.30621) · Continual Harness (2605.09998) · DemoEvolve (2605.24539) · SIA (2605.27276) · Darwin Gödel Machine (2505.22954) · Hyperagents (2603.19461) · AlphaEvolve (2506.13131) · ShinkaEvolve (2509.19349) · ThetaEvolve (2511.23473) · GEPA (2507.19457) · Promptbreeder (2309.16797) · STOP (2310.02304, COLM 2024) · ADAS (2408.08435, ICLR 2025) · AFlow (2410.10762, ICLR 2025) · Learning to Discover at Test Time (2601.16175) · Epistemic Uncertainty for Test-Time Discovery (2605.11328) 所属主线：T0、T15、T24 · 材料来源：Lil'Log 2026 文；报告第 5.2 节

1. 一句话定位

这是 Lil'Log 优化阶梯的完整实例集。**context 一级**：ACE 把上下文当作可演化的 playbook

(Generator/Reflector/Curator 三角色，增量条目式更新避免上下文塌缩)；MCE 做双层优化（外层演化技能即上下文管理机制，内层优化任务上下文）；GEPA 的反思式 prompt 演化优于 RL；Promptbreeder 的自指式 prompt 演化连突变 prompt 本身也被演化。**workflow 一级**：ADAS 用 meta agent 在代码空间搜索 agent 设计；AFlow 用 MCTS 在代码表示的工作流空间生成 agentic workflow。**harness code 一级**：Meta-Harness 用 coding agent 优化 harness 代码并输出 Pareto 前沿；Self-Harness 做 weakness mining → bounded proposal → held-in/held-out 双重回归验证；AHE 以可观测性为核心（组件/经验/决策三层可观测，每次编辑是可证伪的文件级声明）；Continual Harness 在长程游戏中同时更新 harness 与蒸馏策略；DemoEvolve 用人类示范补稀疏反馈；SIA 让 Feedback-Agent 决定本轮更新 harness 还是权重。**optimizer code 一级**：STOP 优化 improver 而非解本身（弱模型下退化）；DGM 让 coding agent 修改自身 harness 代码库并开放式演化（SWE-bench 20% → 50%）；Hyperagents 引入元代理控制如何修改任务代理。**程序演化搜索**：AlphaEvolve 用冻结 LLM 生成程序 diff 的演化搜索（EVOLVE-BLOCK 标记可改区域）；ShinkaEvolve 的开放式样本高效演化（新颖性拒绝采样、多模型集成）；ThetaEvolve 的测试时学习；Learning to Discover / Epistemic Uncertainty 的测试时发现。**警示**：Harness Updating Is Not Harness Benefit——写 harness 的能力从 9B 到 Opus 几乎持平，利用 harness 的能力非单调。

2. 对机器人侧的意义（逐级）

- context 一级 → SkillOpt (notes/05)、Harness VLA 的规则记忆 (notes/20)、HyMeS 的代码记忆 (notes/13)。ACE 的"增量条目式更新避免塌缩"与 SkillGLoW 的"文档塌缩 vs 条目膨胀"诊断 (notes/07) 是同一问题。
- workflow 一级 → ASPIRE 的执行引擎 → 诊断 → 修复 → 验证 (notes/04)、HARBOR 的分阶段 agent (notes/08)。

- harness code 一级 → RHO 的策略仓库 (notes/03)、SHAPER 的 context-code harness (notes/07)、Zetta 的运行时 critic (notes/07)。Self-Harness 的 held-in/held-out 双重回归验证是机器人侧最缺的纪律——目前只有 SkillOpt 类工作有 held-out 门。
- optimizer code 一级 → HiSME 的元技能 (notes/41)、SkillOpt-Lite 的 HarnessOpt；机器人侧还没有"演化演化器"的工作。
- 程序演化搜索 → MEMENTO 的模因演化 (notes/07)、ENPIRE 的多 agent 假设搜索 (notes/06)；ShinkaEvolve 的新颖性拒绝采样是缓解 ENPIRE 假设重复（8 倍机器人 2-3 倍加速）的现成工具（报告开放问题第 5 条）。

3. 口径与局限

- SWE-bench 20% → 50% (DGM) 等数字全部是软件基准；它们证明的是搜索方法有效，不是方法在物理反馈下有效。
- 这些方法的验证器（测试用例、基准分数）廉价且客观；搬到机器人时，验证成本会把"每个 epoch 几十个候选编辑" (SkillOpt) 这类预算压到个位数。
- Harness Updating ≠ Harness Benefit 提示：任何机器人 harness 论文报告的收益都应当附带"换基座模型后收益如何变化"。

4. 延伸批判

这组工作的共同结构——提议编辑、验证、接受/拒绝、保留历史——与遗传算法、贝叶斯优化并无本质区别，新的是提议者 (LLM) 能读懂失败日志并给出有语义的编辑。机器人侧真正需要从这里借的不是搜索算法，而是三条纪律：held-out 验证 (Self-Harness)、可观测性与可证伪声明 (AHE)、只读评估器 (AHE)。目前机器人侧的自进化工作里，只有 SkillOpt 类满足第一条，几乎没有工作明确满足第三条——Zetta 与 MEMENTO 都在演化评估器。这是本仓库对 Robot RSI 现状最重要的批评。

43 · AI 研发基准与失败模式合评：PaperBench、RE-Bench、MLE-bench、ScienceAgentBench、CORE-Bench、KernelBench、Early Science Acceleration 与《Why LLMs Aren't Scientists Yet》

覆盖：PaperBench (arXiv 2504.01848, ICML 2025) · RE-Bench (2411.15114, ICML 2025) · MLE-bench (2410.07095) · ScienceAgentBench (2410.05080, ICLR 2025) · CORE-Bench (2409.11363, TMLR 2024) · KernelBench (2502.10517) · Early science acceleration experiments with GPT-5 (2511.16072) · Why LLMs Aren't Scientists Yet (2601.03315) · Emergent autonomous scientific research capabilities of LLMs (2304.05332) 所属主线：T0、T24 · 材料来源：Lil'Log 2026 文的参考文献 [28]–[35]

1. 一句话定位

自动研究需要度量。这组基准从六个角度衡量 AI 做研发的能力：复现论文（PaperBench 评测 AI 复现 AI 研究论文；CORE-Bench 评测计算可复现性）、对比人类专家做 AI R&D（RE-Bench）、机器学习工程（MLE-bench 的 Kaggle 式任务）、数据驱动的科学发现（ScienceAgentBench）、写高效 GPU kernel

（KernelBench, harness 演化常用的可验证任务）。两篇经验报告给出两端：《Early science acceleration experiments with GPT-5》是前沿模型加速科研的早期正面案例集；Trehan & Chopra 的《Why LLMs Aren't Scientists Yet》从四次自主研究尝试里总结出六类失败模式——训练数据默认偏置、实现漂移、记忆退化、过度乐观、领域知识不足、科学品味弱。Boiko 等 2023 年的工作是 LLM 驾驭实验室自动化（含云实验室）的早期案例。

2. 对 Robot RSI 的意义

机器人侧的自动研究（ENPIRE、HARBOR、Nautilus、AgenticRobotics, notes/06、08）目前没有对应的基准。这组软件侧基准提示了三个可以直接移植的维度：

- **复现**（PaperBench/CORE-Bench）：给 coding agent 一篇机器人论文与开源代码，能否在仿真里复现报告的成功率——Nautilus 的"一句 prompt 到复现 workflow"正是这个任务，但没有基准化。
- **与人类专家对比**（RE-Bench）：ENPIRE 的"pin insertion 收敛到 100% 快于一个前沿 human-in-the-loop 方法"是一个孤立的数据点，缺少标准化的人类专家基线。
- **工程任务**（MLE-bench/KernelBench）：HARBOR 的 6 基准 16 任务是机器人 RL 工程的雏形基准。

Trehan & Chopra 的六类失败模式在机器人侧的形态（报告 I9）："仿真 100% 真机 60%"（过度乐观 + 实现漂移）、多个 agent 试同一想法（多样性塌缩，Lil'Log 的表述）、把噪声当信号宣布成功（过度乐观 + 弱评估器）。

3. 口径与局限

- 这些基准评的是软件 agent；机器人 R&D 的额外成本（真机时间、复位、硬件损耗）在任何一个基准里都没有建模。

- 《Early science acceleration》是案例集而非受控评测。
- 六类失败模式来自四次尝试的定性总结，不是统计结论。

4. 延伸批判

本仓库最缺的一份东西，是一个"Robot Autoresearch Bench"：固定仿真环境与真机接口、固定人类基线、固定验证器，让 ENPIRE 式系统的收敛速度、MRU/MTU、假设多样性、真机迁移差距可以跨论文比较。它的设计可以直接借 RE-Bench 的人类对照协议、CORE-Bench 的复现判定与 KernelBench 的可验证任务形式，再加上 EmbodiedGovBench (notes/25) 的治理维度。在这份基准出现之前，本仓库数字口径账本 (insights/12) 里所有"自动研究"的数字都只能定性定位，不能定量比较。